

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



ĐỒ ÁN TỐT NGHIỆP

**TĂNG TỐC PHẦN CỨNG CHO THUẬT TOÁN
HAMMING QUASI CYCLIC ỨNG DỤNG
TRONG MÃ HÓA HẬU LƯỢNG TỬ**

NGÀNH: KỸ THUẬT MÁY TÍNH
HỘI ĐỒNG: 2L KỸ THUẬT MÁY TÍNH

Người hướng dẫn:

PGS. TS. Phạm Quốc Cường - Trường ĐHBK
CN. Tôn Huỳnh Long - Trường ĐHBK

Người Phản biện:

PGS. TS. Trần Ngọc Thịnh - Trường ĐHBK

Sinh viên thực hiện:

Trịnh Vĩnh Thuận - 2213367

Lê Phú Cường - 2210425

Nguyễn Đặng Thức - 2213428

THÀNH PHỐ HỒ CHÍ MINH - THÁNG 6 NĂM 2026

KHOA: KH&KT Máy tính

BỘ MÔN: **Kỹ thuật Máy tính** *Chủ ý: Sinh viên phải dán tờ này vào trang nháp của bản thuyết trình*

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

HỌ VÀ TÊN: **TRỊNH VINH THUẬN**
NGÀNH: **Kỹ thuật Máy tính**
HỌ VÀ TÊN: **NGUYỄN ĐẶNG THỨC**
NGÀNH: **Kỹ thuật Máy tính**
HỌ VÀ TÊN: **LÊ PHÚ CƯỜNG**
NGÀNH: **Kỹ thuật Máy tính**

MSSV: **2213367**
LỚP: **MT22KT04**
MSSV: **2213428**
LỚP: **MT22KT04**
MSSV: **2210425**
LỚP: **MT22KT01**

1. Đầu đề đồ án:

Tăng tốc phần cứng cho thuật toán Hamming Quasi-Cyclic ứng dụng trong mã hóa hậu lượng tử

2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):

Trong đồ án này, sinh viên cần nghiên cứu thuật toán Hamming Quasi-Cyclic (HQC) – lựa chọn của NIST như một giải pháp thay thế cho CRYSTALS-Kyber (nay là ML-KEM). Dựa trên kết quả nghiên cứu, sinh viên phải đề xuất một phương án triển khai thuật toán cũng như các tối ưu hóa khả thi. Sau đó, sinh viên sẽ hiện thực thiết kế của mình trên thiết bị FPGA và tiến hành kiểm thử tính đúng đắn cũng như hiệu năng của nó. Các công việc chi tiết là:

- Nghiên cứu thuật toán HQC.
- Đề xuất thiết kế và kiến trúc hệ thống.
- Triển khai kiến trúc đã đề xuất bằng Verilog-HDL hoặc HLS.
- Triển khai hệ thống trên bo mạch FPGA có sẵn.
- Viết báo cáo.

3. Ngày giao nhiệm vụ đồ án: 01/01/2026

4. Ngày hoàn thành nhiệm vụ: 01/06/2026

5. Họ tên giảng viên hướng dẫn

Phản hướng dẫn:

1) PGS. TS. Phạm Quốc Cường - 90

2) CN. Tôn Huỳnh Long - 10

Nội dung và yêu cầu ĐATN đã được thông qua Bộ môn.

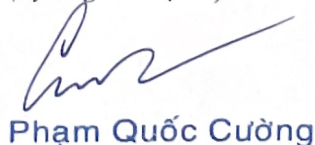
Ngày tháng năm

TRƯỞNG BỘ MÔN

(Ký và ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN CHÍNH

(Ký và ghi rõ họ tên)



Phạm Quốc Cường

PHẦN DÀNH CHO KHOA, BỘ MÔN:

Người duyệt (chấm sơ bộ): _____

Đơn vị: _____

Ngày bảo vệ: _____

Điểm tổng kết: _____

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KH & KT MÁY TÍNH

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

TP. HCM, ngày 18 tháng 6 năm 2026

PHIẾU CHẤM BẢO VỆ ĐỒ ÁN/ LUẬN VĂN TỐT NGHIỆP
(Dành cho người hướng dẫn)

1. Họ và tên SV: **TRỊNH VINH THUẬN**

MSSV: 2213367

Ngành (chuyên ngành): Kỹ thuật Máy tính

Họ và tên SV: **LÊ PHÚ CƯỜNG**

MSSV: 2210425

Ngành (chuyên ngành): Kỹ thuật Máy tính

Họ và tên SV: **NGUYỄN ĐẠNG THỨC**

MSSV: 2213428

Ngành (chuyên ngành): Kỹ thuật Máy tính

2. Đề tài: Tăng tốc phần cứng cho thuật toán Hamming Quasi-Cyclic ứng dụng trong mã hóa hậu lượng tử

3. Họ tên người hướng dẫn: Phạm Quốc Cường

4. Tổng quát về bản thuyết minh:

Số trang:

Số chương:

Số bảng số liệu:

Số hình vẽ:

Số tài liệu tham khảo:

Phần mềm tính toán:

Hiện vật (sản phẩm):

5. Tổng quát về các bản vẽ:

- Số bản vẽ: Bản A1:

Bản A2:

Khổ khác:

- Số bản vẽ vẽ tay:

Số bản vẽ trên máy tính:

6. Những ưu điểm chính của LVTN:

- SV đã thực hiện tốt yêu cầu đặt ra của ĐATN

- Các kết quả hiện thực, đánh giá và so sánh đáng tin cậy và có giá trị cao

- SV hoàn thành trọn vẹn ĐATN

7. Những thiếu sót chính của LVTN:

- Các so sánh với công trình liên quan còn đơn giản, chưa phân tích sâu các nguyên nhân nhanh hơn và chậm hơn của hệ thống

8. Đề nghị: Được bảo vệ ☒

Bổ sung thêm để bảo vệ ☐

Không được bảo vệ ☐

9. 3 câu hỏi SV phải trả lời trước Hội đồng:

Không có

10. Đánh giá chung (bằng chữ: giỏi, khá, TB): Giỏi

Điểm : 9.7 /10

NGƯỜI VIẾT PHIẾU CHẤM

(ký và ghi rõ họ tên)



Phạm Quốc Cường

PHIẾU CHĂM BẢO VỆ ĐỒ ÁN/ LUẬN VĂN TỐT NGHIỆP
(Dành cho người phản biện)

1. Họ và tên SV: TRỊNH VĨNH THUẬN	
MSSV: 2213367	Ngành (chuyên ngành): Kỹ thuật Máy tính
Họ và tên SV: LÊ PHÚ CƯỜNG	
MSSV: 2210425	Ngành (chuyên ngành): Kỹ thuật Máy tính
Họ và tên SV: NGUYỄN ĐẶNG THỨC	
MSSV: 2213428	Ngành (chuyên ngành): Kỹ thuật Máy tính

2. Đề tài: Tăng tốc phần cứng cho thuật toán Hamming Quasi-Cyclic ứng dụng trong mã hóa hậu lượng tử

3. Họ tên người phản biện: Trần Ngọc Thịnh

4. Tổng quát về bản thuyết minh:

Số trang: 121	Số chương: 6
Số bảng số liệu: 23	Số hình vẽ: 43
Số tài liệu tham khảo: 35	Phần mềm tính toán: Vivado
Hiện vật (sản phẩm): Mã nguồn mô phỏng thuật toán HQC phần cứng	

5. Tổng quát về các bản vẽ:

- Số bản vẽ:	Bản A1:	Bản A2:	Khổ khác:
- Số bản vẽ vẽ tay:	Số bản vẽ trên máy tính:		

6. Những ưu điểm chính của LVTN:

- Đồ án nghiên cứu tăng tốc phần cứng cho thuật toán Hamming Quasi-Cyclic (HQC) - một ứng viên mật mã hậu lượng tử (PQC) dựa trên lý thuyết mã sửa sai (code-based cryptography) được NIST đề xuất.
- Thay vì nhân bản các lõi tính toán đắt đỏ, đề tài đề xuất một kiến trúc hợp nhất tích hợp cả ba quy trình: Sinh khóa (KeyGen), Đóng gói khóa (Encap) và Giải gói khóa (Decap). Nhờ cơ chế dồn kênh (Multiplexing Network), hệ thống cho phép chia sẻ các khối tài nguyên tốn kém nhất là bộ nhân đa thức POLY_MUL và lõi băm SHAKE256, giúp giảm tối đa diện tích trên FPGA.
- Kết quả hiện thực phần cứng hoàn chỉnh, xây dựng thành công lõi băm SHAKE256 dựa trên khối hoán vị Keccak-f[1600], các module lấy mẫu vector thừa phức tạp, và lõi nhân đa thức Quasi-Cyclic vòng lặp trực tiếp đều hiện thực ở mức RTL.
- Việc sử dụng kỹ thuật phân mảnh bộ nhớ (memory partitioning) trên chip để tăng băng thông đọc/ghi đồng thời, giải quyết tốt bài toán trễ dữ liệu khi xử lý các đa thức kích thước lớn.

7. Những thiếu sót chính của LVTN:

Hiệu năng số chu kỳ vẫn còn khoảng cách lớn so với hiện thực state-of-the-art Antognazza et al. Việc ưu tiên tối thiểu hóa diện tích logic, số chu kỳ thực thi của hệ thống cao hơn khoảng 2.5 lần so với các thiết kế của Antognazza et al. Có thể việc chưa song song hóa bộ giải mã Reed-Solomon sử dụng thuật toán ePIBMA hay eCSEE là nguyên nhân cốt lõi dẫn đến giới hạn này.

Trễ giao tiếp tại bộ Wrapper: Việc nạp các đa thức và bản mã có kích thước cực lớn thông qua bus điều khiển AXI4-Lite 32-bit của Wrapper tạo ra một trễ hao phí không nhỏ, ảnh

hường đến thông lượng thực tế của hệ thống.

Diện tích bộ nhớ trên chip chưa được tối ưu: Hệ thống tiêu thụ tới 46 khối BRAM, cho thấy chiến lược quản lý bộ nhớ đệm trung gian vẫn còn tương đối cồng kềnh và có thể tiếp tục thu gọn.

8. Đề nghị: Được bảo vệ ☒ Bổ sung thêm để bảo vệ ☐ Không được bảo vệ ☐

9. 3 câu hỏi SV phải trả lời trước Hội đồng:

1. Tại sao kiến trúc HQC IP đề xuất lại lựa chọn mô hình Kiến trúc Hợp nhất (Joint Architecture) thay vì tách biệt các khối KeyGen, Encap, Decap thành các IP độc lập? Sự đánh đổi về mặt tài nguyên và hiệu năng ở đây là gì?
2. Phân tích sự chênh lệch số chu kỳ xung nhịp (khoảng 2.5 lần) giữa thiết kế của nhóm và thiết kế của Antognazza et al. Những cải tiến cụ thể nào (ví dụ ở khối giải mã Reed-Solomon) cần được hiện thực để thu hẹp khoảng cách này?
3. Giao tiếp qua Wrapper hiện tại sử dụng AXI4-Lite 32-bit tạo ra điểm nghẽn khi xử lý liên tục. Làm thế nào để thay thế bằng giao thức AXI4-Stream kết hợp bộ điều khiển DMA?

10. Đánh giá chung (bằng chữ: giỏi, khá, TB): Giỏi

Điểm : 9 /10

NGƯỜI VIẾT PHIẾU CHẤM
(ký và ghi rõ họ tên)



Trần Ngọc Thịnh

This thesis is dedicated for our parents and our instructors at HCMUT.

LỜI CAM ĐOAN

Chúng tôi là nhóm sinh viên thực hiện đồ án tốt nghiệp bao gồm Trịnh Vĩnh Thuận, Lê Phú Cường và Nguyễn Đặng Thúc. Chúng tôi xin cam đoan rằng đồ án mang tên "Tăng tốc phần cứng cho thuật toán Hamming Quasi Cyclic ứng dụng trong mã hóa hậu lượng tử" là công trình nghiên cứu của nhóm và được thực hiện dưới sự hướng dẫn trực tiếp của Phó Giáo sư Tiến sĩ Phạm Quốc Cường và Cử nhân Tôn Huỳnh Long. Trong quá trình thực hiện, nhóm có kế thừa và tham khảo các giải thuật cùng phương pháp hiện thực từ những công trình đã được công bố. Tuy nhiên, toàn bộ mã nguồn RTL Verilog đều do chính chúng tôi tự tay xây dựng, gỡ lỗi và kiểm chứng, đồng thời mọi nguồn tham khảo đều được trích dẫn tường minh trong báo cáo. Toàn bộ số liệu cùng với các kết quả được trình bày trong cuốn đồ án này là hoàn toàn trung thực cũng như chưa từng được công bố tại bất kỳ công trình khoa học nào khác trước đây. Thêm vào đó, tất cả những lý thuyết hay thông tin được trích dẫn từ các tài liệu tham khảo đều được chúng tôi ghi rõ nguồn gốc xuất xứ một cách minh bạch ở phần danh mục cuối quyển. Cuối cùng, chúng tôi xin chịu hoàn toàn mọi trách nhiệm trước Hội đồng bảo vệ cùng nhà trường về tính xác thực của nội dung đồ án này.

Trân trọng,
Nhóm tác giả

Trịnh Vĩnh Thuận
Lê Phú Cường
Nguyễn Đặng Thúc

LỜI CẢM ƠN

Trước hết, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn, PGS.TS. Phạm Quốc Cường, vì sự hỗ trợ tận tình và định hướng quý báu trong suốt quá trình thực hiện đề tài. Sự hướng dẫn của thầy đã đóng vai trò quan trọng trong việc định hình con đường học thuật của chúng tôi, mang đến những góc nhìn sâu sắc, sự động viên liên tục và những lời khuyên chuyên môn giá trị. Nếu không có sự đồng hành và hỗ trợ tận tâm của thầy ở mọi giai đoạn, luận văn này khó có thể được hoàn thành.

Chúng tôi xin gửi lời cảm ơn chân thành đến thầy Tôn Huỳnh Long thuộc phòng CE LAB vì đã nhiệt tình chia sẻ thời gian và chuyên môn. Sự hỗ trợ, trao đổi kiến thức và các buổi thảo luận kỹ thuật của các thầy đã giúp chúng tôi mở rộng hiểu biết và đóng góp đáng kể vào việc hoàn thiện đề tài.

Chúng tôi cũng xin trân trọng cảm ơn quý thầy cô thuộc Khoa Khoa học và Kỹ thuật Máy tính, cùng Trường Đại học Bách khoa – Đại học Quốc gia Thành phố Hồ Chí Minh. Trong suốt bốn năm học tập, sự tận tâm trong giảng dạy của quý thầy cô đã mang lại cho chúng tôi nền tảng kiến thức vững chắc. Sự động viên, hỗ trợ và những ý tưởng quý báu từ quý thầy cô đã góp phần quan trọng vào sự thành công của luận văn.

Cuối cùng nhưng không kém phần quan trọng, chúng tôi xin gửi lời tri ân sâu sắc đến gia đình và bạn bè. Tình yêu thương, sự quan tâm và ủng hộ không ngừng từ gia đình, cùng với sự đồng hành của bạn bè đã giúp chúng tôi vượt qua những khó khăn, giữ vững tinh thần và tiếp tục theo đuổi mục tiêu của mình. Đây chính là nguồn động lực to lớn giúp chúng tôi đạt được cột mốc quan trọng này.

Xin kính chúc quý thầy cô, gia đình và bạn bè luôn dồi dào sức khỏe và thành công trong mọi lĩnh vực của cuộc sống.

Trân trọng,
Nhóm tác giả

Trịnh Vĩnh Thuận
Lê Phú Cường
Nguyễn Đăng Thức

TÓM TẮT

Đề tài nghiên cứu này tập trung vào việc thiết kế và triển khai một bộ tăng tốc phần cứng cho thuật toán Hamming Quasi-Cyclic (HQC) – một lược đồ mật mã hậu lượng tử (Post-Quantum Cryptography – PQC) – trên nền tảng mảng cổng lập trình được (FPGA). Tính cấp thiết của nghiên cứu xuất phát từ mối đe dọa ngày càng gia tăng của máy tính lượng tử đối với các hệ mật mã truyền thống như RSA và ECC, vốn dựa trên độ khó của các bài toán phân tích số nguyên lớn hoặc logarit rời rạc – những bài toán có thể được giải hiệu quả bằng các thuật toán lượng tử. HQC, một cơ chế đóng gói khóa (Key Encapsulation Mechanism – KEM) dựa trên lý thuyết mã (code-based cryptography), được đánh giá cao nhờ tính bảo mật vững chắc và khả năng chống lại cả tấn công cổ điển lẫn lượng tử.

Đề tài khai thác công nghệ FPGA nhằm tăng tốc các phép tính phức tạp trong thuật toán HQC, với mục tiêu cải thiện hiệu năng so với các cài đặt phần mềm truyền thống. Các khối chức năng quan trọng của HQC, bao gồm các phép toán trên đa thức nhị phân, các phép biến đổi tuyến tính và các hàm băm mật mã (như SHAKE256), được thiết kế và tối ưu hóa trên phần cứng nhằm tận dụng khả năng xử lý song song của FPGA.

Thiết kế phần cứng sau khi hoàn thiện được kiểm thử, đánh giá hiệu năng và so sánh với các mô hình trước đó. Các tiêu chí đánh giá bao gồm thời gian thực thi (đo bằng số chu kỳ xung nhịp), mức độ sử dụng tài nguyên phần cứng (phần tử logic, bộ nhớ, khối DSP) và so sánh với các phiên bản phần mềm.

Thiết kế cuối cùng được triển khai và trình diễn trên bo mạch Xilinx Kria KV260 Vision AI Starter Kit, sử dụng nền tảng Zynq UltraScale+ MPSoC. Nền tảng này tích hợp hệ thống xử lý hiệu năng cao (MPSoC) cùng logic lập trình (FPGA) linh hoạt, cho phép xây dựng các hệ thống tăng tốc mật mã hiệu quả và phù hợp với các ứng dụng thực tế.

MỤC LỤC

Lời cam đoan	vii
Lời cảm ơn	ix
Tóm tắt	xi
Danh sách bảng	xvii
Danh sách hình vẽ	xix
1 GIỚI THIỆU	1
1.1 Bối cảnh	1
1.2 Mục tiêu đề tài	2
1.3 Phạm vi và giới hạn của đề tài.	2
1.4 Cấu trúc báo cáo.	3
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN	5
2.1 Tổng quan về mật mã hậu lượng tử (PQC) và thuật toán HQC	5
2.2 Các khái niệm liên quan đến thuật toán HQC	6
2.2.1 Ký hiệu	6
2.2.2 Lý thuyết mã hoá dựa trên Code-based.	7
2.2.3 Các định nghĩa về bảo mật	9
2.2.4 SHA-3 và SHAKE256	10
2.2.5 Lấy mẫu vector	13
2.2.6 Phép nhân đa thức (Polynomial Multiplication)	15
2.2.7 Mã nối Reed-Muller và Reed-Solomon	18
2.3 Chi tiết giải thuật	25
2.3.1 Key Generation	25
2.3.2 Encapsulation.	26
2.3.3 Decapsulation.	28

2.4	Field Programmable Gate Array	30
2.4.1	Look-Up Tables	30
2.4.2	Flip Flop	31
2.4.3	DSP	33
2.4.4	BRAM.	33
2.5	Tổng quan về nền tảng phần cứng Kria KV260	35
2.5.1	Kiến trúc Kria K26 System-on-Module (SOM)	35
2.5.2	Bo mạch đệm (Carrier Card) và Giao tiếp	36
2.6	Nền tảng phát triển PYNQ	37
2.6.1	Tổng quan về PYNQ	37
2.6.2	Kiến trúc Overlay và Cơ chế hoạt động	37
2.6.3	Giao tiếp dữ liệu trong PYNQ.	38
2.7	Các công trình nghiên cứu liên quan	38
2.7.1	Efficient Hardware Implementation of Constant Time Sampling for HQC [1]	38
2.7.2	Towards Automating Crypto HW-Implementation a Case Study of HQC [2]	40
2.7.3	Code-based Cryptography in IoT: A HW/SW Co-Design of HQC [3]	41
2.7.4	Fast and Efficient Hardware Implementation of HQC [4]	42
2.7.5	An Efficient and Unified RTL Accelerator Design for HQC-128, HQC-192, and HQC-256 [5]	44
3	KIẾN TRÚC ĐỀ XUẤT	47
3.1	Kiến trúc hệ thống	47
3.2	Hamming Quasi-Cyclic Wrapper	49
3.2.1	Giao diện giao tiếp	49
3.2.2	Mô tả các tín hiệu.	49
3.2.3	Sơ đồ khối	51
3.3	Hamming Quasi-Cyclic IP	52
3.3.1	Giao diện giao tiếp	52
3.3.2	Mô tả các tín hiệu.	53
3.3.3	Sơ đồ khối	54
4	HIỆN THỰC	59
4.1	Hiện thực hệ thống	59
4.2	Hamming Quasi-Cyclic Wrapper	60
4.2.1	Đồng bộ hóa dữ liệu và phân tích điều khiển	60

4.2.2	Logic điều khiển	62
4.3	Hamming Quasi-Cyclic IP	63
4.3.1	Shake256.	63
4.3.2	Polynomial Multiplication	64
4.3.3	Xor_based_adder	70
4.3.4	Loc_based_adder.	72
4.3.5	Fix Weight Vector Generator	74
4.3.6	Encode	76
4.3.7	Decode	79
4.3.8	Key Generation	82
4.3.9	Encapsulation.	88
4.3.10	Decapsulation.	92
5	ĐÁNH GIÁ	99
5.1	Tiêu chí đánh giá	99
5.2	Kết quả thực nghiệm	100
5.2.1	Môi trường và kịch bản đánh giá.	100
5.2.2	Kiểm chứng tính đúng đắn của thuật toán.	101
5.2.3	Đánh giá tài nguyên phần cứng	103
5.2.4	Đánh giá tần số hoạt động tối đa.	103
5.2.5	Đánh giá thời gian thực thi	104
5.2.6	Đánh giá Throughput, Freq/Ccs và $A \times T$	105
5.2.7	Đánh giá công suất và năng lượng tiêu thụ.	106
5.3	So sánh hiệu năng.	110
5.3.1	Phần mềm	110
5.3.2	Phần cứng	112
6	KẾT LUẬN	117
6.1	Ưu điểm của thiết kế đề xuất	117
6.2	Hạn chế của thiết kế hiện tại	118
6.3	Hướng phát triển trong tương lai	118
	Tài liệu tham khảo	121

DANH SÁCH BẢNG

2.1	Hiện thực của các phép nhân HQC trên $\mathbb{F}_2[x]$	17
2.2	Các mã Reed-Solomon gốc và rút gọn.	19
3.1	Bảng đặc tả tín hiệu giao tiếp của HQC Wrapper	50
3.2	Bảng đặc tả tín hiệu giao tiếp của HQC IP Core	53
4.1	Các dữ liệu tính toán chứa trong khối bộ nhớ	61
4.2	Mô tả thanh ghi điều khiển GPIO	61
4.3	Bảng đặc tả tín hiệu giao tiếp của Module SHAKE256	63
4.4	Bảng đặc tả tín hiệu giao tiếp của Module Poly Mul	67
4.5	Bảng đặc tả tín hiệu giao tiếp của Module xor_based_adder	70
4.6	Bảng đặc tả tín hiệu giao tiếp của Module loc_based_adder	72
4.7	Bảng đặc tả tín hiệu giao tiếp của Module KEYGEN	83
4.8	Bảng đặc tả tín hiệu giao tiếp của Module Encap	91
4.9	Bảng đặc tả tín hiệu giao tiếp của Module Decap	94
5.1	Phân bố độ trễ end-to-end của một chu trình KEM hoàn chỉnh	104
5.2	Số chu kỳ và thời gian tính toán thuần phần cứng	105
5.3	Các chỉ số hiệu năng tổng hợp của thiết kế HQC-128	106
5.4	Phân rã công suất tiêu thụ trên Kria KV260	108
5.5	Năng lượng tiêu thụ trên mỗi phép toán (tại $F_{\max} = 109.82$ MHz)	109
5.6	Đối sánh hiệu năng HQC-128 với các software implementations tiêu biểu (chu kỳ)	111
5.7	Tỷ lệ cải thiện hiệu năng của thiết kế phần cứng so với các software implementation	111
5.8	Phân tích trade-off đa chiều giữa đồ án và Antognazza et al. [5] (HQC-128)	113
5.9	Đối sánh đồ án với Aguilar-Melchor et al. [6] (HQC-128)	114
5.10	Đối sánh đồ án với Schöffel et al. [3] (HQC-128)	115

DANH SÁCH HÌNH VẼ

2.1	Trò chơi an toàn IND-CPA cho hệ mật khóa công khai PKE [7].	9
2.2	Trò chơi an toàn IND-CCA2 của một cơ chế đóng gói khóa (KEM) [7].	10
2.3	Kiến trúc của thuật toán Keccak-1600.	11
2.4	Sơ đồ giải thuật fix weight sampling	14
2.5	D Flip Flop	31
2.6	Chuỗi các sự kiện trong D Flip Flop	32
2.7	Block RAM	34
2.8	Board mạch Kria KV260	35
2.9	Tổng quan đơn giản hóa cao độ về quy trình HLS[2]	40
2.10	Tăng tốc phần cứng cho HQC[3]	42
2.11	Kiến trúc ePIBMA [5]	44
2.12	Kiến trúc eCSEE [5]	45
3.1	Kiến trúc tổng thể của hệ thống	47
3.2	Giao diện giao tiếp của HQC Wrapper	50
3.3	Sơ đồ khối HQC Wrapper Architecture	51
3.4	Giao diện kết nối của HQC IP	52
3.5	Sơ đồ khối của HQC IP	55
4.1	Hiện thực hệ thống trên Xilinx SoC	59
4.2	HQC Wrapper FSM	62
4.3	Giao diện kết nối của module SHAKE256	63
4.4	Sơ đồ khối thiết kế phần cứng của module POLY_MUL . .	65
4.5	Giao diện kết nối của module Poly Mul	66
4.6	Poly Mul FSM	69
4.7	Giao diện kết nối của bộ cộng xor_based_adder	70
4.8	Xor_based_adder FSM	71
4.9	Giao diện kết nối của bộ cộng loc_based_adder	72
4.10	Loc_based_adder FSM	74
4.11	Sơ đồ khối thiết kế phần cứng của Barret Reduction	75
4.12	Quy trình thực hiện Encode	76

4.13	Kiến trúc module RS encode dựa trên LFSR	76
4.14	Kiến trúc phần cứng của Reed Muller Encode	78
4.15	Quy trình thực hiện Decode	79
4.16	Kiến trúc phần cứng BuffterFly Core	80
4.17	Kiến trúc phần cứng của module Reed Solomon decode . .	81
4.18	Sơ đồ khối thiết kế phần cứng của module KEYGEN	82
4.19	Giao diện kết nối của module KEYGEN	86
4.20	Key Generation FSM	87
4.21	Sơ đồ khối thiết kế phần cứng của module ENCAP	89
4.22	Giao diện kết nối của module ENCAP	90
4.23	Sơ đồ khối thiết kế phần cứng của module DECAP	93
4.24	Giao diện kết nối của module DECAP	97
5.1	Tài nguyên phần cứng sử dụng (<i>post-implementation</i>) . . .	103
5.2	Báo cáo công suất từ Vivado Power Analyzer cho thiết kế HQC-128 trên Kria KV260	107

1

GIỚI THIỆU

1.1. BỐI CẢNH

Trong kỷ nguyên số hóa hiện nay, mật mã học khóa công khai (Public Key Cryptography) đã trở thành một "tấm khiên vô hình" bảo vệ hầu hết mọi khía cạnh trong cuộc sống thường nhật của con người. Từ những thao tác vô cùng quen thuộc như gửi một tin nhắn bảo mật, đăng nhập vào mạng lưới nội bộ, cho đến các giao dịch tài chính ngân hàng và thương mại điện tử quy mô lớn, tất cả đều đang phụ thuộc hoàn toàn vào độ an toàn của các hệ mật mã truyền thống như RSA hay Elliptic Curve Cryptography (ECC) [8]. Nền tảng bảo mật của các hệ thống này được xây dựng trên niềm tin vững chắc vào tính bất khả thi của các bài toán toán học cực kỳ phức tạp đối với máy tính cổ điển, điển hình như bài toán phân tích thừa số nguyên tố hay logarit rời rạc.

Tuy nhiên, sự phát triển vượt bậc của máy tính lượng tử trong những năm gần đây đã đặt ra mối đe dọa hiện hữu đối với các hệ thống mật mã khóa công khai hiện tại như RSA, ECC hay Diffie-Hellman. Sử dụng thuật toán Shor, một máy tính lượng tử đủ lớn có thể dễ dàng giải quyết các bài toán phân tích thừa số nguyên tố và logarit rời rạc, làm sụp đổ nền tảng bảo mật của các giao thức truyền thông ngày nay. Để đối phó với thách thức này, Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) đã khởi xướng quá trình tiêu chuẩn hóa Mật mã học Hậu lượng tử (Post-Quantum Cryptography - PQC) [9]. Trong số các thuật toán hậu lượng tử được đề xuất, các hệ mã dựa trên lý thuyết mã hóa (Code-based cryptography) [10] nổi bật nhờ nền tảng toán học lâu đời và độ an toàn đã được kiểm chứng qua nhiều thập kỷ. Thuật toán Hamming Quasi-Cyclic (HQC) [11]

1

là một trong những ứng cử viên sáng giá thuộc nhóm này, dựa trên bài toán giải mã khó (Decoding Problem) của mã tuyến tính. HQC không chỉ cung cấp độ an toàn cao dựa trên lý thuyết mã hóa mà còn có cấu trúc đại số rõ ràng.

Mặc dù HQC mang lại mức độ an toàn lý thuyết vững chắc, độ phức tạp tính toán trong các pha sinh khóa (Key Generation), đóng gói khóa (Encapsulation) và giải mã (Decapsulation) vẫn là một thách thức lớn khi triển khai trên các thiết bị hạn chế tài nguyên hoặc yêu cầu thông lượng cao. Do đó, việc tăng tốc phần cứng (Hardware Acceleration) cho HQC trên các nền tảng như FPGA (Field Programmable Gate Array) là một bước đi thiết yếu để đưa thuật toán này vào các ứng dụng thực tế.

1.2. MỤC TIÊU ĐỀ TÀI

Mục tiêu chính của đề tài là nghiên cứu và hiện thực thuật toán HQC (Hamming Quasi-Cyclic) trên nền tảng FPGA nhằm đánh giá hiệu năng và khả năng ứng dụng thực tế. Đề tài không đi sâu vào giải thích lý thuyết mật mã mà tập trung vào cách hiện thực thuật toán trên phần cứng. Cụ thể, đề tài tập trung vào các mục tiêu sau:

- **Nghiên cứu lý thuyết:** Tìm hiểu cơ sở toán học và nguyên lý hoạt động của HQC, bao gồm các bước sinh khóa (KeyGen), mã hóa (Encapsulation) và giải mã (Decapsulation).
- **Thiết kế và hiện thực phần cứng:** Xây dựng kiến trúc HQC bằng ngôn ngữ Verilog với các module chức năng chính như nhân đa thức modulo, sinh số ngẫu nhiên và các bộ cộng sau đó tích hợp thành khối top-level hoạt động hoàn chỉnh.
- **Mô phỏng và kiểm chứng trên board mạch:** Thực hiện mô phỏng kiểm tra chức năng để đảm bảo thiết kế đáp ứng đúng yêu cầu thuật toán.
- **Đánh giá hiệu năng:** Triển khai trên phần cứng các nhân tăng tốc để đo đạc các chỉ số hiệu năng và so sánh với các phiên bản phần mềm của thuật toán.

1.3. PHẠM VI VÀ GIỚI HẠN CỦA ĐỀ TÀI

Đề tài tập trung nghiên cứu và triển khai thuật toán HQC (Hamming Quasi-Cyclic) trên nền tảng FPGA bằng ngôn ngữ Verilog, trong phạm vi cơ chế bao gói khóa (Key Encapsulation Mechanism - KEM). Việc hiện

thực được giới hạn ở mức thiết kế các khối chức năng cốt lõi của HQC theo hướng kiến trúc tối ưu tài nguyên (resource-optimized) và chia sẻ phần cứng, tiến hành mô phỏng và kiểm chứng hoạt động. Sau đó, đề án thực hiện tổng hợp và đánh giá hiệu năng phần cứng dựa trên các tiêu chí cốt lõi như tốc độ xử lý, mức độ sử dụng tài nguyên (Area/Resource utilization) và công suất tiêu thụ.

Đề tài không mở rộng sang các thuật toán hậu lượng tử khác, không triển khai toàn bộ hệ thống truyền thông bảo mật hoàn chỉnh, và chưa tích hợp với các thuật toán mã hóa đối xứng để mã hóa dữ liệu thực tế. Bên cạnh đó, việc thực nghiệm chỉ tiến hành trên một dòng FPGA cụ thể, do đó kết quả có thể khác biệt khi áp dụng trên các nền tảng phần cứng khác. Ngoài ra, nghiên cứu chỉ tập trung vào khía cạnh hiệu năng phần cứng, chưa đi sâu vào chứng minh lý thuyết về mức độ an toàn mật mã của HQC.

1.4. CẤU TRÚC BÁO CÁO

Báo cáo được tổ chức thành sáu chương chính như sau:

- **Chương 1: Giới thiệu đề tài**

Trình bày mục tiêu, lý do lựa chọn, phạm vi nghiên cứu và cấu trúc tổng thể của báo cáo. Chương này nhằm giúp người đọc nắm được định hướng nghiên cứu và những giới hạn của đề tài.

- **Chương 2: Cơ sở lý thuyết**

Trình bày các khái niệm nền tảng liên quan đến thuật toán HQC và mã hóa hậu lượng tử, các vấn đề kỹ thuật liên quan đến đề tài, cũng như tổng quan các công trình nghiên cứu trước đó.

- **Chương 3: Thiết kế hệ thống và kiến trúc**

Mô tả kiến trúc tổng quan của hệ thống System on Chip (SoC), bao gồm CPU, bộ nhớ, các nhân tăng tốc, bus. Đồng thời đi sâu vào kiến trúc từng khối thành phần.

- **Chương 4: Hiện thực**

Trình bày quá trình hiện thực hóa hệ thống trên FPGA, ngôn ngữ lập trình phần cứng sử dụng (Verilog), đi sâu chi tiết vào cách thức thực hiện hệ thống bao gồm các Nhân tăng tốc (Hardware-Accelerators), Wrapper và giao tiếp với hệ thống Bus.

- **Chương 5: Đánh giá**

Kiểm chứng tính đúng đắn của hệ thống trên nền tảng thực nghiệm

1

và phân tích chi tiết các chỉ số hiệu năng phần cứng. Cuối cùng, tiến hành đối sánh hiệu năng của thiết kế với các công trình phần mềm và phần cứng khác.

- **Chương 6: Kết luận**

Tóm tắt các kết quả đạt được, đánh giá những hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

2

CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN

2.1. TỔNG QUAN VỀ MẬT MÃ HẬU LƯỢNG TỬ (PQC) VÀ THUẬT TOÁN HQC

Sự phát triển nhanh chóng của máy tính lượng tử đặt ra mối đe dọa trực tiếp đối với các hệ mật mã khoá công khai truyền thống vốn dựa trên bài toán phân tích số và logarit rời rạc. Với thuật toán Shor, các cơ chế như RSA và ECC có thể bị phá vỡ trong thời gian đa thức [12], khiến các phương pháp bảo mật hiện tại trở nên không còn an toàn trong bối cảnh hậu lượng tử. Mật mã hậu lượng tử (Post-Quantum Cryptography – PQC) hướng tới việc xây dựng những thuật toán mới vẫn đảm bảo an toàn ngay cả khi kẻ tấn công sở hữu khả năng tính toán lượng tử, đồng thời vẫn duy trì hiệu năng chấp nhận được trên các nền tảng tính toán cổ điển như CPU, vi điều khiển hoặc FPGA.

Trong các nhánh nghiên cứu về PQC – bao gồm mật mã dựa trên mạng tinh (lattice-based), đa biến (multivariate), isogeny, và mật mã dựa trên mã hoá (code-based) – nhóm code-based đặc biệt nổi bật bởi nền tảng lý thuyết vững chắc và khả năng chống phân tích đã được kiểm nghiệm qua nhiều thập kỷ. HQC (Hamming Quasi-Cyclic) là một cơ chế đóng gói khoá (Key Encapsulation Mechanism – KEM) dựa trên code-based [10], đã được NIST lựa chọn tiêu chuẩn hoá trong chương trình PQC. HQC khai thác độ khó của bài toán giải mã mã tuyến tính trong metric Hamming – một bài toán đã được chứng minh là NP-hard [13].

2

Nhờ cấu trúc đại số đơn giản (chủ yếu là các phép xử lý vector nhị phân và toán học đại số tuyến tính), HQC có tính tương thích cao đối với triển khai phần cứng, đặc biệt trên FPGA.

Thuật toán HQC bao gồm ba giai đoạn chính: Sinh khoá (KeyGen), Bao gói khoá (Encapsulation), và Giải bao gói (Decapsulation). Trong quá trình encapsulation, một thông điệp ngẫu nhiên được mã hoá bằng khoá công khai, sau đó được băm để sinh khoá phiên dùng chung. Ciphertext và khoá phiên thu được có thể được sử dụng để thiết lập kênh liên lạc bảo mật. Với đặc tính cấu trúc tuyến tính và thao tác đơn giản trên bit, HQC là ứng viên phù hợp cho các hệ thống PQC hiệu năng cao, có thể tăng tốc bằng phần cứng.

2.2. CÁC KHÁI NIỆM LIÊN QUAN ĐẾN THUẬT TOÁN HQC

2.2.1. KÝ HIỆU

Các ký hiệu trong bài viết này được tham khảo từ [7]. Cụ thể như sau:

- Ký hiệu $\mathbb{Z}$ là tập các số nguyên.
- $\mathbb{F}_2$ là trường nhị phân,
- $\mathcal{B}$ là tập $\{0, \dots, 255\}$ của các số nguyên 8-bit không dấu.
- Với một tập S , ký hiệu $s \xleftarrow{\$} S$ nghĩa là s được chọn ngẫu nhiên đều từ S .
- Ký hiệu $\mathcal{V}$ là không gian vector có kích thước n trên $\mathbb{F}_2$ với $n \in \mathbb{Z}^+$. Các phần tử của $\mathcal{V}$ có thể được xem như vectơ hàng hoặc vectơ cột, hoặc như đa thức trong vành $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$. Các vector (và đa thức tương ứng) được biểu diễn bằng chữ thường đậm, trong khi ma trận được biểu diễn bằng chữ hoa đậm.
- Một số nguyên nguyên tố n được gọi là *nguyên tố sơ cấp* (primitive) nếu đa thức $(X^n - 1)/(X - 1)$ là bất khả quy trong $\mathcal{R}$. Với $\mathbf{u}, \mathbf{v} \in \mathcal{V}$, ta định nghĩa tích của chúng trong $\mathcal{R}$, ký hiệu $\mathbf{w} = \mathbf{uv} \in \mathcal{V}$ như sau:

$$w_k = \sum_{i+j \equiv k \pmod{n}} u_i \cdot v_j, \quad \forall k \in \{0, 1, \dots, n-1\}.$$

- Với một vector $\mathbf{v} \in \mathbb{F}_2^n$, ký hiệu $\text{rot}(\mathbf{v})$ là ma trận tuần hoàn được sinh

ra bởi $\mathbf{v}$, cụ thể:

$$\text{rot}(\mathbf{v}) = \begin{pmatrix} v_0 & v_{n-1} & \dots & v_1 \\ v_1 & v_0 & \dots & v_2 \\ \vdots & \vdots & \ddots & \vdots \\ v_{n-1} & v_{n-2} & \dots & v_0 \end{pmatrix} \in \mathbb{F}_2^{n \times n}.$$

2

- Dễ thấy rằng tích của hai phần tử bất kỳ $\mathbf{u}, \mathbf{v} \in R$ có thể được biểu diễn như một phép nhân vectơ-ma trận (hoặc ma trận-vectơ) thông qua phép toán $\text{rot}(\cdot)$ như sau:

$$\mathbf{u} \cdot \mathbf{v} = \mathbf{u} \times \text{rot}(\mathbf{v})^\top = (\text{rot}(\mathbf{u}) \times \mathbf{v}^\top)^\top = \mathbf{v} \times \text{rot}(\mathbf{u})^\top = \mathbf{v} \cdot \mathbf{u}.$$

- Ký hiệu $\omega(\cdot)$ là trọng số Hamming của một vector, tức là số lượng các phần tử khác 0 của nó. Với một số nguyên dương ω , ta định nghĩa:

$$R_\omega = \{\mathbf{v} \in R \mid \omega(\mathbf{v}) = \omega\},$$

là tập các vector có trọng số Hamming bằng ω .

- Cho $\mathbf{v} = (v_0, \dots, v_{n-1}) \in \{0, 1\}^n$ với chỉ số 0 là bit ít quan trọng nhất. Với $0 \leq n' \leq n$, ta định nghĩa hàm $\text{Truncate}(\mathbf{v}, n')$ là hàm loại bỏ $n - n'$ bit có trọng số cao nhất của $\mathbf{v}$ và giữ lại n' bit có trọng số thấp nhất.

2.2.2. LÝ THUYẾT MÃ HOÁ DỰA TRÊN CODE-BASED

Trong phần này, chúng tôi sẽ cung cấp một số định nghĩa và tính chất cơ bản trong lý thuyết mã hóa theo [14], HQC thuộc nhóm mật mã hậu lượng tử dựa trên mã tuyến tính. Ý tưởng chính là mã hóa được xây dựng dựa trên bài toán “Syndrome Decoding Problem”, vốn đã được chứng minh là NP-khó. Code-based crypto có ưu điểm nổi bật: độ an toàn cao kể cả khi xuất hiện máy tính lượng tử, do không dựa trên các giả định như phân tích số nguyên lớn hay logarit rời rạc.

Định nghĩa 2.2.2.1 (Mã tuyến tính). *Mã tuyến tính $\mathcal{C}$ (linear code) có độ dài n và k chiều (được kí hiệu $[n, k]$) là một không gian con thuộc $\mathcal{R}$ có k chiều. Các phần tử thuộc $\mathcal{C}$ được gọi là từ mã (codewords).*

Định nghĩa 2.2.2.2 (Ma trận sinh). *Ta nói rằng $G \in \mathbb{F}_2^{k \times n}$ là một ma trận sinh cho mã $\mathcal{C} [n, k]$ nếu*

$$\mathcal{C} = \{mG, \text{ for } m \in \mathbb{F}_2^k\}. \quad (4)$$

Định nghĩa 2.2.2.3 (Ma trận kiểm tra chẵn lẻ). Cho một mã $\mathcal{C}[n, k]$, ta nói rằng $H \in \mathbb{F}_2^{(n-k) \times n}$ là một Ma trận kiểm tra chẵn lẻ cho $\mathcal{C}$ nếu H là một ma trận sinh của mã đối ngẫu $\mathcal{C}^\perp$. Cụ thể hơn

2

$$\mathcal{C} = \{v \in \mathbb{F}_2^n \mid Hv^\top = 0\}, \quad \text{tường đường } \mathcal{C}^\perp = \{uH, \text{ for } u \in \mathbb{F}_2^{n-k}\}. \quad (5)$$

Định nghĩa 2.2.2.4 (Syndrome). Cho $H \in \mathbb{F}_2^{(n-k) \times n}$ là một ma trận kiểm tra chẵn lẻ của mã $\mathcal{C}[n, k]$, và $v \in \mathbb{F}_2^n$ là một từ. Vậy syndrome của v là $Hv^\top$ (hay vector hội chứng của v), ký hiệu là $S(v)$, và khi đó $v \in \mathcal{C} \Leftrightarrow Hv^\top = 0$.

Định nghĩa 2.2.2.5 (Khoảng cách tối thiểu). Cho mã tuyến tính $\mathcal{C}[n, k]$ trên $\mathcal{R}$ và cho ω là một chuẩn trên $\mathcal{R}$. Khoảng cách tối thiểu của $\mathcal{C}$ được định nghĩa là

$$d = \min_{u, v \in \mathcal{C}, u \neq v} \omega(u - v). \quad (6)$$

Một mã với khoảng cách tối thiểu d có khả năng giải mã các mẫu lỗi bất kỳ lên tới $\delta = \lfloor \frac{d-1}{2} \rfloor$. Các tham số mã được kí hiệu là $[n, k, d]$. Mật mã dựa trên mã (code-based cryptography) thường gặp vấn đề với kích thước khóa rất lớn. Để giữ cho hệ mật mã hiệu quả, chúng ta sử dụng chiến lược của Gaborit nhằm rút ngắn khóa. Điều này dẫn đến mã Quasi-Cyclic (QC), được định nghĩa như sau.

Định nghĩa 2.2.2.6 (Quasi-Cyclic Codes [15]). Xét một vector $\mathbf{c} = (\mathbf{c}_0, \dots, \mathbf{c}_{s-1})$ thuộc $\mathbb{F}_2^{sn}$ như là các khối liên tiếp (n -tuples). Một $[sn, k, d]$ mã tuyến tính $\mathcal{C}$ gọi là Quasi-Cyclic (QC) với chỉ số s nếu, với mọi $\mathbf{c} = (\mathbf{c}_0, \dots, \mathbf{c}_{s-1}) \in \mathcal{C}$, vector thu được sau khi áp dụng dịch vòng đồng thời từng khối $\mathbf{c}_0, \dots, \mathbf{c}_{s-1}$ cũng là một từ mã.

Một cách chính xác hơn, bằng cách xem mỗi khối $\mathbf{c}_i$ như một đa thức trong $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$, mã $\mathcal{C}$ là QC với chỉ số s nếu với mọi $\mathbf{c} = (\mathbf{c}_0, \dots, \mathbf{c}_{s-1}) \in \mathcal{C}$ ta có $(X \cdot \mathbf{c}_0, \dots, X \cdot \mathbf{c}_{s-1}) \in \mathcal{C}$.

Định nghĩa 2.2.2.7 (Mã Quasi-Cyclic hệ thống). Một mã Quasi-Cyclic hệ thống $[sn, n]$ với chỉ số s và tỉ lệ $1/s$ là một mã quasi-cyclic với một ma trận kiểm tra chẵn lẻ kích thước $(s-1)n \times sn$ có dạng

$$H = \begin{bmatrix} I_n & 0 & \cdots & 0 & A_0 \\ 0 & I_n & \cdots & 0 & A_1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & \cdots & \cdots & I_n & A_{s-2} \end{bmatrix}$$

trong đó $A_0, \dots, A_{s-2}$ là ma trận circulant $n \times n$.

Định nghĩa về mã quasi-cyclic hệ thống với chỉ số s có thể được mở rộng cho mọi tỉ lệ ℓ/s , $\ell = 1, \dots, s-1$. Tuy nhiên, ở đây chúng ta chỉ sử dụng các mã QC hệ thống với tỉ lệ $1/2$ và $1/3$, và sử dụng ký hiệu rút gọn như định nghĩa trên. Trong các phần tiếp theo, khi nhắc đến mã QC hệ thống, ngầm hiểu rằng tỉ lệ mặc định là $1/s$. Lưu ý rằng không phải mọi mã QC đều tương đương với mã QC hệ thống.

2

2.2.3. CÁC ĐỊNH NGHĨA VỀ BẢO MẬT

Trong phần này, chúng tôi giới thiệu được tham khảo từ [7] các định nghĩa về bảo mật được sử dụng để phân tích bảo mật của HQC-PKE và HQC-KEM.

Định nghĩa 2.2.3.1. Cho $PKE = (\text{Keygen}, \text{Encrypt}, \text{Decrypt})$ là một hệ mã hóa khóa công khai. Trò chơi $PKE.\text{IND-CPA}$ được định nghĩa như trong Hình 2.1, và lợi thế IND-CPA của một đối thủ $\mathcal{A} = (\mathcal{A}_{\text{choose}}, \mathcal{A}_{\text{guess}})$ chống lại PKE được định nghĩa là:

$$\text{Adv}_{PKE}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr[PKE.\text{IND-CPA}(\mathcal{A}) = 1] - \frac{1}{2} \right|.$$

Exp $PKE.\text{IND-CPA}(\mathcal{A})$

1. $(ek_{PKE}, dk_{PKE}) \leftarrow \text{Keygen}()$
2. $(m_0, m_1) \leftarrow \mathcal{A}_{\text{Choose}}(ek_{PKE})$
3. $b \xleftarrow{\$} \{0, 1\}$
4. $\theta \xleftarrow{\$} \mathcal{B}^{|\theta|}$, $c_{PKE} \leftarrow \text{Encrypt}(ek_{PKE}, m_b, \theta)$
5. $b' \leftarrow \mathcal{A}_{\text{Guess}}(ek_{PKE}, c_{PKE})$
6. **return** $(b = b')$

Hình 2.1: Trò chơi an toàn IND-CPA cho hệ mã khóa công khai PKE [7].

Định nghĩa 2.2.3.2. Cho $KEM = (\text{Keygen}, \text{Encaps}, \text{Decaps})$ là một cơ chế đóng gói khóa (key encapsulation mechanism). Trò chơi $KEM.\text{IND-CCA2}$ được định nghĩa như trong Hình 2.2, và lợi thế IND-CCA2 của một kẻ tấn công $\mathcal{A} = (\mathcal{A}_{\text{Choose}}, \mathcal{A}_{\text{Guess}})$ đối với KEM được định nghĩa như sau:

$$\text{Adv}_{KEM}^{\text{IND-CCA2}}(\mathcal{A}) = \left| \Pr[KEM.\text{IND-CCA2}(\mathcal{A}) \Rightarrow 1] - \frac{1}{2} \right|.$$

Exp KEM.IND-CCA2($\mathcal{A}$)

1. $(ek_{\text{KEM}}, dk_{\text{KEM}}) \leftarrow \text{Keygen}()$
2. $b \xleftarrow{\$} \{0, 1\}$
3. $(K_0, c_{\text{KEM}}) \leftarrow \text{Encaps}(ek_{\text{KEM}})$
4. $K_1 \xleftarrow{\$} \mathcal{B}^{|K|}$
5. $b' \leftarrow \mathcal{A}_{\text{Guess}}^{\text{Decaps}()}(K_b, c_{\text{KEM}})$
6. **return** $(b = b')$

Hình 2.2: Trò chơi an toàn IND-CCA2 của một cơ chế đóng gói khóa (KEM) [7].

2.2.4. SHA-3 và SHAKE256

Tổng quan về SHA-3 và Keccak-1600

Keccak-1600 là hàm hoán vị mật mã dựa trên mô hình bọt biển, đóng vai trò là lõi cho họ hàm băm SHA-3 [16; 17]. Nó đã được chọn là hàm băm chiến thắng trong cuộc thi hàm băm của NIST năm 2012 và được chuẩn hóa thành FIPS 202 [16]. Keccak hoạt động ở trạng thái 1600 bit và hoạt động dựa trên dựa trên sự hoán vị nhằm để hỗ trợ đầu ra có nhiều kích thước khác nhau, khiến nó trở nên rất linh hoạt cho các ứng dụng mật mã đa dạng.

SHA-3 là viết tắt của Secure Hash Algorithm 3, một tập hợp các hàm băm mật mã được ban hành bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) [16]. Nó được phát triển từ một cuộc thi thiết kế và được công bố dưới dạng tiêu chuẩn vào năm 2015, với cấu trúc bên trong khác biệt so với các phiên bản trước như SHA-1 và SHA-2. SHA-3 dựa trên cấu trúc bọt biển tạo ra các hàm băm với kích thước đầu ra cố định (SHA3-224, SHA3-256, SHA3-384, SHA3-512) và các hàm có đầu ra có thể mở rộng (SHAKE128, SHAKE256).

Keccak thuộc nhóm các hàm bọt biển với phép hoán vị Keccak-p[b, 12+2 ℓ] với quy tắc đệm pad10*1 [17; 18]. Nhóm này được tham số hóa bởi bất kỳ lựa chọn nào của tỷ lệ r và dung lượng c sao cho $r + c \in \{25, 50, 100, 200, 400, 800, 1600\}$, nghĩa là khi giới hạn trong trường hợp $b = 1600$, họ Keccak được ký hiệu là Keccak[c]; trong trường hợp này, r được xác định bởi lựa chọn c.

$$\text{Keccak}[c] = \text{Sponge}[\text{Keccak} - p[1600, 24], \text{pad10}^*1, 1600 - c] \quad (2.1)$$

Vì vậy, với một chuỗi bit đầu vào N và chiều dài đầu ra d, ta có:

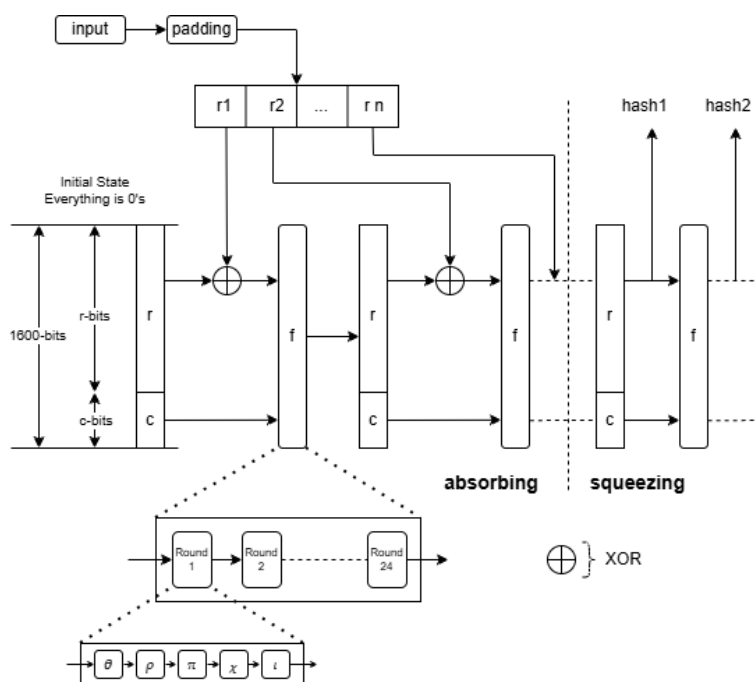
$$\text{Keccak}[c](N, d) = \text{Sponge}[\text{Keccak} - p[1600, 24], \text{pad10}^*1, 1600 - c](N, d). \quad (2.2)$$

Cấu trúc bọt biển Keccak-1600

Keccak-1600 gồm 2 giai đoạn chính:

- Absorb (hấp thụ): Thuật toán sẽ chia input thành các block với mỗi block có kích thước 1088 bit. Nếu input không đủ kích thước là bội của 1088 thì sẽ áp dụng kĩ thuật padding cho input. Sau đó cho mỗi block vào các state của nó.
- Squeeze (vắt): Sau khi xử lý tất cả các khối, hàm Keccak sẽ đưa ra số bit mong muốn.

2



Hình 2.3: Kiến trúc của thuật toán Keccak-1600.

Trong quá trình hoán vị, thuật toán Keccak thực hiện năm phép toán trong mỗi vòng như mô tả trong hình bên trên. Cụ thể:

1. Thuật toán Theta (θ):

$$C[x] = \bigoplus_{y=0}^4 A[x, y] \quad \text{với } x \in \{0, \dots, 4\} \quad (2.3)$$

$$D[x] = C[(x-1) \bmod 5] \oplus \text{ROT}(C[(x+1) \bmod 5], 1) \quad (2.4)$$

$$A'[x, y] = A[x, y] \oplus D[x] \quad \text{với mọi } (x, y) \quad (2.5)$$

2. Thuật toán Rho (ρ):

$$A'[x, y] = \text{ROT}(A[x, y], \rho(x, y)) \quad (2.6)$$

3. Thuật toán Pi (π):

$$A'[x, y] = A[(x+3y) \bmod 5, x] \quad (2.7)$$

4. Thuật toán Chi (χ):

$$A'[x, y] = A[x, y] \oplus ((\neg A[(x+1) \bmod 5, y]) \wedge A[(x+2) \bmod 5, y]) \quad (2.8)$$

5. Thuật toán Iota (ι):

$$A'[0, 0] = A[0, 0] \oplus RC[r] \quad (2.9)$$

$B[x, y]$, $C[x]$ và $D[x]$ là các biến trung gian. $\text{ROT}(W, r)$ là phép dịch chuyển vòng bit, di chuyển bit ở vị trí i đến vị trí $i+r$ (modulo 64). Các hằng số $RC[i]$ và $r[x, y]$ lần lượt là độ lệch dịch chuyển vòng và hằng số làm tròn.

Hàm băm SHAKE256

SHA-3 bắt nguồn từ hàm hoán vị Keccak, kí hiệu là Keccak-p, được định nghĩa bằng 2 tham số:

- Độ rộng hay kích thước của trạng thái bên trong nó.
- Số lượng vòng trong quá trình hoán vị.

Vì vậy, ta có:

$$\text{Keccak-p}[\text{width}, \text{rounds}]$$

Keccak-f là trường hợp đặc biệt của Keccak-p với rounds = 24. Do đó:

$$\text{Keccak-f}[\text{width}] = \text{Keccak-p}[\text{width}, 24]$$

SHA-3 và tất cả các thuật toán thuộc họ này đều sử dụng Keccak-f[1600], hay nói cách khác Keccak-p sẽ có trạng thái bên trong là 1600 bit và 24 rounds. Keccak-f[1600] còn được kí hiệu là Keccak[c]. Do đó:

$$\text{Keccak}[c](N, d)$$

2

trong đó, c là dung lượng (capacity), N là đầu vào và d là độ dài của đầu ra theo bit.

Hai hàm có thể mở rộng đầu ra SHAKE128 và SHAKE256 được định nghĩa dựa trên hàm Keccak[c] sao cho một hậu tố bốn bit được nối vào M cho bất kỳ độ dài đầu ra d nào. Các định nghĩa theo [16] như sau:

$$\text{SHAKE128}(M, d) = \text{Keccak}[256](M \parallel 1111, d);$$

$$\text{SHAKE256}(M, d) = \text{Keccak}[512](M \parallel 1111, d).$$

Trong đó M là thông điệp đầu vào, d là độ dài của chuỗi bit đầu ra. Hậu tố bốn bit 1111 này cho phép *domain separation* bằng cách phân biệt đầu vào cho các hàm SHAKE với các miền khác, đảm bảo tách biệt rõ ràng giữa việc sử dụng SHAKE và các ứng dụng mật mã khác.

Trong HQC, SHAKE256 [7] được sử dụng với nhiều mục đích:

- **Hàm sinh giả ngẫu nhiên PRNG** được sử dụng cho việc tạo vector trọng số cố định và tạo vector ngẫu nhiên trong việc sinh khóa (Keygen). Tạo vector trọng số cố định trong việc mã hóa (Encrypt)
- **Hàm băm** được sử dụng để băm trong đóng gói (Encapsulation) và mở gói (Decapsulation).

2.2.5. LẤY MẪU VECTOR

Trong mật mã HQC, tính an toàn của hệ thống phụ thuộc vào việc sinh ra các đa thức ngẫu nhiên chất lượng cao. Quá trình này được chia làm hai loại: lấy mẫu ngẫu nhiên đều (cho khóa công khai h) và lấy mẫu trọng số cố định (cho khóa bí mật x, y và các vector lỗi r_1, r_2, e). Theo [7]:

Hàm mở rộng đầu ra (XOF)

Mọi quy trình lấy mẫu trong HQC đều bắt đầu từ một hạt giống (seed) ngẫu nhiên. HQC sử dụng hàm băm SHAKE256 (một biến thể của họ SHA-3) làm hàm mở rộng đầu ra (XOF - Extensible Output Function).

- **Đầu vào:** Một hạt giống seed (thường là 40 byte).

2

- **Đầu ra:** Một chuỗi bit ngẫu nhiên giả (pseudo-random) có độ dài tùy ý. Chuỗi bit này đóng vai trò là nguồn entropy để tạo ra các vector.

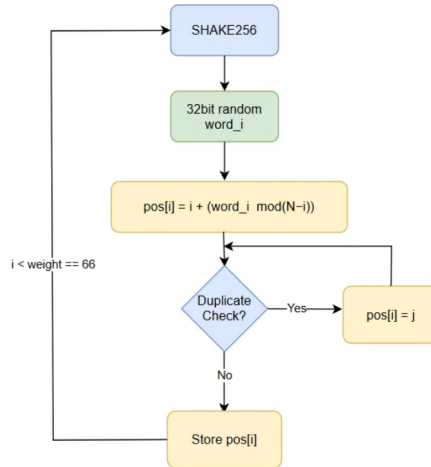
Lấy mẫu ngẫu nhiên đều (Uniform Sampling)

Việc lấy mẫu một đa thức $h \in \mathcal{R}$ có phân phối đều khá đơn giản.

- Hệ thống sử dụng SHAKE256 để sinh ra một chuỗi bit có độ dài N .
- Các bit này được gán trực tiếp làm hệ số cho đa thức h . Trong trường hợp N không chia hết cho 8, các bit dư thừa ở byte cuối cùng sẽ bị loại bỏ để đảm bảo độ dài chính xác.

Lấy mẫu trọng số cố định (Fix Weight Sampling)

Hình 2.4 là kỹ thuật phức tạp hơn, dùng để sinh ra các vector có chính xác w bit 1 (trọng số Hamming là w) tại các vị trí ngẫu nhiên. **Quy trình**



Hình 2.4: Sơ đồ giải thuật fix weight sampling

thực hiện:

- **Sinh dữ liệu:** Sử dụng SHAKE256 để sinh ra dòng 32 bit (word) ngẫu nhiên.
- **Tính toán vị trí bit:** $\text{pos}[i] = i + (\text{word} \bmod (n-1))$
- **Kiểm tra điều kiện (Duplicate check):**
 - *Điều kiện 1 (Valid):* Chỉ số phải nằm trong khoảng hợp lệ ($0 < n$).

- *Điều kiện 2 (Distinct)*: Chỉ số này chưa từng xuất hiện trước đó trong danh sách các vị trí đã chọn (để đảm bảo không có hai bit 1 trùng nhau tại cùng một vị trí).
- **Cập nhật**: Nếu thỏa mãn cả hai điều kiện, vị trí *index* được thêm vào danh sách và đặt bit tại đó bằng 1.
- **Lặp lại**: Quá trình lặp lại cho đến khi thu thập đủ w vị trí khác nhau.

2.2.6. PHÉP NHÂN ĐA THỨC (POLYNOMIAL MULTIPLICATION)

Các phép nhân trên $\mathbb{F}_2[X]/(X^n - 1)$ được thực hiện mà không xét đến độ thưa của đa thức để tránh rò rỉ thông tin. Các phép nhân được tính toán bằng cách kết hợp hai thuật toán *Toom-Cook* và *Karatsuba* theo [7].

Phép nhân Toom-Cook trên $\mathbb{F}_2[X]$

Giả sử cần nhân hai đa thức tùy ý trên $\mathbb{F}_2[X]$ có bậc tối đa $N - 1$, sử dụng thuật toán Toom-Cook. Các phương pháp chi tiết đã được nghiên cứu rộng rãi trong tài liệu. Giả sử A và B là hai đa thức nhị phân có bậc tối đa $N - 1$. Các đa thức này được đóng gói thành bảng gồm các từ 64 bit, sao cho kích thước của bảng là $\lceil N/64 \rceil$. Giả sử $t = 3n$ với một giá trị n đảm bảo $t \geq \lceil N/64 \rceil$. Khi đó, A và B được xem như các đa thức có bậc tối đa $64t - 1$. Ta chia A và B thành ba phần và muốn tính kết quả $C = A \cdot B$ như sau:

$$A = a_0 + a_1 \cdot X^{64n} + a_2 \cdot X^{2 \cdot 64n} \in \mathbb{F}_2[X],$$

$$B = b_0 + b_1 \cdot X^{64n} + b_2 \cdot X^{2 \cdot 64n} \in \mathbb{F}_2[X],$$

(với bậc tối đa là $64t - 1$, và a_i, b_i có bậc tối đa là $64n - 1$), và

$$C = c_0 + c_1 \cdot X^{64n} + c_2 \cdot X^{2 \cdot 64n} + c_3 \cdot X^{3 \cdot 64n} + c_4 \cdot X^{4 \cdot 64n} \in \mathbb{F}_2[X],$$

với bậc tối đa là $6 \cdot 64n - 2$.

Phiên bản “word-aligned” (căn theo từ) đánh giá đa thức tại các giá trị $0, 1, x = X^w, x+1 = X^w+1$, và ∞ , trong đó w là kích thước của từ (word size), thường là 64 trong các bộ xử lý hiện đại. Trên các bộ xử lý Intel, ta có thể chọn $w = 256$ để tận dụng các tập lệnh vector hóa AVX-AVX2 (Advanced Vector Extension) với cái giá phải trả là giảm kích thước một chút. Sau pha đánh giá, ta tiến hành nội suy (interpolation) để được các hệ số kết quả.

Với pha đánh giá, ta có:

2

$$C(0) = a_0 \cdot b_0,$$

$$C(1) = (a_0 + a_1 + a_2) \cdot (b_0 + b_1 + b_2),$$

$$C(x) = (a_0 + a_1 x + a_2 x^2) \cdot (b_0 + b_1 x + b_2 x^2),$$

$$C(x+1) = (a_0 + a_1(x+1) + a_2(x^2+1)) \cdot (b_0 + b_1(x+1) + b_2(x^2+1)),$$

$$C(\infty) = a_2 \cdot b_2.$$

Việc thực hiện giai đoạn này khá đơn giản, với điều kiện phép nhân $a_i \cdot b_i$ là một phép nhân Toom-Cook hoặc Karatsuba khác. Ta có thể nhận thấy rằng phép nhân với x hoặc x^2 hầu như là các phép dịch chuyển từ tự do. Sau khi có các giá trị $C(0)$, $C(1)$, $C(x)$, $C(x+1)$, $C(\infty)$, ta tiến hành pha nội suy (interpolation phase) thu các hệ số $c_0, \dots, c_4$ của kết quả C .

$$c_0 = C(0),$$

$$c_1 = \frac{x^2 + x + 1}{x^2 + x} C(0) + C(1) + \frac{C(x)}{x} + \frac{C(x+1)}{x+1} + (x^2 + x) C(\infty),$$

$$c_2 = \frac{C(1)}{x^2 + x} + \frac{C(x)}{x+1} + \frac{C(x+1)}{x} + (x^2 + x + 1) C(\infty),$$

$$c_3 = \frac{C(0)}{x^2 + x} + \frac{C(1)}{x^2 + x} + \frac{C(x)}{x+1} + \frac{C(x+1)}{x^2 + x},$$

$$c_4 = C(\infty).$$

Thuật toán Karatsuba.

Giả sử ta có hai đa thức nhị phân A và B có bậc tối đa $N-1$. Các đa thức này được gói vào bảng gồm các từ 64-bit, với kích thước $\lceil N/64 \rceil$ (lấy phần nguyên). Ký hiệu $t = 2^r$ với r là giá trị nhỏ nhất sao cho $t \geq \lceil N/64 \rceil$. Khi đó, A và B được xem như các đa thức có bậc tối đa $64 \cdot t - 1$. Thuật toán nhân tương ứng được mô tả trong phần *Karatsuba*. Trong thuật toán này, hai đa thức A và B được chia thành hai phần để xử lý, tuy nhiên có thể mở rộng thành các biến thể khác với nhiều phần tách khác nhau. Trong thực thi, ta dùng phép chia thành 3 phần (gọi là *3-Karatsuba*) như trong phép nhân Toom-Cook cơ bản cho HQC-128 và HQC-192, và phép chia thành 5 phần (gọi là *5-Karatsuba*) cho phép nhân Toom-Cook trong HQC-256. Phép nhân trong dòng thứ hai (ký hiệu là `Mult64`) có thể được biểu diễn bằng một lệnh trên bộ xử lý, cụ thể là lệnh (`pc1mul` for carry-less multiplier) giúp nhân các từ nhị phân hiệu quả.

Ứng dụng cho phép nhân HQC. Các tham số của HQC quyết định đến việc lựa chọn cách chia và loại thuật toán nhân cần dùng. Cụ thể, các tham số này dẫn tới việc xây dựng phép nhân trong $\mathbb{F}_2[X]$ như được miêu tả trong Bảng 2.1, theo hướng tối ưu hóa hiệu năng khi triển khai.

2

Algorithm 1 Thuật toán Karatsuba

Input: A, B gồm $t = 2^r$ từ máy tính

Output: $R = A \times B$

```

1: if  $t = 1$  then
2:   return Mult64( $A, B$ )
3: else
4:   Tách thành 2 nửa có kích thước  $t/2$ 
5:    $A = A_0 + x^{64t/2} A_1$ 
6:    $B = B_0 + x^{64t/2} B_1$ 
7:   Nhân đệ quy
8:    $R_0 \leftarrow \text{Karatsuba}(A_0, B_0, t/2)$ 
9:    $R_1 \leftarrow \text{Karatsuba}(A_1, B_1, t/2)$ 
10:   $R_2 \leftarrow \text{Karatsuba}(A_0 + A_1, B_0 + B_1, t/2)$ 
11:  Tái cấu trúc
12:   $R \leftarrow R_0 + (R_0 + R_1 + R_2)x^{64t/2} + R_1x^{64t}$ 
13:  return  $R$ 
14: end if

```

Trường hợp	HQC-1	HQC-3	HQC-5
Kích thước HQC (bit)	17 669	35 851	57 637
Phép nhân chính	Toom3-Karatsuba3 18 048	Toom3-Karatsuba3 36 480	Toom-Cook3 59 904
Phép nhân cơ bản	3-Karatsuba 6 144	3-Karatsuba 12 288	5-Karatsuba 20 480

Bảng 2.1: Hiện thực của các phép nhân HQC trên $\mathbb{F}_2[x]$

Kỹ thuật Nhân Đa thức thưa

Bên cạnh các giải thuật nhân tổng quát như Toom-Cook hay Karatsuba, việc hiện thực HQC trên phần cứng (FPGA) có thể được tối ưu hóa mạnh mẽ bằng cách tận dụng đặc tính “thưa” (sparse) của các vector tham số (như x, y, r_1, r_2, e). Trong HQC, các phép nhân thường có dạng $A \cdot B$, trong đó A là một đa thức ngẫu nhiên dày đặc (dense arbitrary polynomial) và B là một đa thức thưa có trọng số cố định (sparse fixed-weight polynomial). Thay vì thực hiện phép nhân thông thường, kỹ thuật này thực hiện phép nhân thông qua các thao tác dịch (shift) và cộng XOR (tương đương phép cộng trong $\mathbb{F}_2$) kết hợp với phép rút gọn modulo $X^n - 1$ xen kẽ [7].

2.2.7. MÃ NỘI REED-MULLER VÀ REED-SOLOMON

HQC sử dụng một mã C và các thuật toán $C.Encode$ và $C.Decode$, được khởi tạo bằng cách kết hợp các mã nội Reed-Muller và Reed-Solomon như mô tả trong [19].

Định nghĩa 2.2.7.1 (Mã nội). *Một mã nội bao gồm một mã bên ngoài $[n_e, k_e, d_e]$ trên $\mathbb{F}_q$ và một mã bên trong $[n_i, k_i, d_i]$ trên $\mathbb{F}_2$, với $q = 2^k$. Ta sử dụng một song ánh giữa các phần tử của $\mathbb{F}_q$ và các từ của mã bên trong để thu được phép biến đổi:*

$$\mathbb{F}_q^{n_e} \rightarrow \mathbb{F}_2^N$$

trong đó $N = n_e n_i$. Mã bên ngoài được biến đổi thành một mã nhị phân có các tham số:

$$[N = n_e n_i, K = k_e k_i, D \geq d_e d_i].$$

Đối với mã bên ngoài, ta sử dụng mã Reed-Solomon có kích thước 32 trên $\mathbb{F}_{256}$. Đối với mã bên trong, ta sử dụng mã Reed-Muller $[128, 8, 64]$, cái mà ta sẽ nhân lên 3 hoặc 5 lần (tức là nhân từng bit để thu được các mã có tham số $[384, 8, 192]$ và $[640, 8, 320]$). Giải mã khả năng tối đa được thực hiện trên mã bên trong. Sau đó, ta thu được một vector của $\mathbb{F}_q^{n_e}$, được giải mã bằng bộ giải đại số của mã Reed-Solomon.

Mã Reed-Solomon.

Giả sử p là một số nguyên tố và q là một lũy thừa bất kỳ của p . Theo [20], một mã Reed-Solomon $RS[n, k, d_{\min}]$ với các ký hiệu trong $\mathbb{F}_q$ có các tham số sau:

- Độ dài khối: $n = q - 1$;

- Số lượng các chữ số kiểm tra (parity-check digits): $n - k = 2\delta$, với δ là khả năng sửa lỗi của mã và k là số lượng bit thông tin;
- Khoảng cách tối thiểu: $d_{\min} = 2\delta + 1$.

Giả sử α là phần tử nguyên thủy trong $\mathbb{F}_{2^m}$, đa thức sinh $g(x)$ của $RS[n, k, \delta]$ được cho bởi:

$$g(x) = (x + \alpha)(x + \alpha^2) \cdots (x + \alpha^{2\delta}).$$

Các mã Reed-Solomon rút gọn dùng trong HQC.

Tùy theo tham số của HQC, ta có thể xây dựng các mã Reed-Solomon rút gọn (RS-S1, RS-S2 và RS-S3) sao cho k bằng 16, 24 hoặc 32, dựa trên các mã RS-S1, RS-S2 và RS-S3 trong [20]. Các mã rút gọn này được xây dựng bằng cách trừ 209 khỏi các tham số n và k của mã RS-1, trừ 199 khỏi các tham số của mã RS-2, và trừ 165 khỏi các tham số của mã RS-3. Do đó, ta thu được các mã Reed-Solomon rút gọn sau.

- RS-S1 [$46 = 255 - 209$, $16 = 225 - 209$, 31];
- RS-S2 [$56 = 255 - 199$, $24 = 223 - 199$, 33];
- RS-S3 [$90 = 255 - 165$, $32 = 197 - 165$, 49].

Cần lưu ý rằng việc rút gọn mã Reed-Solomon không làm giảm khả năng sửa lỗi của nó. Bảng 2.2 dưới đây thể hiện các tham số của cả mã Reed-Solomon gốc và mã Reed-Solomon đã được rút gọn.

Code	n	k	δ
RS-1	255	225	15
RS-2	255	223	16
RS-3	255	197	29
RS-S1	46	16	15
RS-S2	56	24	16
RS-S3	90	32	29

Bảng 2.2: Các mã Reed-Solomon gốc và rút gọn.

Các đa thức sinh.

Trong HQC, ta làm việc trong trường $\mathbb{F}_{2^m}$ với $m = 8$. Để xây dựng trường này, ta sử dụng đa thức nguyên thủy $1 + a^2 + a^3 + a^4 + a^8$ có bậc 8 (theo

[20]). Ta ký hiệu các đa thức sinh của RS-S1, RS-S2 và RS-S3 lần lượt là $g_1(x)$, $g_2(x)$ và $g_3(x)$ - tương ứng với các đa thức sinh của các mã Reed-Solomon gốc RS-1, RS-2 và RS-3. Các đa thức này đã được tiền tính toán và được trình bày dưới đây. Ta cũng có thể sử dụng các hàm trong file `reed_solomon.h` để tái tạo lại các đa thức sinh này khi cần.

Đa thức sinh của RS-S1:

$$\begin{aligned} g_1(x) = & 89 + 692x + 153x^2 + 116x^3 + 176x^4 + 111x^5 + 75x^7 + 73x^8 + 233x^9 \\ & + 242x^{10} + 233x^{11} + 65x^{12} + 210x^{13} + 124x^{14} + 139x^{15} + 67x^{18} + 118x^{19} \\ & + 105x^{20} + 210x^{21} + 174x^{22} + 110x^{23} + 74x^{24} + 69x^{25} + 255x^{28} + 181x^{29} + x^{30}. \end{aligned}$$

Đa thức sinh của RS-S2:

$$\begin{aligned} g_2(x) = & 45 + 216x + 239x^2 + 162x^3 + 40x^4 + 107x^8 + 50x^9 + 163x^{10} \\ & + 210x^{11} + 227x^{12} + 134x^{13} + 128x^{14} + 189x^{19} + 238x^{20} + 164x^{21} + 182x^{22} \\ & + 43x^{23} + 15x^{24} + 232x^{25} + 246x^{26} + 142x^{27} + 50x^{28} + 189x^{29} + 29x^{30} + 232x^{31} + x^{32}. \end{aligned}$$

Đa thức sinh của RS-S3:

$$\begin{aligned} g_3(x) = & 49 + 167x + 49x^2 + 39x^3 + 200x^4 + 121x^5 + 124x^6 + 91x^7 + 240x^8 + 63x^9 \\ & + 148x^{10} + 71x^{11} + 150x^{12} + 123x^{13} + 87x^{14} + 101x^{15} + 32x^{16} + 215x^{17} \\ & + 159x^{18} + 71x^{19} + 201x^{20} + 115x^{21} + 97x^{22} + 210x^{23} + 186x^{24} \\ & + 183x^{25} + 141x^{26} + 217x^{27} + 123x^{28} + 12x^{29} + 31x^{30} + 243x^{31} + 180x^{32} \\ & + 219x^{33} + 152x^{34} + 239x^{35} + 99x^{36} + 141x^{37} + 4x^{38} + 246x^{39} + 191x^{40} \\ & + 144x^{41} + 8x^{42} + 232x^{43} + 47x^{44} + 27x^{45} + 141x^{46} + 178x^{47} + 130x^{48} \\ & + 64x^{49} + 124x^{50} + 47x^{51} + 39x^{52} + 188x^{53} + 216x^{54} + 48x^{55} + 199x^{56} \\ & + 187x^{57} + x^{58}. \end{aligned}$$

Mã hóa mã Reed-Solomon rút gọn.

Trong phần này, ta trình bày việc mã hóa cho các mã Reed-Solomon và cách dùng chúng để mã hóa các mã Reed-Solomon rút gọn. Ta ký hiệu $u(x) = u_0 + u_1x + \dots + u_{k-1}x^{k-1}$ là đa thức tương ứng với thông điệp cần truyền $u = (u_0, \dots, u_{k-1})$ sẽ được mã hóa, và $g(x)$ là đa thức sinh. Chúng ta sử dụng dạng hệ thống của mã hóa, trong đó k bit ngoài cùng bên phải của đa thức từ mã là các bit thông điệp và $n - k$ bit ngoài cùng bên trái là các bit kiểm tra chẵn lẻ. Theo [7], từ mã được tính như sau $c(x) = b(x) + x^{n-k}u(x)$, trong đó $b(x)$ là phần dư khi chia đa thức $x^{n-k}u(x)$ cho đa thức sinh $g(x)$. Do đó, quá trình mã hóa theo dạng hệ thống bao gồm ba bước:

1. Nhân đa thức thông điệp $u(x)$ với x^{n-k} .
2. Tính phần dư $b(x)$ bằng phép chia $x^{n-k}u(x)$ cho đa thức sinh $g(x)$.
3. Kết hợp $b(x)$ và $x^{n-k}u(x)$ để thu được đa thức mã $c(x) = b(x) + x^{n-k}u(x)$.

2

Giải mã các mã Reed–Solomon rút gọn.

Giải mã của các mã Reed–Solomon cổ điển có thể được sử dụng để giải mã các mã Reed–Solomon rút gọn. Để đơn giản, ta sẽ trình bày quá trình giải mã của mã Reed–Solomon cổ điển. Theo [20], xét một mã Reed–Solomon có các tham số $[n, k, d_{\min}]$ với $n = 2^m - 1$ ($m \geq 0$ là số nguyên dương) và đa thức từ mã được biểu diễn như sau:

$$v(x) = v_0 + v_1x + \cdots + v_{n-1}x^{n-1}.$$

Giả sử từ mã $v(x)$ được truyền đi và thu nhận bởi $r(x) = v(x) + e(x)$, trong đó $e(x)$ là đa thức lỗi, và một số hệ số trong $e(x)$ khác 0 khi có lỗi xảy ra tại các vị trí tương ứng.

Ta ký hiệu $e(x) = e_{j_1}x^{j_1} + e_{j_2}x^{j_2} + \cdots + e_{j_t}x^{j_t}$, với t là số lượng lỗi tại các vị trí $j_1, j_2, \dots, j_t$. Khi đó, ta có $r(x) = v(x) + e(x)$ và $r(\alpha^i) = e(\alpha^i)$ (vì $v(\alpha^i) = 0$ do $v(x)$ là từ mã). Giả sử $e(x)$ có t lỗi tại các vị trí $j_1, j_2, \dots, j_t$. Khi đó:

$$e(x) = e_{j_1}x^{j_1} + e_{j_2}x^{j_2} + \cdots + e_{j_t}x^{j_t}.$$

Ta thu được hệ phương trình sau:

$$\begin{cases} S_1 = e_{j_1}\alpha^{j_1} + e_{j_2}\alpha^{j_2} + \cdots + e_{j_t}\alpha^{j_t}, \\ S_2 = e_{j_1}(\alpha^{j_1})^2 + e_{j_2}(\alpha^{j_2})^2 + \cdots + e_{j_t}(\alpha^{j_t})^2, \\ S_3 = e_{j_1}(\alpha^{j_1})^3 + e_{j_2}(\alpha^{j_2})^3 + \cdots + e_{j_t}(\alpha^{j_t})^3, \\ \vdots \\ S_{2t} = e_{j_1}(\alpha^{j_1})^{2\delta} + e_{j_2}(\alpha^{j_2})^{2\delta} + \cdots + e_{j_t}(\alpha^{j_t})^{2\delta}, \end{cases}$$

trong đó $S_1, S_2, \dots, S_{2t}$ là các syndrome, còn các giá trị $\alpha^{j_1}, \alpha^{j_2}, \dots, \alpha^{j_t}$ là các ẩn chưa biết.

Mục tiêu của thuật toán giải mã Reed–Solomon là giải hệ phương trình trên để tìm ra các lỗi e_j . Ta định nghĩa các số của vị trí lỗi bằng $\beta_i = \alpha^{j_i}$, tức là các giá trị vị trí lỗi trong trường F_{2^m} . Các phương trình ở trên có thể viết lại như sau:

$$\begin{cases} S_1 = e_{j_1}\beta_1 + e_{j_2}\beta_2 + \cdots + e_{j_t}\beta_t, \\ S_2 = e_{j_1}\beta_1^2 + e_{j_2}\beta_2^2 + \cdots + e_{j_t}\beta_t^2, \\ S_3 = e_{j_1}\beta_1^3 + e_{j_2}\beta_2^3 + \cdots + e_{j_t}\beta_t^3, \\ \vdots \\ S_{2\delta} = e_{j_1}\beta_1^{2\delta} + e_{j_2}\beta_2^{2\delta} + \cdots + e_{j_t}\beta_t^{2\delta}, \end{cases}$$

Ta định nghĩa đa thức vị trí lỗi (error location polynomial) như sau:

$$\sigma(x) = (1 + \beta_1 x)(1 + \beta_2 x) \cdots (1 + \beta_t x) = 1 + \sigma_1 x + \sigma_2 x^2 + \cdots + \sigma_t x^t$$

Ta có thể thấy rằng các nghiệm của $\sigma(x)$ là $\beta_1^{-1}, \beta_2^{-1}, \dots, \beta_t^{-1}$, đây là các nghịch đảo của các chỉ số vị trí lỗi. Sau khi tìm được các hệ số của $\sigma(x)$, ta có thể tính được giá trị lỗi tương ứng. Cho:

$$Z(x) = 1 + (S_1 + \sigma_1)x + (S_2 + \sigma_1 S_1 + \sigma_2)x^2 + \cdots + (S_t + \sigma_1 S_{t-1} + \sigma_2 S_{t-2} + \cdots + \sigma_t)x^t$$

thì giá trị lỗi tại vị trí β_i được cho bởi công thức [21]:

$$e_{j_i} = \frac{Z(\beta_i^{-1})}{\prod_{\substack{j=1 \\ j \neq i}}^t (1 + \beta_j \beta_i^{-1})}$$

Quá trình giải mã hoàn tất khi tính được:

$$r(x) = c(x) - e(x).$$

Tóm tắt quá trình giải mã

Toàn bộ quá trình giải mã Reed–Solomon có thể được tóm tắt như sau:

1. Tính 2δ hội chứng (syndromes) từ đa thức nhận được $r(x)$. Các hội chứng này được tính theo cách thông thường bằng cách đánh giá $r(\alpha^i)$ với mỗi giá trị i .
2. Tính đa thức vị trí lỗi $\sigma(x)$ từ 2δ hội chứng đã thu được ở bước đầu tiên. Ở đây, ta sử dụng thuật toán Berlekamp's [20].
3. Tìm các chỉ số vị trí lỗi bằng cách tính nghiệm của đa thức $\sigma(x)$ và lấy nghịch đảo của chúng. Bước này được thực hiện bằng thuật toán Additive Fast Fourier Transform như mô tả trong [22].

4. Tính đa thức $Z(x)$.
5. Tính giá trị lỗi tại các vị trí đã xác định.
6. Hiệu chỉnh các lỗi trong đa thức đã nhận được $r(x)$.

2

Các mã Reed–Mullers nhân đôi

Với mọi số nguyên dương m và r sao cho $0 \leq r \leq m$, ta có thể xây dựng một mã Reed–Muller nhị phân bậc r ký hiệu là $RM(r, m)$ với các tham số như sau:

- Độ dài mã: $n = 2^m$;
- Số chiều: $k = \sum_{i=0}^r \binom{m}{i}$;
- Khoảng cách tối thiểu: $d_{\min} = 2^{m-r}$.

Trong HQC, ta sử dụng các mã Reed–Muller nhân đôi. Cụ thể, ta sử dụng mã Reed–Muller bậc nhất, ký hiệu là $RM(1, 7)$ – đây là mã nhị phân với các tham số [128, 8, 64].

Giải mã các mã Reed–Mullers bên trong.

Mã Reed–Mullers bậc 1 có thể được giải mã bằng phép biến đổi Hadamard nhanh (xem chương 14 trong [21]). Tuy nhiên, khi giải mã các mã nhân đôi, thuật toán cần được điều chỉnh một chút cho phù hợp. Ví dụ, khi mã Reed–Mullers được nhân đôi ba lần, ta tạo hàm $F: \mathbb{F}_2^3 \rightarrow \{3, 1, -1, -3\}$, trong đó ta bắt đầu bằng việc biến đổi từng khối ba bit $x_1 x_2 x_3$ của vector nhận được theo công thức:

$$(-1)^{x_1} + (-1)^{x_2} + (-1)^{x_3}$$

Sau đó, ta áp dụng phép biến đổi Hadamard lên hàm F . Ta chọn giá trị lớn nhất trong $\hat{F}$ và vector $x \in \mathbb{F}_2^3$ sao mà giá trị $|\hat{F}|$ đạt cực đại. Nếu $F(x)$ là số dương, thì từ mã gần nhất là xG , trong đó G là ma trận sinh của mã Hadamard (không chứa vector toàn số 1). Nếu $\hat{F}(x)$ là âm, thì ta cần thêm vector toàn số 1 vào.

Mã hóa mã Reed–Mullers nhân đôi.

Theo [21], việc mã hóa được thực hiện theo cách cổ điển bằng phép nhân ma trận vector. Ma trận sinh G được biểu diễn dưới dạng hexa bên dưới:

2

$$G = \begin{pmatrix} \text{aaaaaaaa} & \text{aaaaaaaa} & \text{aaaaaaaa} & \text{aaaaaaaa} \\ \text{cccccccc} & \text{cccccccc} & \text{cccccccc} & \text{cccccccc} \\ \text{f0f0f0f0} & \text{f0f0f0f0} & \text{f0f0f0f0} & \text{f0f0f0f0} \\ \text{ff00ff00} & \text{ff00ff00} & \text{ff00ff00} & \text{ff00ff00} \\ \text{ffff0000} & \text{ffff0000} & \text{ffff0000} & \text{ffff0000} \\ \text{00000000} & \text{ffffff} & \text{00000000} & \text{ffffff} \\ \text{00000000} & \text{00000000} & \text{ffffff} & \text{ffffff} \\ \text{ffffff} & \text{ffffff} & \text{ffffff} & \text{ffffff} \end{pmatrix}$$

Nếu thông điệp được biểu diễn bởi:

$$\mathbf{m} = (m_0, \dots, m_7) \in \mathbb{F}_2^8$$

thì vector mã được tính theo:

$$\mathbf{c} = \mathbf{mG}$$

Sau đó, codeword cuối cùng được tạo bằng cách lặp lại vector $\mathbf{c}$ 3 hoặc 5 lần, tùy thuộc vào mức độ bảo mật của hệ thống.

Giải mã các mã Reed-Muller nhân đôi.

Theo [21], việc giải mã các mã Reed-Muller nhân đôi được thực hiện trong 3 bước:

1. Tính hàm F được mô tả và áp dụng vào các từ mã nhận được. Ta sẽ cung cấp chi tiết về cách thực hiện quy trình này khi số bội bằng 2.
2. Giả sử v là một mã Reed-Muller nhân đôi, nó có thể được biểu diễn $v = (a_1 b_1, \dots, a_{n_2} b_{n_2})$ trong đó mỗi a_i, b_i có kích thước 128 bit ($a_i = (a_{i_0}, \dots, a_{i_{128}})$ và $b_i = (b_{i_0}, \dots, b_{i_{128}})$). Phép biến đổi F được áp dụng cho từng phần tử trong v như sau: $((-1)^{a_{i_0}} + (-1)^{b_{i_0}}, \dots, (-1)^{a_{i_0}} + (-1)^{b_{i_{128}}})$. Trường hợp bội số bằng 3 hoặc 5 cũng được xử lý tương tự.
2. Tính phép biến đổi Hadamard, đây là pha đầu tiên của Green machine.
3. Xác định vị trí có giá trị lớn nhất trong đầu ra của bước trước. Đây là pha thứ hai của Green machine. Khi giá trị đỉnh là dương, ta cộng thêm vector toàn các phần tử bằng 1. Nếu có hai đỉnh có giá trị bằng nhau, ta chọn đỉnh có 7 bit thấp nhất nhỏ hơn.

2.3. CHI TIẾT GIẢI THUẬT

2.3.1. KEY GENERATION

Thuật toán KeyGen dùng để sinh ra cặp khóa công khai và bí mật cho HQC-KEM.

2

Algorithm 2 HQC.PKE.KeyGen() and HQC.KEM.KeyGen() [4]

```

1:  $h \leftarrow \mathcal{R}$ 
2:  $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathcal{R}^2$ 
3:  $\mathbf{s} := \mathbf{x} + h \cdot \mathbf{y}$ 
4: return  $(pk := (h, \mathbf{s}), sk := (\mathbf{x}, \mathbf{y}))$ 

```

Sau đây là giải thích từng bước:

- **Bước 1:** $h \leftarrow \mathcal{R}$
 - Sinh một đa thức (hoặc vector nhị phân) h thuộc $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$, đóng vai trò là thành phần của khóa công khai. Giá trị h được sinh ngẫu nhiên từ `public_seed` bằng bộ sinh ngẫu nhiên mở rộng XOF (SHAKE256), giúp tạo ra chuỗi bit ngẫu nhiên có độ dài tùy ý.
- **Bước 2:** $(x, y)_w \leftarrow \mathcal{R}^2$ (sinh hai vector trọng số cố định)
 - Hai khóa bí mật x và y được sinh sao cho mỗi vector có đúng w bit bằng 1 (Hamming weight = w) bằng cách gọi hàm **Fixed Weight Vector Generation**. Thuật toán này nhận đầu vào gồm $(N, w, seed)$ và thực hiện như sau:
 1. Sinh ra $32 \times w$ bit ngẫu nhiên bằng PRNG (SHAKE256) từ `secret_seed`, chia thành w từ 32-bit.
 2. Với mỗi từ 32-bit r_i , tính chỉ số vị trí bit 1 theo công thức $pos[i] = i + (r_i \bmod (N - i))$, tạo thành danh sách w vị trí khác nhau trong khoảng $[0, N - 1]$.
 3. Kiểm tra và loại bỏ các phần tử trùng lặp; nếu trùng thì gán lại $pos[j] = j$ để đảm bảo tính duy nhất và giữ đúng trọng số w .
 4. Trả về mảng vị trí $pos[0..w - 1]$, thể hiện các vị trí bit bằng 1 trong vector. Trong phần cứng, chỉ lưu mảng này để tiết kiệm bộ nhớ thay vì lưu toàn bộ vector n -bit.
- **Bước 3:** $\mathbf{s} := x + h \cdot \mathbf{y}$

Algorithm 3 Constant Weight Word / Fixed Weight Vector Generation**Input:** $N, w, seed$ **Output:** w distinct elements in range 0 to $N - 1$

```

1:  $rand\_bits \leftarrow prng(input = seed, output\_size = 32 \times w)$ 
2: for  $i \leftarrow 0$  to  $w - 1$  do
3:    $pos[i] \leftarrow i + (rand\_bits[32 + 32 \times i - 1 : 32 \times i]) \bmod (N - i)$ 
4: end for
5: for  $j \leftarrow w - 1$  to 0 do
6:    $duplicate\_found \leftarrow 0$ 
7:   for  $k \leftarrow j + 1$  to  $w - 1$  do
8:     if  $pos[j] == pos[k]$  then
9:        $duplicate\_found \leftarrow 1$ 
10:    end if
11:  end for
12:  if  $duplicate\_found == 1$  then
13:     $pos[j] \leftarrow j$ 
14:  end if
15: end for
16: return  $pos$ 

```

- Tạo thành phần s của khóa công khai bằng cách nhân đa thức h với vector thừa y trong $\mathbb{F}_2[X]/(X^n - 1)$, sau đó cộng XOR với x . Kết quả $s = x + h \cdot y$ tạo nên cấu trúc mã tuyến tính, là phần cốt lõi quyết định độ khó của bài toán giải mã trong HQC.

- **Bước 4: Trả về** ($pk = (h, s)$, $sk = (x, y)$)

- Thuật toán trả về cặp khóa: khóa công khai $pk = (h, s)$ và khóa bí mật $sk = (x, y)$. Khóa công khai được sử dụng trong giai đoạn mã hóa hoặc trao đổi khóa (Encaps), trong khi khóa bí mật được giữ kín để sử dụng trong giai đoạn giải mã (Decaps).

2.3.2. ENCAPSULATION

Đây là bước Encapsulation (bọc khóa) trong HQC-KEM, người gửi dùng public key (h, s) của người nhận để sinh ra khóa bí mật tạm K và gói dữ liệu gửi đi $(c, d, salt)$. Sau đây là giải thích từng bước:

- **Bước 1:** $m \leftarrow \mathbb{F}_2^k$

- Sinh một thông điệp ngẫu nhiên m gồm k bit. Đây không phải

Algorithm 4 $\text{HQC.KEM.Encapsulate}(pk = (h, s))$ [4]

```

1:  $\mathbf{m} \leftarrow \mathbb{F}_2^k$ 
2:  $\text{salt} \leftarrow \mathbb{F}_2^{128}$ 
3:  $\theta := \mathcal{G}(\mathbf{m} || pk || \text{salt})$ 
4:  $\mathbf{c} := (u, v) = \text{HQC.PKE.Encrypt}(pk, \mathbf{m}, \theta)$ 
5:  $K := \mathcal{K}(\mathbf{m}, \mathbf{c})$ 
6:  $\mathbf{d} := \mathcal{H}(\mathbf{m})$ 
7: return  $(K, (\mathbf{c}, \mathbf{d}, \text{salt}))$ 

```

2

dữ liệu người dùng thật mà là thông điệp ngẫu nhiên nội bộ dùng để sinh khóa.

- **Bước 2:** $\text{salt} \leftarrow \mathbb{F}_2^{128}$
 - Sinh thêm một giá trị ngẫu nhiên salt dài 128 bit.
- **Bước 3:** $\theta := \mathcal{G}(\mathbf{m} || pk || \text{salt})$
 - Tạo chuỗi ngẫu nhiên θ bằng cách băm (hash) thông điệp m , public key $pk = (h, s)$, và salt lại với nhau.
- **Bước 4:** $\mathbf{c} := (u, v) = \text{HQC.PKE.Encrypt}(pk, \mathbf{m}, \theta)$
 - Mã hóa m bằng HQC-PKE (Public Key Encryption) với public key $pk = (h, s)$ và randomness θ .

Algorithm 5 $\text{HQC.PKE.Encrypt}(pk = (h, s), m, \theta)$ [4]

```

1:  $(\mathbf{r}_1, \mathbf{r}_2, \mathbf{e}) \xleftarrow[\omega, \theta]{\mathcal{R}^3}$ 
2:  $\mathbf{u} := \mathbf{r}_1 + h \cdot \mathbf{r}_2$ 
3:  $\mathbf{t} := \text{Encode}(m)$ 
4:  $\mathbf{v} := \mathbf{t} + s \cdot \mathbf{r}_2 + \mathbf{e}$ 
5: return  $\mathbf{c} := (\mathbf{u}, \mathbf{v})$ 

```

Sau đây là giải thích các bước của Algorithm 5.

1. Sinh ba vector ngẫu nhiên: r_1, r_2 và e từ hàm PRNG (pseudorandom number generator) với seed θ .
2. Phép nhân và cộng trong trường $\text{GF}_2[X]/(X^n - 1)$
3. Hàm $\text{Encode}()$ biến thông điệp m (chuỗi bit ngắn) thành vector mã t có độ dài n . Hàm này dùng mã Reed–Solomon hoặc BCH để tăng tính chống nhiễu (error correction)

2

4. Tiếp tục nhân s (từ public key) với r_2 , rồi cộng với t và e .
Tất cả đều trong $\mathbb{GF}_2$, nên phép cộng là XOR.
 5. Trả về ciphertext gồm hai phần (u, v) .
- **Bước 5:** $K := \mathcal{H}(\mathbf{m}, \mathbf{c})$
 - Sinh khóa bí mật dùng chung K bằng cách hash thông điệp m và ciphertext c . Đây chính là shared secret key mà hai bên sẽ dùng để mã hóa dữ liệu đối xứng sau này
 - **Bước 6:** $\mathbf{d} := \mathcal{H}(\mathbf{m})$
 - Tính một hash xác thực của thông điệp m . Dùng để kiểm tra khi giải mã xem m' (m đã giải) có trùng với m ban đầu không.
 - **Bước 7:** **return** $(K, (\mathbf{c}, \mathbf{d}, \text{salt}))$
 - K : khóa bí mật chia sẻ giữa 2 bên (chỉ bên giải mã mới có thể khôi phục).
 - $(\mathbf{c}, \mathbf{d}, \text{salt})$: gói dữ liệu gửi đi cho người nhận.

2.3.3. DECAPSULATION

Thuật toán **Decapsulate** được sử dụng để giải mã bản mã $(\mathbf{c}, \mathbf{d})$ bằng khóa bí mật $(\mathbf{x}, \mathbf{y})$, từ đó khôi phục khóa phiên chung K' . Thuật toán này là phần quan trọng của cơ chế trao đổi khóa HQC-KEM, đảm bảo tính bảo mật IND-CCA2 thông qua cơ chế mã hóa lại (re-encryption) và kiểm tra hằng thời gian.

Algorithm 6 HQC.KEM.Decapsulate($\text{sk} = (\mathbf{x}, \mathbf{y}), \mathbf{c}, \mathbf{d}, \text{salt}$) [4]

```

1:  $\mathbf{m}' \leftarrow \text{HQC.PKE.Decrypt}(\text{sk}, \mathbf{c})$ 
2:  $\theta' \leftarrow \mathcal{G}(\mathbf{m}' || \text{pk} || \text{salt})$ 
3:  $\mathbf{c}' \leftarrow (\mathbf{u}', \mathbf{v}') = \text{HQC.PKE.Encrypt}(\text{pk}, \mathbf{m}', \theta')$ 
4:  $\mathbf{d}' \leftarrow \mathcal{H}(\mathbf{m}')$ 
5:  $K' \leftarrow \mathcal{H}(\mathbf{m}', \mathbf{c})$ 
6: if  $(\mathbf{c} \neq \mathbf{c}') \vee (\mathbf{d} \neq \mathbf{d}')$  then
7:   return  $(K', 0)$ 
8: else
9:   return  $(K', 1)$ 
10: end if

```

Giải thích chi tiết từng bước:

• **Bước 1: Giải mã bản mã c để thu được m'**

- Thuật toán gọi hàm **HQC.PKE.Decrypt** để giải mã ciphertext $c = (u, v)$ bằng khóa bí mật $sk = (x, y)$:

$$m' := \text{Decode}(v - u \cdot y)$$

- Trong đó, phép nhân $u \cdot y$ được thực hiện trong không gian $\mathbb{F}_2[X]/(X^n - 1)$, và phép trừ tương đương với phép XOR.
- Hàm **Decode** là bộ giải mã mã tuyến tính (ví dụ: Reed–Solomon hoặc Reed–Muller) được sử dụng trong HQC, giúp khôi phục thông điệp m' từ bản mã bị nhiễu.
- Nếu số lỗi vượt quá khả năng sửa, m' có thể sai, nhưng thuật toán vẫn tiếp tục để đảm bảo *constant-time*.

• **Bước 2: Sinh lại chuỗi ngẫu nhiên θ' từ m' , pk và $salt$**

- Hàm $\mathcal{G}$ là một hàm băm SHAKE256, được sử dụng như bộ sinh ngẫu nhiên xác định (deterministic PRNG).
- m' , pk và $salt$ được nối lại và sau đó θ' là chuỗi ngẫu nhiên được sinh ra từ kết quả đó thông qua hàm $\mathcal{G}$, đảm bảo rằng nếu m' đúng, θ' sẽ trùng khớp với giá trị dùng trong quá trình mã hóa.

$$\theta' := \mathcal{G}(m' || pk || salt)$$

• **Bước 3: Mã hóa lại thông điệp m' để tạo bản mã c'**

- Dùng lại public key $pk = (h, s)$ và chuỗi ngẫu nhiên θ' để tái mã hóa:

$$c' = (u', v') := \text{HQC.PKE.Encrypt}(pk, m', \theta')$$

- Bản mã c' sau đó được so sánh với c để xác minh tính hợp lệ của ciphertext.

• **Bước 4: Tạo giá trị kiểm tra d'**

- Hàm băm $\mathcal{H}$ tạo ra giá trị hash d' từ m' :

$$d' := \mathcal{H}(m')$$

- Việc so sánh d' với d giúp phát hiện nếu ciphertext đã bị thay đổi.

- **Bước 5: Tạo khóa phiên K'**

- Hàm $K(\cdot)$ là hàm dẫn xuất khóa (KDF) dùng để sinh khóa bí mật chia sẻ:

$$K' := \mathcal{K}(\mathbf{m}', \mathbf{c})$$

- K' là khóa phiên cuối cùng dùng cho mã hóa đối xứng (như AES-256 hoặc ChaCha20) giữa hai bên.
- Dù quá trình xác thực thất bại, K' vẫn được tính toán để tránh rò rỉ thời gian thực thi (giữ *constant-time*).

- **Bước 6: Kiểm tra và trả kết quả**

- Nếu $\mathbf{c} \neq \mathbf{c}'$ hoặc $\mathbf{d} \neq \mathbf{d}'$, thuật toán trả về $(K', 0)$ — tức giải mã thất bại.
- Nếu hai cặp $(\mathbf{c}, \mathbf{c}')$ và $(\mathbf{d}, \mathbf{d}')$ khớp nhau, thuật toán trả về $(K', 1)$ — tức quá trình decapsulation thành công và K' là khóa phiên hợp lệ.

2.4. FIELD PROGRAMMABLE GATE ARRAY

FPGA, viết tắt của field programmable gate array, là một loại mạch tích hợp có khả năng cao, một mạch điện tử trong một gói duy nhất [23]. Phần field programmable trong tên cho biết rằng FPGA có thể được lập trình lại khi ở trong field (nghĩa là không cần phải trả lại cho nhà sản xuất). Phần gate array biểu thị rằng một FPGA được tạo thành từ một lưới hai chiều có một số lượng lớn các cổng logic, đơn vị cơ bản của logic kỹ thuật số.

2.4.1. LOOK-UP TABLES

Trong mục này, chúng ta sẽ mở rộng chủ đề về cách hoạt động của bảng chân trị và thảo luận về các phương trình đại số Boole phức tạp hơn.

Đầu tiên cần lưu ý rằng tất cả các cổng logic rời rạc mà chúng ta đã thảo luận trước đây (AND, OR, v.v.) thực sự không tồn tại bên trong một FPGA! Tuy nhiên có thể thực hiện được các chức năng đó. Cách mà các FPGA có thể thực hiện đại số Boolean là sử dụng Look-Up Tables (LUT). Look-Up Tables là một khối chức năng riêng biệt có thể được lập trình bởi Nhà thiết kế kỹ thuật số. LUT sử dụng khái niệm bảng chân lý tương tự để liên kết đầu ra với đầu vào. Sau đây là một ví dụ.

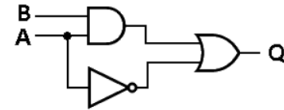
Tạo bảng chân trị cho phương trình Boolean sau: $Q = A * B + \bar{A}$. Trong đó:

* AND

+ OR

Vì vậy, ta có thể phát biểu rằng phương trình Boolean $Q = A*B + \bar{A}$ có thể được đọc là "Đầu ra Q nhận được từ A và B hoặc phủ định A". Chúng ta hãy nhìn vào bảng chân trị và sơ đồ luận cổ luận lí được tạo bởi phương trình này. Như có thể thấy trong hình bên dưới, phải mất tổng cộng ba cổng để tạo ra mạch này.

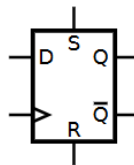
Truth Table		
Input A	Input B	Output Q
0	0	1
0	1	1
1	0	0
1	1	1



Bảng chân lý trong ví dụ trên có hai đầu vào A và B, có nghĩa là có thể có bốn khả năng đầu ra. Mỗi đầu vào tăng số lượng đầu ra có thể có theo hệ số 2. Vì vậy, đối với một đầu vào có 2 khả năng đầu ra, đối với 2 đầu vào có 4 khả năng đầu ra, đối với 3 đầu vào có 8 khả năng đầu ra, v.v. Về mặt toán học, điều này có thể được biểu thị bằng 2^n , với n là số lượng đầu vào.

2.4.2. FLIP FLOP

Trong phần trước, chúng ta đã thảo luận về thành phần Look-Up Tables (LUT). Đây là một trong hai thành phần quan trọng nhất bên trong FPGA, thành phần quan trọng nhất còn lại là Flip Flop. Có một số loại Flip Flop khác nhau (JK, T, D) nhưng loại được sử dụng thường xuyên nhất là D Flip Flop.



Hình 2.5: D Flip Flop

Trong hình trên có một chân ở trên cùng của thiết bị có nhãn S, đây được gọi là chân Set. Ở dưới cùng của thiết bị có một chân có nhãn R, chân này được gọi là chân Reset. Ngoài ra, ở cạnh phải của máy còn có một chân

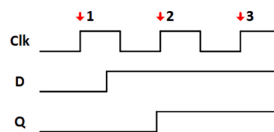
ghi nhãn $\overline{Q}$, chân này sẽ luôn chứa giá trị ngược lại với giá trị trên chân Q. Ba chân quan trọng nhất trên Flip Flop là:

2

- D đầu vào dữ liệu cho flip-flop
- Q đầu ra dữ liệu của flip-flop
- > xung clock cho đầu vào

Đồng hồ kỹ thuật số là thứ làm cho hầu hết các mạch kỹ thuật số hoạt động. Một cách để hình dung về đồng hồ trong một hệ thống là nghĩ về bộ bánh răng. Để bất kỳ bánh răng nào trong hệ thống quay, phải có một số bánh răng chính dẫn động tất cả các bánh răng khác. Đó thực sự là mục đích của đồng hồ trong logic kỹ thuật số. Nó cung cấp một luồng ổn định các chuyển đổi điện áp từ thấp đến cao xuống thấp cho phép FPGA hoạt động. Thêm vào đó, sự tương tự rất thú vị vì các bánh răng trông hơi giống sóng vuông của đồng hồ kỹ thuật số.

Xung Clock là thứ cho phép Flip Flop được sử dụng như một phần lưu trữ dữ liệu. Bất kỳ phần tử lưu trữ dữ liệu nào đều được gọi là logic tuần tự hoặc logic đã đăng ký. Logic tuần tự hoạt động dựa trên sự chuyển mức của tín hiệu clock. 99,9% thời gian đây sẽ là cạnh lên (khi xung clock chuyển từ mức 0 lên mức 1). Khi Flip Flop nhìn thấy cạnh lên của xung clock, nó sẽ ghi dữ liệu từ đầu vào D sang đầu ra Q. Flip Flop là thứ giúp cho các FPGA phức tạp trở nên khả thi. Chúng ta hãy nhìn vào dạng sóng của một vài sự kiện.



Hình 2.6: Chuỗi các sự kiện trong D Flip Flop

Dạng sóng trên hiển thị ba chu kỳ xung clock, được biểu thị bằng mũi tên màu đỏ ở cạnh lên của xung clock. Ở giữa cạnh lên thứ nhất và thứ hai của clock, đầu vào D chuyển từ thấp lên cao. Đầu ra Q thấy rằng D đã chuyển từ mức thấp lên mức cao ở cạnh tăng của chu kỳ xung nhịp thứ hai. Tại thời điểm này, Q trở thành cùng giá trị với đầu vào D. Ở cạnh lên thứ ba, Q lại kiểm tra giá trị của D và ghi giá trị của D vào Q (đây là lý do tại sao Flip Flop thường được gọi là thanh ghi). Vì D không thay đổi nên Q vẫn ở mức cao.

2.4.3. DSP

DSP, viết tắt của Digital Signal Processing, đề cập đến việc triển khai các chức năng xử lý tín hiệu số trên FPGA, có các khối phần cứng chuyên dụng để tăng tốc các tác vụ như nhân và lọc. FPGA rất phù hợp cho DSP vì chúng có thể xử lý dữ liệu song song, có cấu hình cao và có thể đạt được độ trễ cũng như hiệu suất năng lượng thấp so với chip DSP truyền thống cho một số ứng dụng nhất định.

Ưu điểm của DSP:

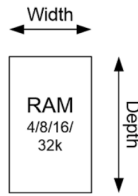
- Hiệu suất cao: FPGA có thể cung cấp độ trễ cực thấp và thông lượng cao nhờ kiến trúc song song của chúng.
- Hiệu quả sử dụng năng lượng và tài nguyên: Việc triển khai DSP trên FPGA có thể tiết kiệm năng lượng và tài nguyên hơn đối với một số công việc nhất định như multiplier, add, subtract, counter,...
- Độ tin cậy: FPGA có thể cung cấp mức độ cách ly cao giữa các tác vụ, điều này có thể đơn giản hóa việc xác minh và bảo trì.

DSP thường được sử dụng trong:

- Khối DSP chuyên dụng: Các FPGA hiện đại chứa các khối phần cứng chuyên dụng được tối ưu hóa cho các hoạt động DSP, chẳng hạn như bộ nhân, bộ tích lũy và bộ cộng.
- Xử lý song song: FPGA có thể ánh xạ toàn bộ thuật toán tới cấp độ phần cứng, cho phép thực thi song song hàng loạt với tốc độ rất nhanh, trái ngược với tính chất tuần tự của DSP dựa trên phần mềm.
- Tính linh hoạt và tùy chỉnh: Phần cứng có thể được cấu hình lại cho các ứng dụng cụ thể, mang lại sự cân bằng giữa hiệu suất và việc sử dụng tài nguyên. Ví dụ: bạn có thể tối ưu hóa thông lượng hoặc hiệu quả tài nguyên bằng cách điều chỉnh thiết kế.
- Các ứng dụng phổ biến: FPGA được sử dụng cho các tác vụ DSP phức tạp như Biến đổi Fourier nhanh (FFT), bộ lọc đáp ứng xung hữu hạn (FIR) và các hoạt động lọc kỹ thuật số khác.

2.4.4. BRAM

BRAM là viết tắt của Block Random Access Memory. Đó là một loại bộ nhớ trên chip được nhúng trong FPGA. BRAM được tổ chức thành các



Hình 2.7: Block RAM

khối (block) có thể được cấu hình để lưu trữ dữ liệu và thường được sử dụng để truy cập bộ nhớ nhanh, hiệu suất cao trong kết cấu FPGA.

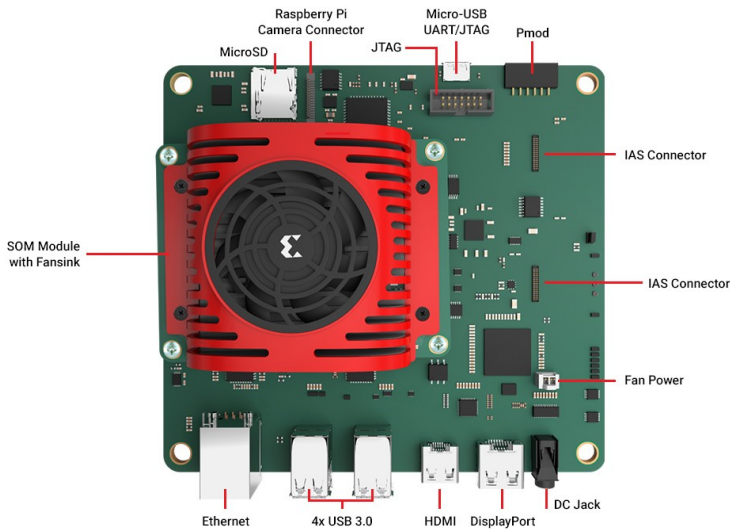
Đặc điểm chính của BRAM.

- Kích thước và cấu trúc cố định. BRAM có dạng khối có kích thước cố định (ví dụ: 18Kb hoặc 36Kb trong Xilinx FPGA). Chúng có thể được cấu hình dưới dạng bộ nhớ một cổng hoặc hai cổng.
- Tốc độ cao và độ trễ thấp. Vì BRAM được nhúng trong kết cấu FPGA nên chúng cung cấp khả năng truy cập bộ nhớ rất nhanh và có thể dự đoán được, điều này rất quan trọng đối với các ứng dụng nhạy cảm với thời gian.
- BRAM có thể được cấu hình. Ta có thể điều chỉnh độ rộng (số bit trên mỗi từ) và độ sâu (số từ) của bộ nhớ. Ngoài ra, BRAM có thể được cấu hình các chế độ như RAM, ROM hoặc thậm chí là FIFO (First-In-First-Out).
- Vận hành đồng bộ. BRAM hoạt động đồng bộ với xung clock của FPGA, đảm bảo truy cập dữ liệu được phối hợp.
- Khả năng cổng kép. Nhiều BRAM hỗ trợ hoạt động cổng kép, cho phép hai hoạt động đọc/ghi độc lập đồng thời, điều này có lợi cho việc xử lý song song.

Nhìn chung, Block RAM đóng một vai trò quan trọng trong hiệu suất và chức năng của FPGA, cho phép chúng lưu trữ và xử lý dữ liệu một cách hiệu quả cho nhiều ứng dụng, từ viễn thông và mạng đến xử lý hình ảnh và tín hiệu. Tính linh hoạt và tốc độ của nó làm cho nó trở thành một nguồn tài nguyên quan trọng cho các nhà thiết kế đang tìm cách triển khai các hệ thống hiệu suất cao và độ trễ thấp bằng cách sử dụng FPGA.

2.5. TỔNG QUAN VỀ NỀN TẢNG PHẦN CỨNG KRIA KV260

Để đáp ứng yêu cầu tính toán hiệu năng cao cho thuật toán mã hóa hậu lượng tử HQC, đề tài lựa chọn sử dụng bộ công cụ phát triển Kria KV260 Vision AI Starter Kit của hãng Xilinx (nay là AMD). Đây là nền tảng được thiết kế chuyên biệt cho các ứng dụng tăng tốc phần cứng tại biên (edge computing), kết hợp giữa tính linh hoạt của phần mềm và sức mạnh xử lý song song của FPGA.



Hình 2.8: Board mạch Kria KV260

2.5.1. KIẾN TRÚC KRIA K26 SYSTEM-ON-MODULE (SOM)

Trái tim của bộ Kria KV260 là module tính toán Kria K26 SOM [24]. Khác với các dòng bo mạch phát triển truyền thống (như ZedBoard hay Zybo), Kria K26 được thiết kế dưới dạng module tích hợp sẵn (System-on-Module), chứa toàn bộ các thành phần cốt lõi như vi xử lý, bộ nhớ và quản lý nguồn [24].

Thành phần trung tâm của K26 SOM là chip SoC Zynq UltraScale+ MPSoC (mã XCK26-SFVC784-2LV-C). Kiến trúc của chip này được chia làm hai phần chính: Hệ thống xử lý (Processing System - PS) và Logic lập trình được (Programmable Logic - PL).

1. Hệ thống xử lý (Processing System - PS)

2

Khối PS đóng vai trò điều khiển chung, chạy hệ điều hành (Linux) và thực hiện các tác vụ giao tiếp mạng hoặc xử lý tuần tự [25]:

- **Application Processor Unit (APU):** Gồm 4 nhân ARM Cortex-A53 64-bit hoạt động ở xung nhịp lên đến 1.5 GHz. Đây là nơi thực thi các phần mềm điều khiển và driver giao tiếp với phần cứng.
- **Real-time Processor Unit (RPU):** Gồm 2 nhân ARM Cortex-R5F 32-bit hoạt động ở xung nhịp 600 MHz, dùng cho các tác vụ thời gian thực (nếu cần).
- **Bộ nhớ:** SOM tích hợp sẵn 4GB bộ nhớ DDR4 (64-bit) tốc độ 2400 Mbps, cung cấp băng thông lớn cho cả CPU và FPGA truy xuất dữ liệu [24].

2. Logic lập trình được (Programmable Logic - PL)

Đây là vùng tài nguyên FPGA dùng để hiện thực hóa các mạch phần cứng tăng tốc cho HQC. Với công nghệ 16nm FinFET+, chip XCK26 cung cấp nguồn tài nguyên dồi dào [24; 25]:

- **System Logic Cells:** 256,200 cells (tương đương khoảng 1.2 triệu cổng logic ASIC), cho phép triển khai các mạch số phức tạp.
- **CLB Flip-Flops:** 234,240 (dùng để lưu trạng thái mạch).
- **CLB LUTs (Look-up Tables):** 117,120 (dùng để thực hiện các hàm logic tổ hợp).
- **Block RAM (BRAM):** 144 khối (tổng dung lượng 5.1 Mb) và **UltraRAM:** 64 khối (tổng dung lượng 18 Mb). Lượng bộ nhớ nội này cực kỳ quan trọng để lưu trữ các đa thức lớn của HQC ngay trong chip, giảm độ trễ truy cập DRAM bên ngoài.
- **DSP Slices:** 1,248 khối DSP48E2. Đây là tài nguyên quan trọng để thực hiện các phép nhân đa thức và biến đổi số học tốc độ cao trong thuật toán HQC.

2.5.2. BO MẠCH ĐỆM (CARRIER CARD) VÀ GIAO TIẾP

Module K26 SOM được gắn trên bo mạch đệm (Carrier Card) của bộ KV260 để cung cấp các cổng kết nối vật lý:

- **Giao tiếp dữ liệu:** Hỗ trợ 4 cổng USB 3.0/2.0 và 1 cổng Ethernet 1Gbps, cho phép truyền nhận dữ liệu tốc độ cao giữa bo mạch và máy tính chủ (Host PC).

- **Giao tiếp mở rộng:** Bo mạch cung cấp giao diện Pmod 12-chân, cho phép kết nối với các module ngoại vi đơn giản hoặc dùng để đo đặc tính hiệu logic trong quá trình gỡ lỗi (debug).
- **Nguồn điện:** Sử dụng nguồn 12V-3A, với các mạch ổn áp tích hợp để cấp nguồn cho SOM và các ngoại vi.

2

2.6. NỀN TẢNG PHÁT TRIỂN PYNQ

Để đơn giản hóa quá trình giao tiếp giữa hệ thống xử lý (PS) và logic lập trình được (PL), cũng như hỗ trợ việc kiểm thử và đánh giá hiệu năng thuật toán HQC một cách trực quan, đề tài sử dụng nền tảng PYNQ (Python Productivity for Zynq) do Xilinx phát triển [25].

2.6.1. TỔNG QUAN VỀ PYNQ

PYNQ là một dự án mã nguồn mở nhằm mục đích mang lại lợi ích của logic lập trình được (FPGA) và vi xử lý (Zynq MPSoC) đến với các nhà thiết kế phần mềm và hệ thống nhúng thông qua ngôn ngữ lập trình Python. Thay vì phải viết các trình điều khiển (drivers) phức tạp bằng ngôn ngữ C ở tầng kernel để giao tiếp với phần cứng, PYNQ cho phép lập trình viên truy xuất tài nguyên FPGA trực tiếp từ Python thông qua các thư viện và API được tối ưu hóa.

Trên board Kria KV260, PYNQ chạy trên nền hệ điều hành Linux (cụ thể là Ubuntu), cung cấp môi trường Jupyter Notebook dựa trên trình duyệt. Điều này cho phép thực hiện quy trình thiết kế đồng thời (Co-design): phần cứng HQC chạy trên PL, trong khi phần mềm điều khiển, sinh dữ liệu kiểm thử (test vectors) và phân tích kết quả chạy trên PS bằng Python.

2.6.2. KIẾN TRÚC OVERLAY VÀ CƠ CHẾ HOẠT ĐỘNG

Khái niệm cốt lõi của PYNQ là *Overlay*. Trong ngữ cảnh của PYNQ, một thiết kế phần cứng FPGA sau khi được tổng hợp thành bitstream sẽ được đóng gói thành một Overlay. Overlay hoạt động tương tự như một thư viện phần mềm: nó có thể được nạp động vào FPGA (PL) theo yêu cầu của chương trình Python đang chạy trên CPU (PS).

Quy trình hoạt động của PYNQ trong đề tài này dựa trên các lớp trừu tượng hóa phần cứng (Hardware Abstraction Layers):

- **Lớp Vật lý (PL):** Chứa IP Core HQC và các khối giao tiếp tiêu chuẩn (như AXI GPIO, AXI BRAM Controller).

2

- **Lớp Trình điều khiển (Drivers):** PYNQ cung cấp sẵn các driver Python cho các giao thức chuẩn (như `pynq.lib.axigpio` cho tín hiệu điều khiển, `pynq.MMIO` cho truy xuất bộ nhớ trực tiếp).
- **Lớp Ứng dụng (Python/Jupyter):** Người dùng viết mã Python để nạp Overlay, khởi tạo các đối tượng điều khiển, gửi dữ liệu input (khóa, bản tin) xuống phần cứng và đọc kết quả trả về.

2.6.3. GIAO TIẾP DỮ LIỆU TRONG PYNQ

Để đảm bảo hiệu năng cho thuật toán HQC, PYNQ hỗ trợ quản lý bộ nhớ và di chuyển dữ liệu thông qua chuẩn AMBA AXI:

- **Memory Mapped IO (MMIO):** Được sử dụng để đọc/ghi vào các thanh ghi trạng thái hoặc các vùng nhớ BRAM nhỏ. PYNQ ánh xạ địa chỉ vật lý của các ngoại vi trong PL vào không gian nhớ ảo của Linux, cho phép Python truy cập chúng như một mảng dữ liệu.
- **Direct Memory Access (DMA):** (Nếu đề tài có dùng DMA) PYNQ cung cấp lớp `pynq.lib.dma` hỗ trợ cơ chế Zero-copy, cho phép truyền dữ liệu khối lượng lớn (như các đa thức khóa công khai) giữa bộ nhớ DDR (của PS) và bộ đệm FPGA (của PL) với tốc độ cao mà không làm tải CPU.

Việc sử dụng PYNQ giúp rút ngắn đáng kể thời gian phát triển driver, đồng thời tận dụng được hệ sinh thái thư viện khoa học mạnh mẽ của Python (như NumPy để xử lý mảng dữ liệu mật mã) để đối sánh kết quả tính toán của phần cứng.

2.7. CÁC CÔNG TRÌNH NGHIÊN CỨU LIÊN QUAN

2.7.1. EFFICIENT HARDWARE IMPLEMENTATION OF CONSTANT TIME SAMPLING FOR HQC [1]

Công trình nghiên cứu này tập vào tối ưu thuật toán sinh mẫu ngẫu nhiên (sampling) trong hệ mã HQC-KEM, vốn là một trong những nút thắt lớn về hiệu năng và bảo mật phần cứng của HQC. Trong HQC, bước sinh các đa thức ngẫu nhiên có trọng số cố định (fixed-weight vectors) là thành phần quan trọng, nhưng phiên bản trước của thuật toán này dễ bị tấn công kênh bên (timing side-channel attacks) và tiêu tốn nhiều tài nguyên phần cứng. Các tác giả đề xuất ba cải tiến chính:

- **Giảm độ phức tạp tính toán:** Ở thuật toán gốc, các chỉ số “1” trong đa thức được lưu ở dạng *danh sách vị trí* (support representation),

khiến mỗi giá trị mới phải so sánh với toàn bộ các phần tử trước đó, dẫn đến độ phức tạp $\mathcal{O}(\omega^2)$. Trong thiết kế mới, các tác giả chuyển sang lưu trữ ở dạng *tường minh* (explicit representation), cho phép chỉ cần kiểm tra trực tiếp một bit tại vị trí được chọn, qua đó giảm độ phức tạp xuống còn $\mathcal{O}(\omega)$.

- **Tăng thông lượng:** Nhằm tăng thông lượng phần cứng, nhóm tác giả đề xuất cải tiến thuật toán sinh mẫu hỗ trợ *kỹ thuật đường ống* (pipelining), ví dụ như cho phép xử lý song song nhiều bước sinh tọa độ trong cùng một chu kỳ. Ở thuật toán gốc, hai vòng lặp *mod_loop* và *unique_check_loop* có phụ thuộc dữ liệu: vòng lặp đầu cần phần tử đầu tiên của mảng *randomwords*, trong khi vòng lặp sau cần phần tử cuối, khiến việc pipeline hóa không khả thi. Để khắc phục, nhóm nghiên cứu đảo ngược thứ tự sử dụng của mảng *randomwords*, sao cho giá trị ngẫu nhiên đầu tiên do PRNG (SHAKE) sinh ra được gán cho phần tử cuối *randomwords* [$\omega - 1$] thay vì phần tử đầu. Nhờ vậy, hai vòng lặp được hợp nhất thành một vòng duy nhất, cho phép pipeline hóa hiệu quả và loại bỏ nhu cầu lưu trữ tạm toàn bộ mảng trong BRAM. Việc xử lý trực tiếp trên luồng dữ liệu ngẫu nhiên sinh tuần tự từ SHAKE giúp giảm đáng kể độ trễ và tăng tốc quá trình lấy mẫu. Tuy nhiên, kỹ thuật này làm thay đổi chuỗi giá trị sinh ra từ cùng một seed, do đó mất khả năng tương thích với mã nguồn tham chiếu ban đầu của HQC và cần điều chỉnh lại đặc tả thuật toán.
- **Giảm yêu cầu bộ nhớ:** Để giảm lượng tài nguyên bộ nhớ, nhóm tác giả đề xuất phương pháp hợp nhất hai quá trình. Theo đó, đa thức x được lưu trong *RAM0*, trong khi y được sinh và lưu song song ở hai dạng: biểu diễn tường minh trong *RAM1* và biểu diễn hỗ trợ (support representation) trong *RAM2*. Sau khi sinh xong y , vùng nhớ *RAM1* được tái sử dụng để lưu đa thức h , còn kết quả z được tính và lưu trực tiếp vào *RAM0*, với từng kết quả trung gian được giảm mô-đun theo $X^n - 1$. Cách tiếp cận này mang lại bốn lợi ích chính: (1) loại bỏ nhu cầu thêm một RAM cho việc lưu biểu diễn tường minh của y (2) tận dụng trực tiếp *RAM0* để lưu x và kết quả z (3) giảm một nửa dung lượng bộ nhớ cần thiết cho z và (4) cho phép gộp bước cộng x vào giai đoạn nhân đa thức bằng cách nạp trước x thay vì giá trị 0 trong *RAM0*.

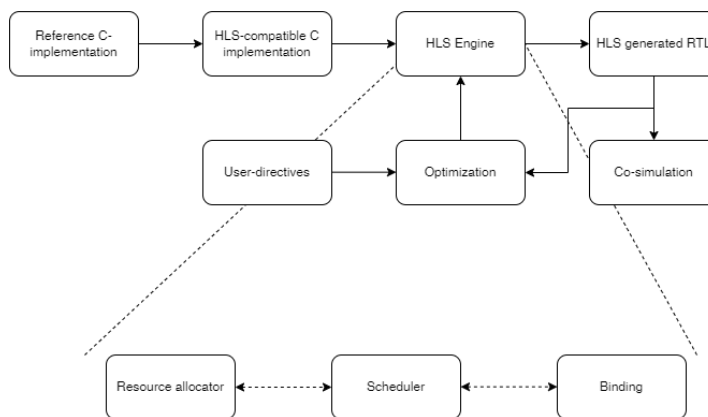
Kết quả cho thấy thiết kế tối ưu mang lại hiệu quả vượt trội so với các phiên bản trước. Cụ thể, độ trễ của khối sinh mẫu được giảm tới 24 lần

2

so với thuật toán gốc. Nhờ đó, tốc độ thực thi toàn hệ thống — ví dụ với HQC-128 trong giai đoạn *KeyGen* — tăng hơn 5 lần, chỉ còn 46 μs so với 269 μs của thiết kế ban đầu. Bên cạnh hiệu năng, hệ thống vẫn duy trì tính an toàn trước các tấn công kênh kề nhờ cơ chế hoạt động thời gian hằng (*constant-time design*). Những cải tiến này giúp rút ngắn đáng kể khoảng cách hiệu năng giữa HQC và Kyber, từ 10.5 $\times$ xuống chỉ còn 1.7 $\times$.

2.7.2. TOWARDS AUTOMATING CRYPTO HW-IMPLEMENTATION A CASE STUDY OF HQC [2]

Công trình này tập trung vào việc tự động hóa thiết kế phần cứng cho thuật toán mã hóa hậu lượng tử Hamming Quasi-Cyclic. Thay vì lập trình tay ở mức RTL như các nghiên cứu truyền thống, nhóm tác giả sử dụng High-Level Synthesis (HLS) – một phương pháp cho phép chuyển đổi mã C/C++ sang ngôn ngữ mô tả phần cứng (VHDL/Verilog), giúp rút ngắn thời gian phát triển và dễ dàng thử nghiệm các cấu hình khác nhau.



Hình 2.9: Tổng quan đơn giản hóa cao độ về quy trình HLS[2]

Trong nghiên cứu, nhóm hiện thực ba khối chính của HQC-KEM gồm *KeyGen*, *Encapsulation* và *Decapsulation* ở mức bảo mật HQC-128 trên nền FPGA **Xilinx Artix-7 xc7a100t-ftg256-1**. Hai phiên bản thiết kế được đề xuất bao gồm:

- **Compact (tiết kiệm tài nguyên)**: ưu tiên giảm sử dụng LUT và BRAM, phù hợp cho thiết bị có tài nguyên hạn chế.
- **Performance (hiệu năng cao)**: ưu tiên tốc độ xử lý và thông lượng.

Nhóm tác giả sử dụng gói mã nguồn chính thức của HQC do nhóm thiết kế thuật toán công bố, bao gồm ba dạng triển khai: phiên bản tham chiếu (reference), phiên bản tối ưu AVX2 chạy thời gian hằng, và phiên bản phần cứng tương thích với High-Level Synthesis (HLS). Trong nghiên cứu, các tác giả tập trung vào thiết kế phần cứng HLS, cho phép mô tả hệ thống bằng ngôn ngữ C/C++ và tự động chuyển đổi sang ngôn ngữ mô tả phần cứng (HDL) như VHDL hoặc Verilog. Phương pháp thiết kế HLS bao gồm ba bước chính: (1) *Resource allocation* – phân bổ hoặc chia sẻ tài nguyên phần cứng (LUT, DSP, BRAM, thanh ghi); (2) *Scheduling* – lập lịch thực thi các phép toán dựa trên phụ thuộc dữ liệu và tài nguyên sẵn có, nhằm khai thác song song tối đa; và (3) *Binding* – ánh xạ các biến và phép toán sang các khối phần cứng cụ thể để tối ưu hóa độ trễ và diện tích. HQC được hiện thực hóa dưới ba hàm chính: `crypto_kem_keygen`, `crypto_kem_enc`, và `crypto_kem_dec`, tương ứng với các giai đoạn tạo khóa, mã hóa, và giải mã trong mô hình KEM (Key Encapsulation Mechanism).

Kết quả đạt được cho thấy thiết kế HLS có thể tiệm cận hiệu năng của các thiết kế viết tay bằng HDL, đồng thời giảm đáng kể thời gian phát triển. Thiết kế HQC-128 đạt tần số hoạt động 125 MHz, độ trễ chỉ khoảng 0.27 ms cho KeyGen, 0.59 ms cho Encapsulation, và 1.27 ms cho Decapsulation trong cấu hình hiệu năng cao. So với các thiết kế HQC RTL trước đó, thiết kế HLS giảm đáng kể lượng bộ nhớ trong FPGA (BRAM) sử dụng, và có khả năng tự động điều chỉnh giữa hiệu năng và tài nguyên mà không cần viết lại toàn bộ mã phần cứng.

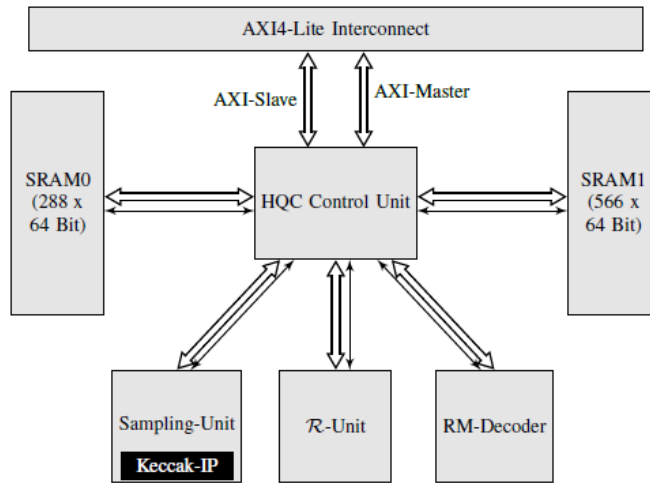
2.7.3. CODE-BASED CRYPTOGRAPHY IN IoT: A HW/SW Co-DESIGN OF HQC [3]

Công trình “Code-based Cryptography in IoT: A HW/SW Co-Design of HQC” nghiên cứu cách tích hợp thuật toán mật mã hậu lượng tử HQC (Hamming Quasi-Cyclic) vào các thiết bị IoT có tài nguyên hạn chế, bằng phương pháp thiết kế đồng thời phần cứng – phần mềm (hardware/software co-design).

Về phương pháp, nhóm tác giả phân tách các khối tính toán nặng của HQC (như nhân đa thức, mã hóa và giải mã Reed–Solomon, Reed–Muller, sinh vector trọng số cố định...) để hiện thực bằng phần cứng trên FPGA, trong khi các phần còn lại được giữ ở phần mềm. Cách tiếp cận này giúp cân bằng giữa hiệu năng, độ trễ và mức tiêu thụ tài nguyên. Họ tối ưu việc chia sẻ tài nguyên phần cứng, song song hóa các khối tính toán, và tối ưu trao đổi dữ liệu giữa CPU và phần cứng để giảm chi phí truyền dữ liệu.

Về kết quả, hệ thống đồng thiết kế đạt được tốc độ cao hơn đáng kể so với triển khai thuần phần mềm, trong khi chỉ sử dụng lượng tài nguyên

FPGA nhỏ, phù hợp với các thiết bị IoT. Cụ thể, độ trễ giảm hơn 10 lần so với phần mềm trên vi xử lý nhúng, trong khi vẫn đảm bảo độ an toàn và khả năng mở rộng cho các cấp bảo mật HQC-128, HQC-192 và HQC-256. Từ kết quả trên cho thấy, cách tiếp cận HW/SW co-design là một hướng khả thi và hiệu quả để triển khai mật mã hậu lượng tử HQC trong môi trường IoT giới hạn tài nguyên.



Hình 2.10: Tăng tốc phần cứng cho HQC[3]

2.7.4. FAST AND EFFICIENT HARDWARE IMPLEMENTATION OF HQC [4]

Tiến sĩ Sanjay Deshpande và các cộng sự đề xuất thiết kế phần cứng tối ưu cho HQC và triển khai thiết kế trên FPGA Xilinx Artix 7. Công trình nghiên cứu tập trung vào việc tối ưu hóa các module chính của HQC-KEM và các building blocks cơ bản, vì chúng chiếm phần lớn tài nguyên phần cứng. Các module cụ thể được đề cập và tối ưu bao gồm:

- **SHAKE256:** Để cân bằng giữa thời gian và diện tích, module áp dụng kiến trúc dual clock domain, nơi SHAKE256 chạy ở tần số thấp hơn (164 MHz) trong khi các module core chạy ở tần số cao hơn, giúp giảm thời gian tổng thể mà không tăng overhead. Ngoài ra, việc chọn giá trị `parallel_slices=16` là để đạt time-area product tốt nhất, dựa vào kết quả synthesis trên FPGA Xilinx Artix 7 xc7a200t.
- **Polynomial Multiplication:** Module được tối ưu hóa bằng cách tận

dụng đặc điểm một trong hai đầu vào là vector thưa (sparse fixed-weight vector) với trọng số cố định để thiết kế một kỹ thuật nhân đa thức thưa (sparse polynomial multiplication) đồng thời kết hợp *interleaved reduction modulo* ($X^n - 1$)

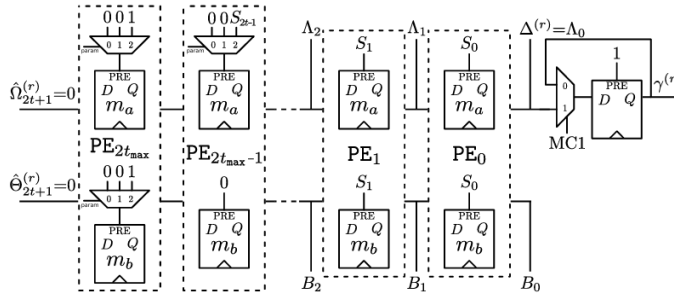
- **Fixed-weight vector generator:** Để tối ưu, SHAKE256 được sử dụng làm bộ sinh số giả ngẫu nhiên từ seed 320-bit, sau đó chia thành các số nguyên 24-bit, thực hiện kiểm tra ngưỡng và giảm modulo n bằng Barrett reduction tối ưu cho hằng số n . Ngoài ra, tham số ACCEPTABLE_REJECTIONS được sử dụng để ép thời gian xử lý luôn cố định, ngay cả khi có ít hoặc nhiều lần bác bỏ.
- **Encode:** Module này được tối ưu hóa bằng cách triển khai hiệu quả mã hóa ghép nối Reed-Solomon và Reed-Muller. Với mã Reed-Solomon, LFSR được dùng để nhân trường Galois, chạy song song số đơn vị nhân bằng bậc đa thức sinh, tạo đầu ra n_1 -bit (368, 448, 720 cho hqc128, hqc192, hqc256). Với mã Reed-Muller, mã hóa vector-ma trận dùng ROM lưu ma trận sinh (G), chọn hàng theo byte đầu vào, và lưu kết quả trong block RAM theo chunk 128-bit. Để tiết kiệm bộ nhớ, chỉ lưu một bản sao mã từ và tính địa chỉ block RAM đạt bội số (3 cho hqc128, 5 cho hqc192, hqc256).
- **Decode:** Module này được tối ưu bằng cách thiết kế riêng cho hai giai đoạn: Reed-Muller và Reed-Solomon. Với Reed-Muller, dữ liệu được gom bằng cách dịch thanh ghi, qua khối Hadamard transform gồm nhiều tầng butterfly radix-2 và xử lý song song trong Find_Peak để tìm vị trí lỗi nhanh chóng. Với Reed-Solomon, các bước tính syndrome, đa thức định vị lỗi, nghiệm và vector lỗi được thực hiện tuần tự bằng phần cứng chuyên biệt, sau đó sửa lỗi trên kết quả từ RM.

Từ những thiết kế tối ưu trên, hệ thống đã đạt được hiệu suất vượt trội so với các thiết kế HLS trước đó và cạnh tranh với các ứng viên khác trong vòng 4 của NIST Post-Quantum Cryptography (BIKE, Classic McEliece, SIKE) cũng như Kyber (chuẩn hóa sau vòng 3). Cụ thể, thiết kế HQC với các biến thể Balanced và HighSpeed, sử dụng domain đơn (SC) hoặc kép (DC), đạt thời gian xử lý ấn tượng: 0.1 ms cho key generation, 0.14 ms cho encapsulation, và 0.23 ms cho decapsulation ở mức bảo mật thấp nhất trên FPGA Xilinx Artix 7, nhanh hơn đáng kể so với HLS (0.3 ms, 0.6 ms, 1.2 ms).

2.7.5. AN EFFICIENT AND UNIFIED RTL ACCELERATOR DESIGN FOR HQC-128, HQC-192, AND HQC-256 [5]

2

Một trong những công trình tiêu biểu là nghiên cứu của nhóm tác giả thuộc Technology Innovation Institute (TII), trong đó đề xuất một kiến trúc phần cứng tối ưu cho HQC với trọng tâm là các khối Reed-Muller encoder/decoder, Reed-Solomon decoder, phép nhân đa thức và SHAKE256. Trong công trình này, nhóm tác giả đã giới thiệu kiến trúc giải mã Reed-Solomon dựa trên thuật toán **ePIBMA (enhanced Parallel Inversionless Berlekamp–Massey Algorithm)**. Đây là phiên bản cải tiến của thuật toán Berlekamp–Massey không sử dụng phép nghịch đảo trực tiếp trong quá trình cập nhật đa thức định vị lỗi (Error Locator Polynomial – ELP).



Hình 2.11: Kiến trúc ePIBMA [5]

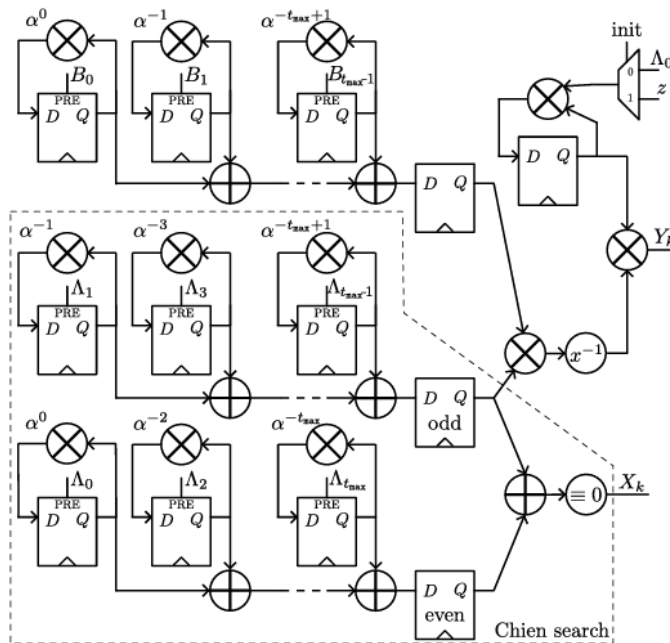
Thuật toán Berlekamp–Massey truyền thống thường yêu cầu phép nghịch đảo trên trường hữu hạn ($GF(2^8)$), gây tốn tài nguyên phần cứng và làm tăng độ trễ xử lý. Để khắc phục vấn đề này, ePIBMA loại bỏ phép chia trực tiếp bằng cách sử dụng các phép nhân và cập nhật song song, giúp kiến trúc phù hợp hơn với FPGA. So với kiến trúc BM truyền thống, ePIBMA có các ưu điểm:

- Giảm số lượng phép nghịch đảo trên trường Galois.
- Tăng khả năng pipeline và xử lý song song.
- Giảm critical path trong phần cứng.
- Cải thiện tần số hoạt động trên FPGA.

Trong thiết kế của nhóm tác giả, ePIBMA được sử dụng để tính đa thức định vị lỗi: $[\sigma(x)]$ trong bộ giải mã Reed-Solomon của HQC. Kiến trúc

được tổ chức theo dạng pipeline nhằm tăng throughput và giảm số chu kỳ xử lý. Bên cạnh các module dưới đây cũng được tối ưu:

- **Reed-Muller encoder:** ăng cách khai thác tính chất lặp của ma trận sinh. Thay vì thực hiện trực tiếp phép nhân vector-ma trận trên dữ liệu 128-bit, thiết kế chia dữ liệu thành các khối 32-bit và tái sử dụng các giá trị XOR trung gian. Nhờ đó số lượng multiplexer và cổng XOR được giảm đáng kể. Sử dụng biến đổi Hadamard nhiều tầng để tận dụng khả năng xử lý song song của FPGA. Kiến trúc Hadamard được pipeline hóa nhằm cải thiện tốc độ xử lý.
- **Sparse polynomial multiplication:** số lượng phép nhân cần thực hiện khi xử lý các đa thức có trọng số thấp. Điều này giúp giảm đáng kể mức sử dụng LUT và DSP trên FPGA.
- **Reed-Solomon decoder:** Công trình tham khảo sử dụng kiến trúc đánh giá đa thức theo từng chu kỳ clock Chien Search nhằm giảm chi phí phần cứng so với việc triển khai song song hoàn toàn.



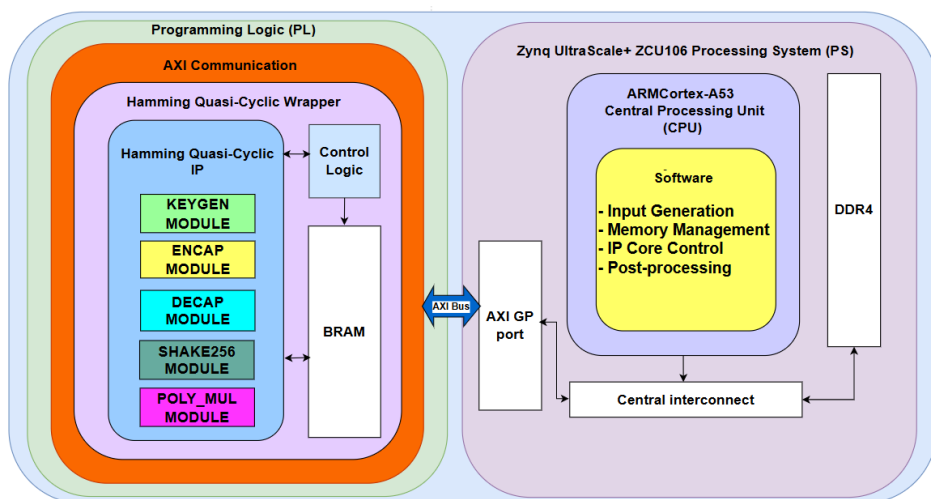
Hình 2.12: Kiến trúc eCSEE [5]

Kết quả thực nghiệm của công trình cho thấy kiến trúc đề xuất đạt hiệu năng cao về throughput và tần số hoạt động.

3

KIẾN TRÚC ĐỀ XUẤT

3.1. KIẾN TRÚC HỆ THỐNG



Hình 3.1: Kiến trúc tổng thể của hệ thống

Hình 3.1 biểu diễn kiến trúc tổng thể của hệ thống, hệ thống này được chia thành 2 thành phần chính:

1. Processing System (PS): Bao gồm ARM Cortex-A53 CPU, DDR4 memory, AXI interfaces, và khối điều khiển trung tâm.

3

- **ARM Cortex-A53 CPU:** Bộ xử lý trung tâm thực hiện lệnh được viết bằng Python thực hiện các tác vụ sau:
 - **Input Generation:** Sinh ra các chuỗi hạt giống (seeds) và bản tin (message) có độ ngẫu nhiên cao. Cụ thể bao gồm: seed_h, seed_kr cho quá trình tạo khóa (KeyGen) và đóng gói (Encap).
 - **Memory Management:** Chịu trách nhiệm quản lý bộ nhớ và đọc/ghi thanh ghi hoặc bộ nhớ.
 - **IP Core Control:** Điều phối luồng hoạt động (Control Flow), đảm bảo sự đồng bộ giữa phần mềm và phần cứng. Thiết lập mã lệnh cho tín hiệu operation để chọn chức năng cần thực thi (00: KeyGen, 01: Encapsulation, 10: Decapsulation). Gửi xung tín hiệu start (mức logic 1) để đánh thức hoạt động của PL.
 - **Post-processing:** Sau khi thu thập dữ liệu từ phần cứng, phần mềm thực hiện các bước xử lý cuối cùng để đánh giá và sử dụng kết quả.
- **DDR4 Memory:** Nơi lưu trữ các dữ liệu vào ra của Hamming Quasi-Cyclic IP.
- **Central Interconnect:** Chịu trách nhiệm điều phối chính của toàn bộ hệ thống SoC, đảm nhiệm chức năng quản lý và điều phối luồng dữ liệu giữa Hệ thống Xử lý (PS) và Logic Lập trình (PL). Trong thiết kế này, nó là trung tâm chuyển mạch hiệu năng cao kết nối bộ xử lý ARM Cortex-A53, bộ nhớ DDR4 và các ngoại vi AXI.
- **AXI GP Port:** Cổng AXI General Purpose (GP) là giao diện giao tiếp đa dụng đóng vai trò cầu nối điều khiển chính giữa PS và PL.

2. Programmable Logic (PL): Vùng Logic Lập trình (PL) được cấu hình để thực thi các tác vụ tính toán chuyên sâu của thuật toán HQC. Thiết kế phần cứng tập trung vào việc tối ưu hóa hiệu năng tính toán song song và quản lý luồng dữ liệu hiệu quả thông qua các khối chức năng sau:

- **Hamming Quasi-Cyclic Wrapper:** Đây là module HQC_wrapper, đóng vai trò là một lớp giao tiếp (Interface Layer) cực kỳ quan trọng giữa phần mềm và lõi xử lý phần cứng HQC IP. Nó giúp CPU có thể giao tiếp, điều khiển và trao đổi dữ liệu với bộ tăng tốc mã hóa HQC thông qua các thanh ghi.
- **HQC Joint Core:** Lõi tính toán chính tích hợp cả ba chức năng KeyGen, Encapsulation và Decapsulation. Lõi này sử dụng kiến

trúc chia sẻ tài nguyên (Resource Sharing) với các bộ nhân đa thức (Polynomial Multiplier) và bộ băm SHAKE256 dùng chung để tiết kiệm diện tích FPGA.

- **Control Logic:** Khối Logic điều khiển hoạt động của module HQC Wrapper.
- **BRAM (Block RAM Generator):** Khối bộ nhớ dùng để lưu trữ dữ liệu tính toán vào ra từ IP.
- **AXI Communication:** Khối AXI chuẩn hóa để giao tiếp bắt tay (handshake) giữa PS và PL.

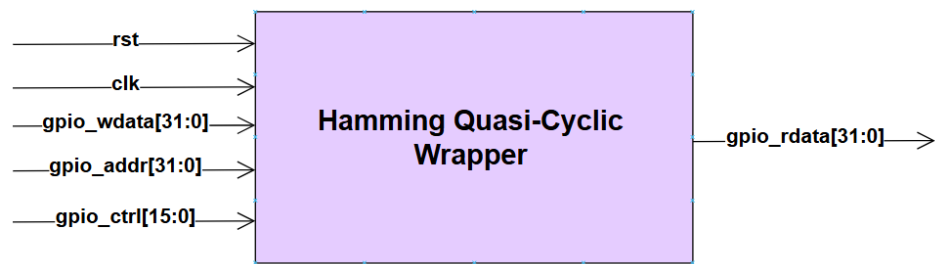
Trong hệ thống đề xuất, quy trình hoạt động bắt đầu bằng việc Hệ thống Xử lý (PS) thực thi kịch bản điều khiển trên nền tảng PYNQ để sinh các tham số khởi tạo và nguồn entropy ngẫu nhiên (Seeds). Các dữ liệu này sau khi được tiền xử lý và định dạng sẽ được ghi vào khối bộ nhớ chia sẻ (Shared BRAM) thông qua cổng giao tiếp AXI General Purpose (AXI-GP). Tiếp theo, PS gửi tín hiệu điều khiển và mã lệnh hoạt động (Operation Code) tới Logic Lập trình (PL) thông qua giao diện AXI GPIO, kích hoạt module HQC Wrapper. Dựa trên các tín hiệu này, Wrapper sẽ điều phối lõi HQC Joint IP Core thực hiện một trong ba tác vụ mật mã: Tạo khóa (KeyGen), Đóng gói (Encapsulation) hoặc Mở gói (Decapsulation). Trong quá trình này, IP Core tự động truy xuất các đa thức lớn từ BRAM và sử dụng các tài nguyên tính toán chia sẻ như Bộ nhân đa thức và SHAKE256 để xử lý dữ liệu. Khi quá trình tính toán hoàn tất, kết quả (như Khóa công khai, Bản mã hoặc Khóa chia sẻ) được ghi ngược lại vào Shared BRAM và một tín hiệu ngắt (Done interrupt) được gửi về PS. Cuối cùng, PS đọc kết quả thông qua giao diện MMIO.

3.2. HAMMING QUASI-CYCLIC WRAPPER

3.2.1. GIAO DIỆN GIAO TIẾP

Hình 3.2 mô tả giao diện điều khiển của module HQC Wrapper đóng vai trò là một lớp giao tiếp trung gian giữa lõi IP và hệ thống bus của bộ xử lý. Mục tiêu chính của Wrapper là biến các giao thức bus phức tạp thành các tín hiệu điều khiển và dữ liệu đơn giản hơn, giúp IP lõi không cần phải thiết kế quá phức tạp để tương thích trực tiếp với bus hệ thống.

3.2.2. MÔ TẢ CÁC TÍN HIỆU



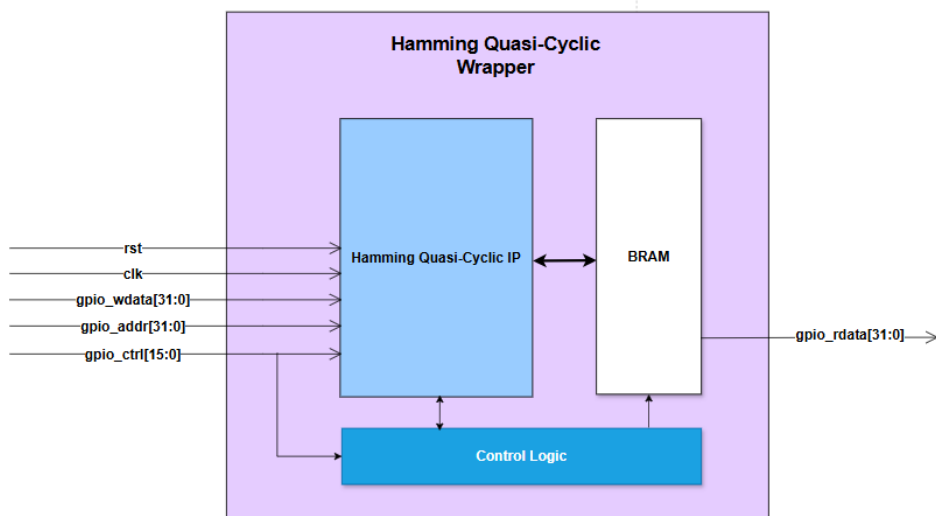
Hình 3.2: Giao diện giao tiếp của HQC Wrapper

Bảng 3.1: Bảng đặc tả tín hiệu giao tiếp của HQC Wrapper

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
Wrapper Interfaces			
clk	In	1	Xung nhịp hệ thống đồng bộ.
rst	In	1	Tín hiệu Reset.
gpio_ctrl	In	16	Control: Các bit tín hiệu điều khiển để ra lệnh cho IP.
gpio_addr	In	32	Address: Bus địa chỉ (32-bit) để xác định thanh ghi (register) cần truy cập bên trong lõi IP.
gpio_wdata	In	32	Write Data: Bus chứa dữ liệu ghi (32-bit) từ bus hệ thống (AXI) gửi vào HQC IP
gpio_rdata	Out	32	Read Data: Bus chứa dữ liệu đọc (32-bit) được xuất từ HQC IP trả về cho bus hệ thống

Bảng 3.1 mô tả các tín hiệu tích hợp vào các hệ thống trên chip (SoC), module HQC được bao bọc bởi một lớp Wrapper với các giao tiếp bằng thanh ghi. Các tín thanh ghi giao tiếp này cho phép bộ điều khiển trung tâm hoặc vi xử lý có thể cấu hình, truyền nhận dữ liệu và giám sát trạng thái của lõi xử lý mật mã một cách hiệu quả.

3.2.3. SƠ ĐỒ KHỐI



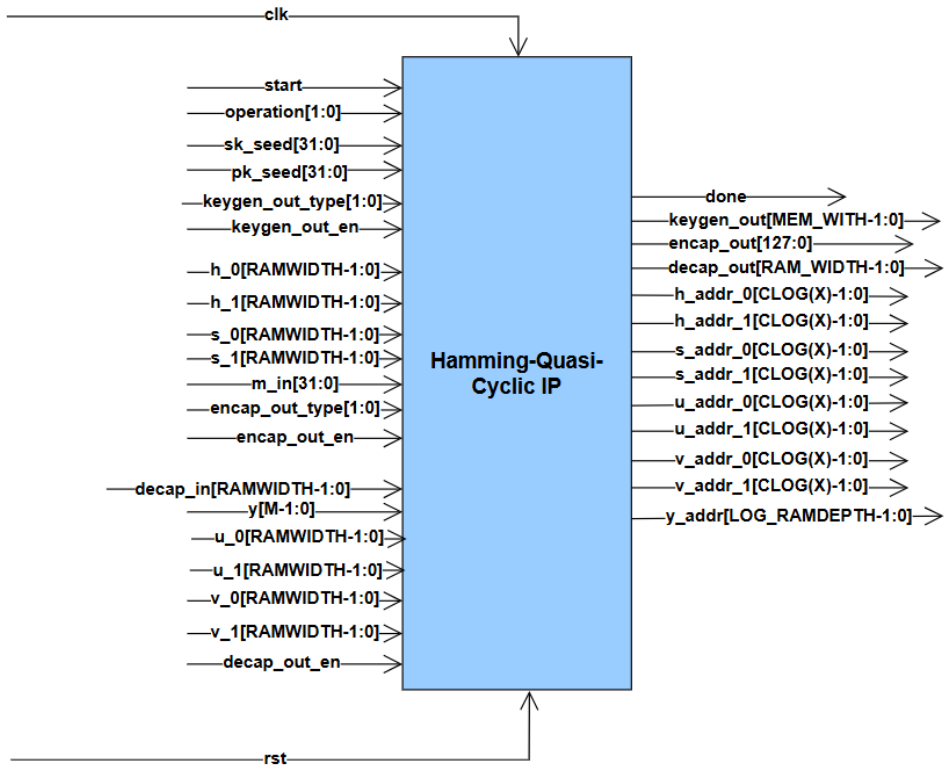
Hình 3.3: Sơ đồ khối HQC Wrapper Architecture

Hình 3.3 biểu diễn Hamming Quasi-Cyclic Wrapper đóng vai trò là lớp giao diện trung gian, thực hiện chức năng cầu nối giữa lõi tính toán IP HQC và hệ thống bộ nhớ bên ngoài (Shared BRAM). Kiến trúc của Wrapper bao gồm ba thành phần chính:

- **Control Logic:** Đây là bộ não của Wrapper, chịu trách nhiệm điều phối toàn bộ hoạt động của IP giao tiếp đọc ghi dữ từ BRAM.
- **BRAM:** Đây là nơi chứa toàn bộ dữ liệu tính toán của HQC-IP.
- **Hamming Quasi-Cyclic IP:** Đây là instance của lõi tính toán Joint Design đã được thiết kế, thực hiện các thuật toán mật mã **KeyGen**, **Encapsulation** và **Decapsulation**.

3.3. HAMMING QUASI-CYCLIC IP

3.3.1. GIAO DIỆN GIAO TIẾP



Hình 3.4: Giao diện kết nối của HQC IP

Hình 3.4 biểu diễn giao diện kết nối (Interfaces) của Hamming Quasi-Cyclic IP (HQC IP) đóng vai trò là nhân tính toán mật mã trung tâm (Cryptographic Kernel) của toàn bộ hệ thống. Đây là một khối phần cứng tùy chỉnh (Custom Hardware Accelerator) được thiết kế để thực thi các thuật toán toán học HQC. Đặc điểm của kiến trúc này:

- **Thiết kế tích hợp (Joint Architecture):** Thay vì phân tách thành ba module vật lý riêng biệt, IP này tích hợp cả ba chức năng Tạo khóa (KeyGen), Đóng gói (Encapsulation) và Mở gói (Decapsulation) trong một khối duy nhất. Kiến trúc này cho phép chia sẻ tối đa tài nguyên phần cứng (như các bộ nhân đa thức và lõi SHAKE256) giữa các chế độ hoạt động, giúp giảm diện tích chiếm dụng trên FPGA.
- **Cơ chế truy cập bộ nhớ chủ động (Master Read Access):** Đối với

các dữ liệu kích thước lớn nằm ở bộ nhớ ngoài (như Public Key h , Secret Key s , Ciphertext u, v), IP được thiết kế các cổng địa chỉ đầu ra ($h_addr, s_addr...$) để chủ động yêu cầu dữ liệu từ BRAM.

- **Băng thông kép (Dual-Port Bandwidth):** Giao diện đọc dữ liệu lớn được thiết kế song song (ví dụ: h_0 và h_1), tận dụng đặc tính Dual-Port của BRAM để nạp 256-bit dữ liệu trong một chu kỳ xung nhịp, giảm thiểu độ trễ cho bộ nhân đa thức.

3.3.2. MÔ TẢ CÁC TÍN HIỆU

Bảng 3.2: Bảng đặc tả tín hiệu giao tiếp của HQC IP Core

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
1. Global Control Interface			
clk	In	1	Xung nhịp hệ thống đồng bộ.
rst	In	1	Tín hiệu Reset.
start	In	1	Xung kích hoạt bắt đầu tính toán.
operation	In	2	Chọn chế độ: 00(KeyGen), 01(Encap), 10(Decap).
done	Out	1	Báo hiệu quá trình xử lý hoàn tất.
2. Key Generation Interface			
sk_seed, pk_seed	In	32	Dữ liệu Seed đầu vào cho khóa bí mật/công khai.
keygen_out	Out	128	Dữ liệu đầu ra KeyGen (x, y, h, s).
keygen_out_type	In	2	Chọn loại dữ liệu: 00(x), 01(y), 10(h), 11(s).
keygen_out_en	In	1	Tín hiệu cho phép đọc kết quả.
3. Encapsulation Interface			
m_in	In	32	Bản tin m đầu vào cần mã hóa.
encap_out	Out	128	Kết quả đầu ra (Ciphertext u, v).
encap_out_type	In	2	Chọn đọc u hoặc v .
encap_out_en	In	1	Cho phép đọc kết quả Encap.
4. Decapsulation Interface			
decap_in	In	Var	Dữ liệu phụ trợ đầu vào.

Tiếp tục ở trang sau...

Bảng 3.2 – tiếp theo từ trang trước

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
y	In	Var	Vector thừa y (đọc từ bộ nhớ ngoài).
decap_out	Out	128	Shared Secret K sau khi giải mã.
decap_out_en	In	1	Cho phép đọc kết quả.

3

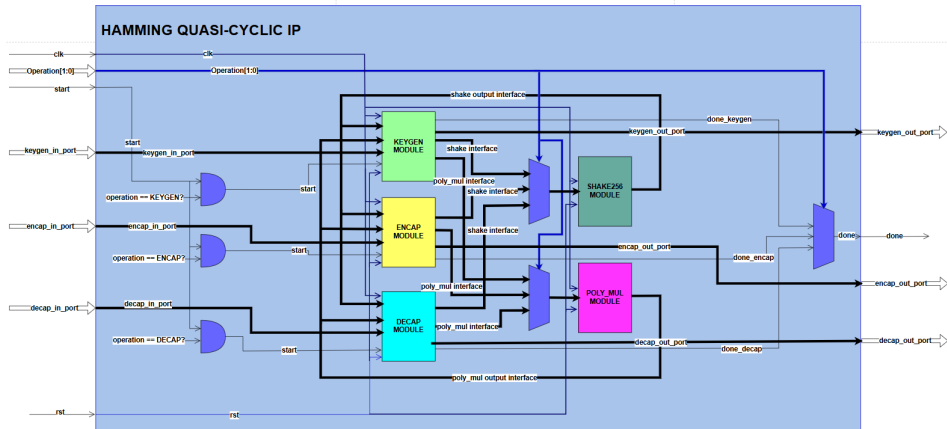
5. External Memory Interface			
h_0, h_1	In	128	Dữ liệu Public Key h (Dual-port Read).
h_addr_0/1	Out	Var	Địa chỉ đọc Public Key.
s_0, s_1	In	128	Dữ liệu Secret Key s (Dual-port Read).
s_addr_0/1	Out	Var	Địa chỉ đọc Secret Key.
u_0/1, v_0/1	In	128	Dữ liệu Ciphertext u, v (Dual-port Read).
u/v_addr_0/1	Out	Var	Địa chỉ đọc Ciphertext.

Bảng 3.2 mô tả chi tiết giao diện mức tín hiệu của bộ tăng tốc phần cứng HQC, bao gồm các tín hiệu điều khiển chế độ, các cổng dữ liệu mật mã và trạng thái hoạt động. Để thực hiện các tác vụ bảo mật, module tiếp nhận các hạt giống ngẫu nhiên (sk_seed, pk_seed) và bản tin đầu vào (m_in) thông qua cơ chế nạp bộ đệm 32-bit. Đặc biệt, thiết kế sử dụng giao diện bộ nhớ ngoài chuyên dụng (External Memory Interface) cho phép IP chủ động truy xuất các đa thức khóa công khai, khóa bí mật và bản mã kích thước lớn từ BRAM chia sẻ. Các tín hiệu đầu ra bao gồm kết quả tạo khóa, đóng gói và mở gói (keygen_out, encap_out, decap_out), cùng với tín hiệu done để báo hiệu quá trình tính toán đã hoàn tất. Giao diện này cho phép tích hợp liền mạch vào hệ thống SoC để xử lý mật mã hiệu năng cao.

3.3.3. SƠ ĐỒ KHỐI

Hình 3.5 minh họa sơ đồ kiến trúc tổng thể của lõi IP Hamming Quasi-Cyclic. Thiết kế áp dụng mô hình Joint Architecture (Kiến trúc tích hợp), hợp nhất cả ba nguyên tắc mật mã (Tạo khóa, Đóng gói, Mở gói) vào một thực thể duy nhất. Đặc điểm của thiết kế này là:

1. Logic điều khiển phân tán (Distributed Control Logic): Thay vì sử



Hình 3.5: Sơ đồ khối của HQC IP

dùng một máy trạng thái trung tâm phức tạp, hệ thống sử dụng cơ chế giải mã lệnh trực tiếp tại đầu vào.

- Dựa trên mã lệnh **Operation[1:0]** và tín hiệu kích hoạt start, khối logic giải mã (được minh họa bằng các cổng AND) sẽ định tuyến tín hiệu kích hoạt đến đúng module chức năng tương ứng (**KEYGEN**, **ENCAP**, hoặc **DECAP**).
- Cơ chế này đảm bảo tại một thời điểm chỉ có một module chức năng được kích hoạt, giúp tối ưu hóa công suất tiêu thụ động của hệ thống.

2. Cơ chế chia sẻ tài nguyên (Resource Sharing Mechanism): Đây là điểm nhấn quan trọng nhất của thiết kế nhằm giảm thiểu diện tích FPGA. Các tác vụ tính toán "nặng" và chiếm nhiều tài nguyên nhất được tách ra thành các khối độc lập dùng chung:

- **Các khối dùng chung (Shared Modules):** Bao gồm bộ nhân đa thức (**POLY_MUL MODULE**) và bộ băm (**SHAKE256 MODULE**).
- **Multiplexing Network (Mạng dồn kênh):** Hệ thống các bộ dồn kênh (Multiplexers) được đặt trước các lối vào của module dùng chung. Tín hiệu Operation[1:0] đóng vai trò là tín hiệu chọn (Selector), cho phép định tuyến dữ liệu và tín hiệu điều khiển từ module chức năng đang hoạt động (ví dụ: KeyGen) tới module dùng chung, trong khi cô lập các module còn lại.

3

- **Vòng phản hồi (Feedback Loop):** Kết quả tính toán từ các module dùng chung (output interface) được phản hồi ngược lại về các module chức năng để tiếp tục quy trình xử lý.

3. Quản lý tín hiệu ngõ ra: Tương tự như luồng dữ liệu, các tín hiệu trạng thái hoàn thành (done_keygen, done_encap, done_decap) cũng được đưa qua một bộ dồn kênh đầu ra để tạo thành một tín hiệu done duy nhất, đơn giản hóa giao diện giao tiếp với bộ điều khiển bên ngoài (Wrapper).

4. Tối ưu hóa vi kiến trúc lõi băm (SHAKE256): SHAKE256 là module dùng chung chịu tải tính toán nặng nhất và được gọi liên tục trong cả 3 quá trình, kiến trúc bên trong của nó được thiết kế chuyên biệt để phá vỡ các nút thắt cổ chai hiệu năng. Trong nghiên cứu này, nhóm đã kế thừa và cải tiến kiến trúc SHAKE hướng lát cắt (slice-oriented) từ thiết kế gốc [26] để tối ưu hóa riêng cho các yêu cầu đặc thù của hệ mật mã HQC. Thiết kế gốc được xây dựng dựa trên **kiến trúc Keccak** theo như **Hình 2.3** có khả năng mở rộng, tập trung vào tính linh hoạt:

- **Kiến trúc hướng lát cắt (Slice-oriented):** Cho phép xử lý song song 2^l lát cắt trong mỗi chu kỳ, giúp cân bằng giữa diện tích chip và hiệu suất.
- **Giao tiếp dựa trên khung dữ liệu (Frame-based):** Sử dụng các khung lệnh (Command), khung độ dài (Length) và khung thông điệp (Message) thông qua giao diện 32-bit.
- **Cơ chế hoạt động:** Module được thiết kế để xử lý các tác vụ băm độc lập; sau mỗi lần hoàn tất việc nén (absorb) và vắt (squeeze), trạng thái nội tại sẽ bị xóa hoặc thiết lập lại cho lần băm tiếp theo.
- **Hạn chế:** Hiệu suất của thiết kế này bắt đầu suy giảm hoặc không ổn định khi tham số song song (p) vượt quá 16 trong môi trường thực thi cụ thể của HQC.

Để đáp ứng yêu cầu khắt khe về thời gian thực thi của HQC, module được cải tiến trọng yếu sau:

- **Tối ưu hóa khả năng song song mức độ cao ($p = 32$):** Tái cấu trúc logic điều khiển để hỗ trợ ổn định mức độ song song lên đến 32 lát cắt. Kết quả thực thi cho thấy tại $p = 32$, tích số Thời gian - Diện tích ($T \times A$) đạt giá trị tối ưu nhất.
- **Cơ chế bảo toàn trạng thái nội tại (State Preservation):** Trong HQC, việc tạo vector trọng số cố định thường đòi hỏi phải lấy thêm dữ

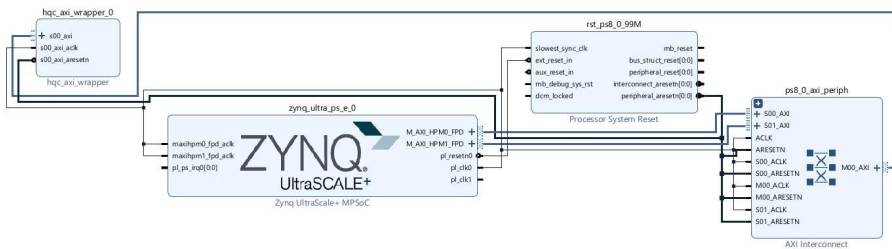
liệu ngẫu nhiên từ cùng một hạt giống (seed) nếu các bit trước đó không thỏa mãn điều kiện. Thay vì khởi động lại toàn bộ quá trình băm (gây lãng phí chu kỳ), module cải tiến cho phép giữ lại trạng thái 1600 bit giữa các lần gọi và tinh chỉnh tín hiệu thoát cưỡng bức (forced exit) để đưa module về trạng thái mặc định khi cần thiết.

4

HIỆN THỰC

4.1. HIỆN THỰC HỆ THỐNG

Tổng thể của hệ thống SoC được biểu diễn trong Hình 4.1 mô tả kết nối giữa Zynq Processing System và Hamming Quasi-cyclic IP. Bao gồm các thành phần như:



Hình 4.1: Hiện thực hệ thống trên Xilinx SoC

- **Zynq UltraScale+ MPSoc (zynq_ultra_ps_e_0):** Bộ xử lý trung tâm, cung cấp giao diện Master AXI cho giao tiếp với Programmable Logic (PL). Chạy phần mềm, điều khiển toàn hệ thống gửi dữ liệu và lệnh xuống phần cứng.
- **Processor System Reset (rst_ps8_0_99M):** Module đồng bộ reset.
 - Input: clock.
 - Input: reset từ PS.

- Output: reset cho AXI interconnect.
- Output: reset cho HQC IP.
- **Hqc_axi_wrapper (hqc_axi_wrapper_0):** Đây là HQC accelerator.
 - s00_axi: IP này là AXI Slave, nhận lệnh từ PS để điều khiển IP.
 - Bên trong hqc_axi_wrapper: HQC Core, Bram, Controller.
- **AXI Interconnect (ps8_0_axi_periph):** Đây là bộ chuyển mạch AXI 1 Master nhiều Slave (IP).
 - S00_AXI $\leftarrow$ nhận từ PS
 - M00_AXI $\rightarrow$ đi tới IP

4.2. HAMMING QUASI-CYCLIC WRAPPER

4.2.1. ĐỒNG BỘ HÓA DỮ LIỆU VÀ PHÂN TÍCH ĐIỀU KHIỂN

Bởi vì HQC IP xử lý với dữ liệu 128-bit trong khi CPU chỉ gửi dữ liệu 32-bit, ta cần cách xử lý cách CPU gửi dữ liệu đến cho IP cụ thể như sau:

- **Chuyển đổi độ rộng dữ liệu đầu vào:** Wrapper này sử dụng một thanh ghi đệm buffer_128 và tín hiệu word_sel để gom 4 lần gửi 32-bit từ CPU thành 1 khối 128-bit trước khi nạp vào các bộ nhớ RAM các thành phần mã hóa/giải mã.
- **Đọc dữ liệu đầu ra:** Tương tự, khi CPU muốn đọc kết quả (Keygen, Encap, Decap), Wrapper sẽ tách khối 128-bit thành các phần 32-bit để trả về qua gpio_rdata[31:0].
- 6 khối bộ nhớ quan trọng sử dụng mem_dual - RAM 2 cổng như **Bảng 4.1**. Cho phép lõi HQC truy cập dữ liệu ở Port 0 để tính toán, trong khi CPU có thể truy cập Port 1 để nạp khóa hoặc đọc kết quả mà không gây xung đột.

Các thanh ghi của Wrapper:

- gpio_addr[15:0]: Address: Bus địa chỉ (32-bit) để xác định thanh ghi (register) cần truy cập bên trong lõi IP.
- gpio_wdata[31:0]: Write Data: Bus chứa dữ liệu ghi (32-bit) từ bus hệ thống (AXI) gửi vào HQC IP
- gpio_rdata[31:0]: Read Data: Bus chứa dữ liệu đọc (32-bit) được xuất từ HQC IP trả về cho bus hệ thống

Bảng 4.1: Các dữ liệu tính toán chứa trong khối bộ nhớ

Khối bộ nhớ	Mô tả
RAM H, S, U, V	Chứa các đa thức và bản mã.
RAM Y	Chứa khóa bí mật (dưới dạng vector thưa).
RAM D	Chứa dữ liệu bổ trợ.

- gpio_ctrl[15:0]: Control: Các bit tín hiệu điều khiển để ra lệnh cho IP.

Để điều khiển hoạt động của IP, CPU cần ghi vào các bit của thanh ghi **gpio_ctrl[15:0]**. Tín hiệu **gpio_ctrl- 16 bit** được giải mã thành các lệnh điều khiển chi tiết cho lõi HQC như Bảng 4.2.

Bảng 4.2: Mô tả thanh ghi điều khiển GPIO

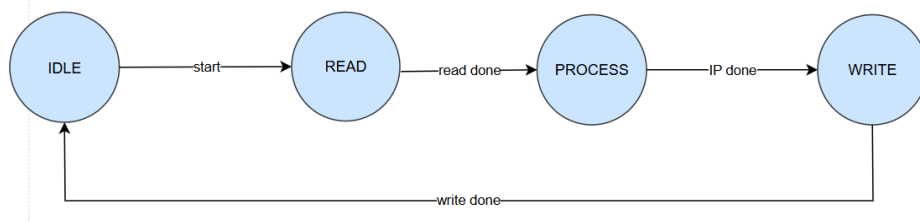
Trường bit	Chức năng
gpio_ctrl[0]	Tín hiệu reset hệ thống.
gpio_ctrl[1]	Tín hiệu bắt đầu thực thi (start).
gpio_ctrl[3:2]	Lựa chọn chế độ hoạt động: • 00 : Sinh khóa (Keygen) • 01 : Mã hóa đóng gói (Encap) • 10 : Giải mã đóng gói (Decap)
gpio_ctrl[9:7] (ram_sel)	Lựa chọn khối bộ nhớ cần thao tác (H, S, U, V, Y hoặc D).
gpio_ctrl[15] (read_src_sel)	Lựa chọn nguồn dữ liệu để đọc: • 0 : Đọc dữ liệu từ RAM • 1 : Đọc trực tiếp từ đầu ra của lõi HQC

Giám sát trạng thái để đọc dữ liệu từ **gpio_rdata[31:0]**. Wrapper theo dõi xung done từ lõi HQC. Khi lõi xong việc, giữ trạng thái này trong done_sticky. CPU có thể kiểm tra xem hệ thống đã xử lý xong chưa bằng

cách đọc địa chỉ đặc biệt **gpio_addr = 32'hFFFFFFF**. Nếu kết quả trả về có bit cuối bằng 1, nghĩa là tác vụ đã hoàn thành và có thể đọc dữ liệu từ Wrapper.

4.2.2. LOGIC ĐIỀU KHIỂN

Hình 4.2 biểu diễn logic điều khiển của HQC Wrapper được quản lý bởi một máy trạng thái hữu hạn (FSM) gồm bốn trạng thái tương tự như quy trình của thuật toán:



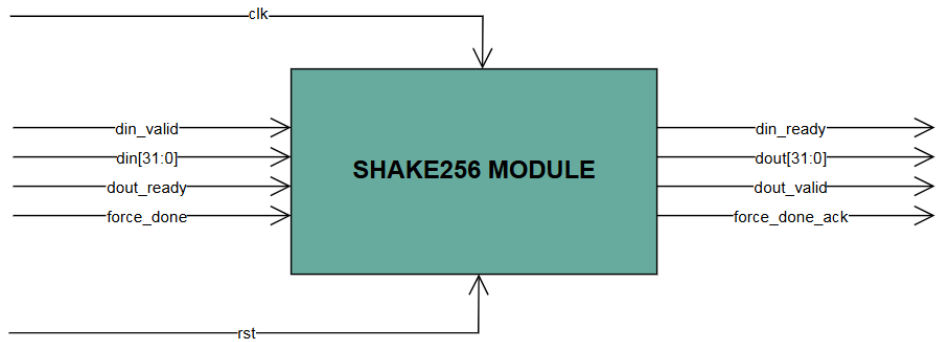
Hình 4.2: HQC Wrapper FSM

- **IDLE:** Hệ thống bắt đầu ở trạng thái IDLE, chờ các tín hiệu điều khiển bên ngoài như start hoặc rst. Khi một trong các tín hiệu này được nhận, FSM sẽ chuyển sang trạng thái READ. Nếu không có tín hiệu nào, FSM sẽ tiếp tục ở trạng thái IDLE.
- **READ:** Trong trạng thái này, FSM đọc dữ liệu từ giao diện bên ngoài (như BRAM/GPIO). Dữ liệu tương ứng với các đầu vào khác nhau như hạt giống khóa (sk_seed, pk_seed) và thông điệp đầu vào (m_in) tùy thuộc vào Operation của thanh ghi Control (gpio_ctrl[15:0]). Khi dữ liệu được đọc đủ, FSM sẽ chuyển sang trạng thái PROCESS.
- **PROCESS:** Sau trạng thái READ, FSM bước vào trạng thái PROCESS, nơi thuật toán lõi HQC được thực thi. Quá trình xử lý các dữ liệu đầu vào thông qua các phép toán (00: Keygen, 01: Encap, 10: Decap) được thực hiện. Khi quá trình xử lý hoàn tất, FSM sẽ chuyển sang trạng thái WRITE.
- **WRITE:** Trong trạng thái này, dữ liệu đã xử lý được ghi trở lại giao diện hoặc xuất ra các cổng đầu ra. FSM sẽ thực hiện ghi dữ liệu, và cờ hoàn thành (done_sticky) được bật lên khi hoạt động ghi hoàn tất. Sau đó, FSM sẽ quay trở lại trạng thái IDLE, sẵn sàng cho các thao tác tiếp theo.

4.3. HAMMING QUASI-CYCLIC IP

4.3.1. SHAKE256

Module SHAKE256 đóng vai trò là bộ tạo số giả ngẫu nhiên (PRNG) then chốt cho các quy trình tạo vector trọng số cố định, tạo khóa và băm dữ liệu. Kiến trúc module dựa trên Keccak Core của Keccak Team [27], xem 2.2.4.



Hình 4.3: Giao diện kết nối của module SHAKE256

Hình 4.3 minh họa sơ đồ giao diện kết nối của module SHAKE256. Module này được thiết kế để hoạt động như một lõi tính toán độc lập, giao tiếp với các module khác (như KeyGen, Encap, Decap) thông qua giao thức bắt tay Ready-Valid (Ready-Valid Handshake Protocol). Bảng 5 mô tả chức năng của từng tín hiệu của module.

Bảng 4.3: Bảng đặc tả tín hiệu giao tiếp của Module SHAKE256

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
1. Global Signals (Tín hiệu hệ thống)			
clk	In	1	Xung nhịp đồng bộ hệ thống.
rst	In	1	Tín hiệu Reset tích cực cao.
2. Data Input Interface (Nạp dữ liệu)			
<i>*Sử dụng giao thức bắt tay Ready-Valid để nạp Seed/Message</i>			
din	In	32	Bus dữ liệu đầu vào (Hạt giống hoặc Bản tin).
din_valid	In	1	Báo hiệu dữ liệu trên din là hợp lệ (Master Valid).

Tiếp tục ở trang sau...

Bảng 4.3 – tiếp theo từ trang trước

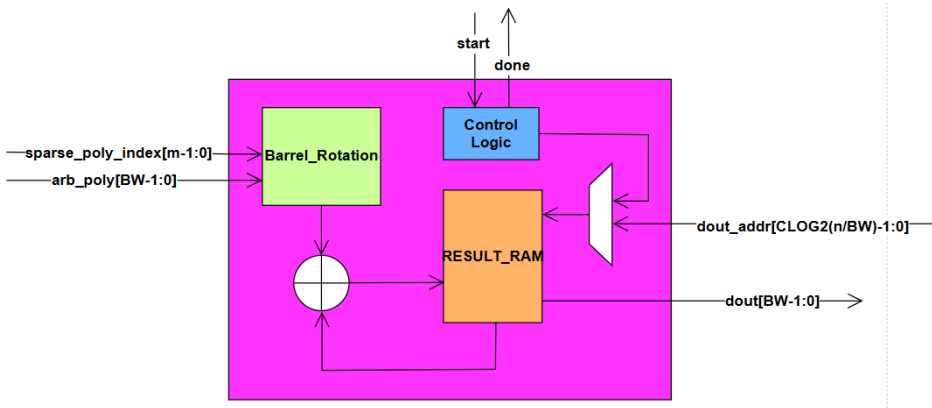
Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
din_ready	Out	1	Báo hiệu SHAKE đã sẵn sàng nhận dữ liệu (Slave Ready).
3. Data Output Interface (Xuất dữ liệu)			
<i>*Trích xuất chuỗi bit ngẫu nhiên hoặc giá trị băm</i>			
dout	Out	32	Bus dữ liệu đầu ra.
dout_valid	Out	1	Báo hiệu dữ liệu dout đã sẵn sàng (Slave Valid).
dout_ready	In	1	Báo hiệu khối đích đã sẵn sàng đọc (Master Ready).
4. State Control Interface (Điều khiển trạng thái)			
<i>*Dùng cho cơ chế Rejection Sampling (Lấy mẫu loại bỏ)</i>			
force_done	In	1	Lệnh ép buộc kết thúc phiên làm việc và xóa trạng thái nội bộ (Soft Reset).
force_done_ack	Out	1	Tín hiệu xác nhận (Acknowledge) đã thực hiện reset thành công.

Bảng 4.3 mô tả các tín hiệu của module SHAKE256 áp dụng giao thức bắt tay Ready-Valid tiêu chuẩn với các tín hiệu **din_valid/din_ready** và **dout_valid/dout_ready** để kiểm soát luồng dữ liệu một cách chính xác, tránh hiện tượng mất mát dữ liệu khi xử lý. Để hỗ trợ cơ chế lấy mẫu loại bỏ (Rejection Sampling), module còn tích hợp giao diện điều khiển trạng thái với cặp tín hiệu **force_done** và **force_done_ack**, cho phép ép buộc kết thúc phiên làm việc và xóa trạng thái nội bộ một cách an toàn. Sự kết hợp giữa các bus dữ liệu 32-bit và các cờ hiệu trạng thái như **read_done** hay **write_done** đảm bảo hệ thống luôn vận hành ổn định và đạt hiệu suất tối ưu trong các phép toán mật mã phức tạp.

4.3.2. POLYNOMIAL MULTIPLICATION

Dựa trên phân tích đặc tả kỹ thuật của HQC-KEM, phép nhân đa thức luôn diễn ra giữa hai đối tượng: một đa thức ngẫu nhiên tùy ý (gọi là **arb_poly** - đa thức dày) và một vector có trọng số Hamming cố định (gọi là **sparse_poly** - đa thức thưa). Do đó, module được thiết kế dựa trên kỹ thuật nhân đa thức thưa (Sparse Polynomial Multiplication) kết hợp với phép giảm modulo xen kẽ (Interleaved Reduction) trên vành $F_2[X]/(X^n -$

1). Hình 4.4 biểu diễn kiến trúc phần cứng của module POLY_MUL, do



Hình 4.4: Sơ đồ khối thiết kế phần cứng của module POLY_MUL

tham số n trong HQC rất lớn, việc sử dụng phương pháp dịch bit dựa trên thanh ghi (register-based) truyền thống sẽ tiêu tốn quá nhiều tài nguyên. Để khắc phục, thiết kế chuyển sang sử dụng kiến trúc dựa trên bộ nhớ:

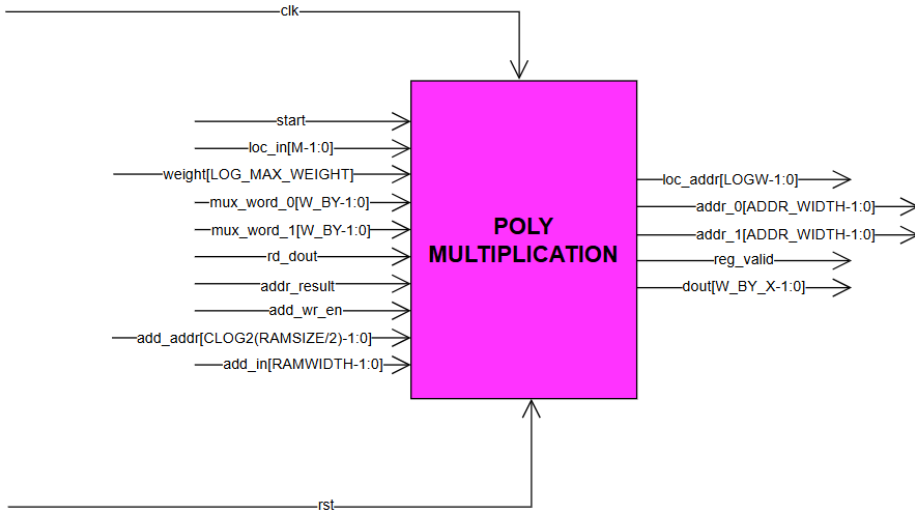
- **Block RAM (BRAM):** Sử dụng BRAM cổng đôi (Dual-port BRAM) để lưu trữ và truy xuất dữ liệu tuần tự.
- **Bộ dịch vòng (Barrel Rotation Unit):** Kết hợp với BRAM để thực hiện phép dịch trái biến thiên tuần tự (Sequential Variable Shift).
- **Tham số BW (Block Width):** Độ rộng của bộ dịch và BRAM được tham số hóa để tối ưu hiệu năng.

Nguyên lý hoạt động:

- **Lưu trữ dữ liệu:** Đa thức dày (arb_poly) được chia nhỏ thành các khối (chunks) có độ rộng bằng BW và nạp tuần tự vào BRAM (từ địa chỉ thấp đến cao, phần dư được đệm thêm số 0). Đa thức thưa (sparse_poly) chỉ lưu trữ các chỉ số (indices) của các phần tử khác 0.
- **Thuật toán Nhân và Modulo (Multiply and Reduce):**
 - Quá trình nhân thực chất là phép dịch trái arb_poly tương ứng với từng chỉ số index của sparse_poly.

- **Interleaved Reduction:** Phép giảm modulo $(X^n - 1)$ được thực hiện xen kẽ ngay trong quá trình dịch. Trong trường F_2 , việc giảm một đa thức $2n$ -bit xuống module $(X^n - 1)$ tương đương với việc cắt phần cao (phần tràn quá n) và thực hiện phép XOR với phần thấp.
- Khi thực hiện dịch trên từng chunk, hệ thống tính toán địa chỉ (bậc) của kết quả. Nếu bậc lớn hơn n , phần dữ liệu đó sẽ được giải mã địa chỉ để XOR ngược lại vào phần thấp của đa thức kết quả.

4



Hình 4.5: Giao diện kết nối của module Poly Mul

Hình 4.5 cung cấp một cái nhìn chi tiết về kiến trúc phần cứng của khối nhân đa thức (Poly Mul), một thành phần cốt lõi trong hệ mật mã HQC được thiết kế với tính linh hoạt và hiệu năng cao. Thông qua các tham số biến đổi như M hay $ADDR_WIDTH$, module cho thấy khả năng tùy biến mạnh mẽ để phù hợp với nhiều bộ tham số bảo mật khác nhau, đồng thời việc tích hợp các bus địa chỉ và dữ liệu kép ($addr_0/1$, $mux_word_0/1$) minh chứng cho khả năng tận dụng bộ nhớ Dual-Port để tối ưu hóa băng thông xử lý. Cơ chế điều khiển của khối được chuẩn hóa thông qua xung kích hoạt $start$ và cờ hiệu reg_valid . Bảng 4.4 đặc tả chi tiết chức năng của từng tín hiệu:

Bảng 4.4: Bảng đặc tả tín hiệu giao tiếp của Module Poly Mul

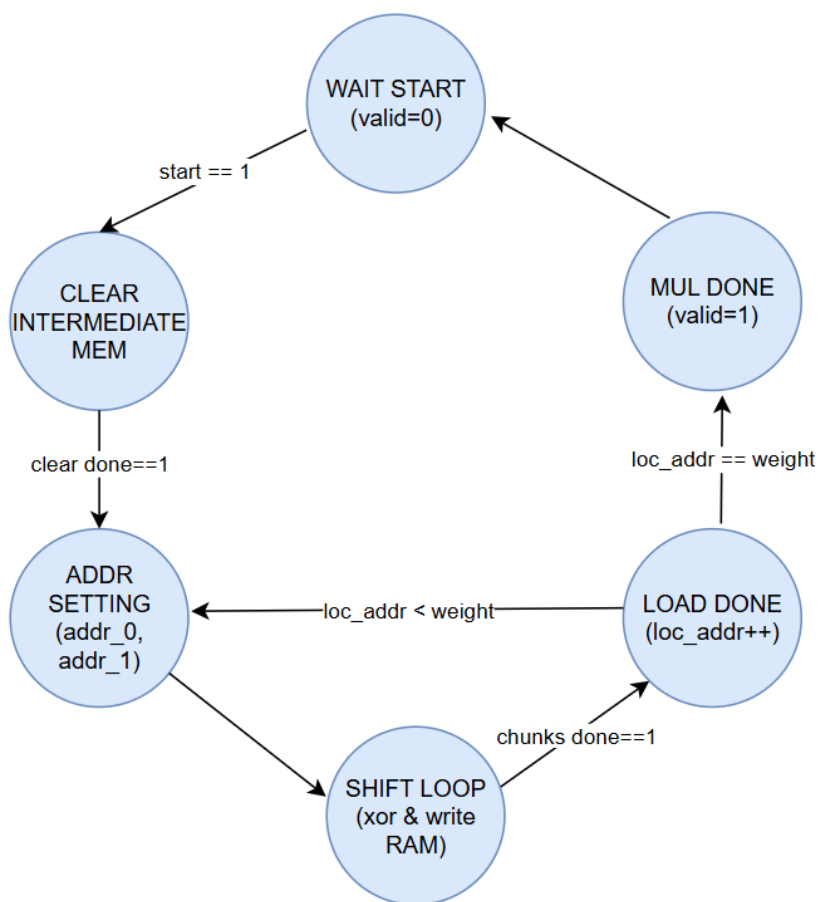
Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
1. Global Signals (Tín hiệu hệ thống)			
clk	In	1	Xung nhịp đồng bộ hệ thống.
rst	In	1	Tín hiệu Reset tích cực cao.
start	In	1	Tín hiệu kích hoạt (xung điều khiển) bắt đầu quá trình nhân đa thức.
2. Nhóm các tín hiệu Input			
loc_in[M-1:0]	In	Var	Bus dữ liệu đầu vào chứa thông tin vị trí (location vector) có độ rộng M bit. Dùng để nạp các vị trí cần thiết cho phép nhân hoặc các vector trọng số trong thuật toán HQC.
weight[LOG_MAX_WEIGHT]	In	Var	Trọng số tối đa.
mux_word_0/1[W_BY-1:0]	In	Var	Các từ dữ liệu được chọn qua bộ dồn kênh (Multiplexer). Nó giúp cấp phát các hệ số đa thức đầu vào từ các nguồn khác nhau vào khối nhân.
rd_dout	In	1	Tín hiệu điều khiển cho phép đọc dữ liệu kết quả ra ngoài.
add_addr[...]	In	Var	Bus địa chỉ dùng để ghi dữ liệu phụ trợ vào RAM.
add_in[RAMWIDTH-1:0]	In	Var	Dữ liệu đầu vào để ghi vào RAM.
add_wr_en	In	1	Tín hiệu cho phép ghi (Write Enable) vào bộ nhớ.
addr_result	In	1	Địa chỉ của kết quả được tính toán bên trong khối, dùng để trở tới vùng nhớ lưu kết quả.
3. Nhóm các tín hiệu Output			
Tiếp tục ở trang sau...			

Bảng 4.4 – tiếp theo từ trang trước

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
loc_addr[LOGW-1:0]	Out	Var	Địa chỉ vị trí đa thức được trích xuất ra, dùng để truy xuất các mảng vị trí trong quá trình nhân.
addr_0/1[ADDR_WIDTH-1:0]	Out	Var	Các bus địa chỉ truy xuất bộ nhớ RAM/ROM. Việc có hai bus addr_0 và addr_1 cho thấy khối này hỗ trợ đọc/ghi đồng thời trên các cổng của bộ nhớ Dual-Port.
reg_valid	Out	1	Cờ báo hiệu (valid) cho biết dữ liệu ở đầu ra đã sẵn sàng và hợp lệ để sử dụng.
dout[W_BY-X-1:0]	Out	Var	Kết quả chính của phép nhân đa thức.

Logic điều khiển được mô tả như trong Hình 4.6

- **WAIT START:** Đây là trạng thái nghỉ của hệ thống. FSM thiết lập lại các bộ đếm count_chunks, cờ valid về 0 và chờ tín hiệu start. Khi nhận được tín hiệu kích hoạt, hệ thống sẽ tính toán các địa chỉ RAM ban đầu dựa trên giá trị dịch (shift) và chuyển sang trạng thái xóa bộ nhớ đệm.
- **CLEAR INTERMEDIATE MEM:** Trong trạng thái này, FSM thực hiện xóa (ghi giá trị 0) vào bộ nhớ trung gian. Việc này đảm bảo kết quả tích lũy của phép nhân không bị ảnh hưởng bởi dữ liệu cũ. Sau khi hoàn tất việc xóa toàn bộ các ô nhớ, FSM chuyển sang **ADDR SETTING**.
- **ADDR SETTING:** FSM thiết lập địa chỉ đọc cho các khối dữ liệu đa thức dựa trên vị trí của các hệ số khác không (được xác định bởi loc_in). Đây là bước chuẩn bị để thực hiện vòng lặp nhân cho từng trọng số (weight) của vector.
- **SHIFT LOOP:** Đây là trạng thái thực thi chính. FSM thực hiện đọc dữ liệu từ bộ nhớ, thực hiện phép dịch bit (dshift) và XOR với giá trị



Hình 4.6: Poly Mul FSM

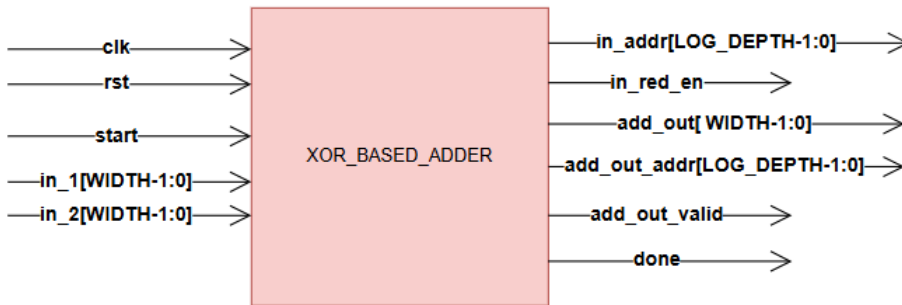
hiện tại trong bộ nhớ trung gian. Quá trình này lặp lại cho đến khi tất cả các phân đoạn (chunks) của đa thức được xử lý.

- **LOAD DONE:** Sau khi xử lý xong một vòng lặp cho một trọng số, FSM kiểm tra xem đã đi qua hết các vị trí yêu cầu (weight) chưa. Nếu còn, nó quay lại **ADDR SETTING** để xử lý hệ số tiếp theo. Nếu đã hoàn tất, FSM chuyển sang trạng thái kết thúc.
- **MUL DONE:** Trạng thái hoàn tất phép nhân. Tín hiệu valid được đẩy lên mức cao để thông báo kết quả đã sẵn sàng trong bộ nhớ đầu ra. Sau đó, FSM quay trở về trạng thái **WAIT START**.

4

4.3.3. XOR_BASED_ADDER

Module xor_based_adder được đặc tả như Hình 4.7, một bộ cộng đa thức trên trường F_2 sử dụng cổng XOR. Module này được thiết kế để hoạt động tuần tự, quét qua bộ nhớ và thực hiện phép cộng từng khối (block). Bảng 4.6 đặc tả chức năng của từng tín hiệu của module:



Hình 4.7: Giao diện kết nối của bộ cộng xor_based_adder

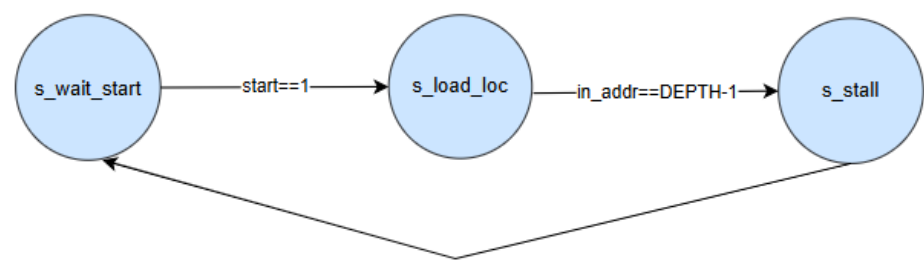
Bảng 4.5: Bảng đặc tả tín hiệu giao tiếp của Module xor_based_adder

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
1. Global Signals (Tín hiệu hệ thống)			
clk	In	1	Xung nhịp đồng bộ hệ thống.
rst	In	1	Tín hiệu Reset tích cực cao.
start	In	1	Xung kích hoạt bắt đầu quá trình cộng.
Tiếp tục ở trang sau...			

Bảng 4.5 – tiếp theo từ trang trước

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
2. Data Input Interface (Nạp dữ liệu)			
in_1 / in_2	In	WIDTH	Dữ liệu từ 2 nguồn cần cộng (nạp từ BRAM bên ngoài).
add_out_addr	In	LOG_DEPTH	Địa chỉ tương ứng để ghi kết quả add_{out} vào BRAM.
3. Data Output Interface (Kết quả)			
in_addr	Out	LOG_DEPTH	Địa chỉ nạp dữ liệu gửi tới BRAM.
in_rd_en	Out	1	Tín hiệu cho phép đọc dữ liệu từ BRAM.
add_out	Out	WIDTH	Kết quả sau khi XOR (in_1 xor in_2).
add_out_valid	Out	1	Cờ báo hiệu dữ liệu đầu ra đang hợp lệ.
done	Out	1	Cờ báo hiệu đã hoàn thành cộng toàn bộ đa thức.

Module này hoạt động như một máy trạng thái (FSM) đơn giản để quét qua toàn bộ không gian bộ nhớ của đa thức.



Hình 4.8: Xor_based_adder FSM

- **s_wait_start (Trạng thái 0):** Module ở trạng thái nghỉ, đợi tín hiệu start. Khi start = 1, nó chuyển sang trạng thái s_load_loc và bắt đầu kích hoạt in_rd_en.
- **s_load_loc (Trạng thái 1):** Đây là trạng thái xử lý chính. Module tự động tăng dần địa chỉ in_addr từ 0 đến DEPTH-1. Với mỗi địa chỉ,

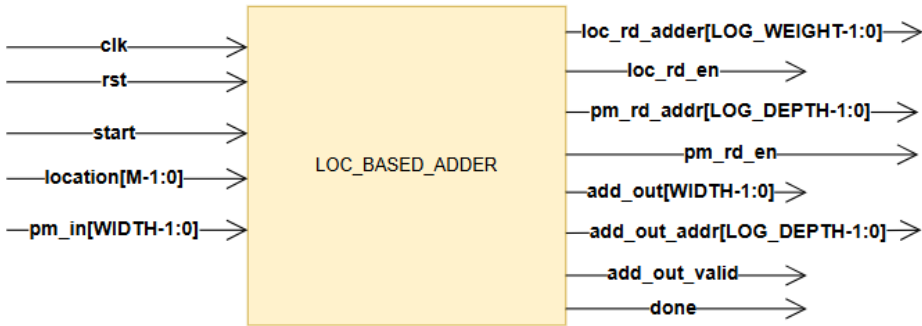
dữ liệu `in_1` và `in_2` được nạp vào và thực hiện phép toán: **`add_out`** $\leq$ **`in_1`** **xor** **`in_2`**. Do đặc điểm của BRAM (thường mất 1 chu kỳ để dữ liệu xuất hiện sau khi gửi địa chỉ), các tín hiệu đầu ra như `add_out_addr` và `add_out_valid` được thiết kế có độ trễ (pipeline) để khớp với dữ liệu đã tính toán.

- **s_stall (Trạng thái 2):** Trạng thái trễ để đợi hoàn tất chu kỳ ghi cuối cùng. Sau đó, cờ `done` được bật lên để báo hiệu kết thúc và quay về trạng thái nghỉ.

4.3.4. LOC_BASED_ADDER

4

Hình 4.9 mô tả giao diện module `loc_based_adder`, một thiết kế tối ưu hóa cực cao cho hệ mật HQC. Khác với bộ cộng XOR quét toàn bộ dữ liệu, module này chỉ tập trung vào các vị trí "thưa" (sparse positions) để thực hiện phép cộng, giúp tiết kiệm thời gian và năng lượng đáng kể. Dưới đây



Hình 4.9: Giao diện kết nối của bộ cộng `loc_based_adder`

là mô tả chi tiết các tín hiệu và nguyên lý vận hành của nó:

Bảng 4.6: Bảng đặc tả tín hiệu giao tiếp của Module `loc_based_adder`

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
1. Global Signals (Tín hiệu hệ thống)			
<code>clk</code>	In	1	Xung nhịp đồng bộ hệ thống.
<code>rst</code>	In	1	Tín hiệu Reset tích cực cao.
<code>start</code>	In	1	Xung kích hoạt bắt đầu quá trình cộng.
Tiếp tục ở trang sau...			

Bảng 4.6 – tiếp theo từ trang trước

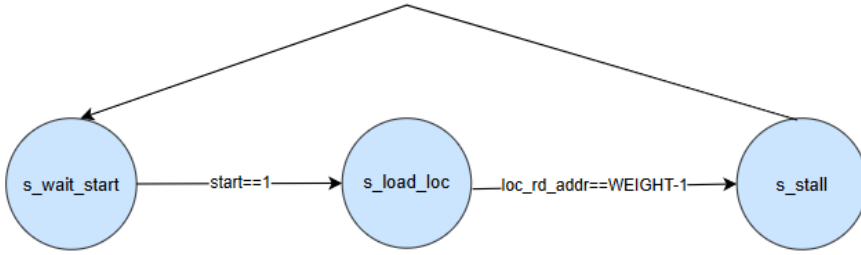
Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
2. Data Input Interface (Nạp dữ liệu)			
location	In	M	Giá trị vị trí (chỉ số bit) của số 1 trong vector thừa.
pm_in	In	WIDTH	Khối dữ liệu (32-bit) đọc từ BRAM (đây là dữ liệu không thừa).
3. Data Output Interface (Kết quả)			
loc_rd_addr	Out	LOG_WEIGHT	Địa chỉ nạp dữ liệu gửi tới BRAM.
pm_rd_addr	Out	LOG_DEPTH	Địa chỉ tính toán được để trở vào đúng khối dữ liệu chứa bit cần đảo.
add_out	Out	WIDTH	Khối dữ liệu sau khi đã đảo bit tại vị trí tương ứng.
add_out_valid	Out	1	Cờ báo hiệu kết quả đảo bit đã sẵn sàng để ghi ngược lại BRAM.
done	Out	1	Báo hiệu đã hoàn thành việc đảo tất cả các bit theo danh sách WEIGHT

4

Nguyên lý hoạt động: Cơ chế "Flip-bit" thay vì thực hiện phép XOR giữa hai mảng dữ liệu khổng lồ, module này hoạt động theo tư duy: Cộng với 1 trong trường $\mathbb{F}_2$ chính là đảo bit (0 thành 1, 1 thành 0). Cách tính toán địa chỉ và vị trí, module thực hiện chia nhỏ giá trị location để xác định mục tiêu:

- **Tính toán địa chỉ (decode_addr):** Lấy các bit cao của location. Ví dụ: nếu bit thứ 135 cần đảo và WIDTH=32, thì địa chỉ BRAM sẽ là $135/32 = 4$
- **Xác định vị trí bit trong khối:** Lấy các bit thấp để xác định vị trí chính xác trong khối 32-bit. Module sử dụng một vòng lặp generate để tạo ra một vector chỉ có duy nhất một bit 1 tại vị trí cần đảo.
- Cuối cùng là thực hiện đảo bit duy nhất vào bit tại vị trí tìm được, các bit khác giữ nguyên.

Máy trạng thái điều khiển hoạt động của module:



Hình 4.10: Loc_based_adder FSM

4

- **s_wait_start (Trạng thái 0):** Module ở trạng thái nghỉ, đợi tín hiệu start. Khi start = 1, nó chuyển sang trạng thái s_load_loc và bắt đầu kích hoạt in_rd_en.
- **s_load_loc (Trạng thái 1):** Module đọc từng vị trí một từ danh sách các số 1 (tổng cộng WEIGHT vị trí).
- **s_stall (Trạng thái 2):** Hoàn tất quá trình và bật cờ done.

Bộ cộng dựa trên vị trí (location-based adder) tận dụng tính thừa của các vector trong thuật toán HQC để tối ưu hóa hiệu suất. Thay vì thực hiện các phép toán trên toàn bộ đa thức, module thực hiện giải mã địa chỉ trực tiếp từ chỉ số vị trí và chỉ thực hiện đảo bit (bit-flipping) tại các điểm cần thiết giúp giảm số lượng truy cập bộ nhớ và rút ngắn thời gian thực thi xuống mức tối thiểu.

4.3.5. FIX WEIGHT VECTOR GENERATOR

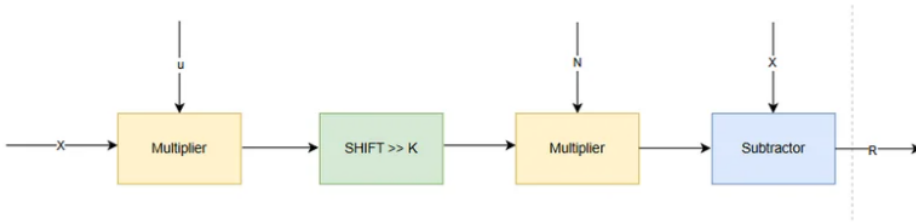
Đây là module dùng để sinh vector có trọng số cố định. Thiết kế của module này dựa trên Giải thuật 2.4 như đã đề cập ở phần cơ sở lý thuyết. Module sử dụng kiến trúc Barret Reduction để tính modulo n thay vì phép modulo thông thường trên FPGA để tăng tốc độ xử lý. Giả sử cần tính:

$$R = X \bmod N$$

với:

$$0 \leq a < N^2$$

Thay vì thực hiện phép chia trực tiếp:



Hình 4.11: Sơ đồ khối thiết kế phần cứng của Barrett Reduction

$$\left\lfloor \frac{X}{N} \right\rfloor$$

4

Barrett Reduction sử dụng một hằng số tiền tính toán trước:

$$\mu = \left\lfloor \frac{2^k}{N} \right\rfloor$$

trong đó k được chọn đủ lớn, thường là:

$$k \geq \lceil \log_2(N^2) \rceil$$

Sau đó giá trị thương xấp xỉ được tính bởi:

$$t = \left\lfloor \frac{X \cdot \mu}{2^k} \right\rfloor$$

Giá trị modulo được suy ra:

$$R = X - t \cdot N$$

Do phép xấp xỉ có thể tạo sai số nhỏ nên cần bước hiệu chỉnh cuối:

$$\text{if } R \geq N, \quad R = R - N$$

hoặc:

$$\text{while } R \geq N, \quad R = R - N$$

Kỹ thuật Barrett Reduction giúp thay thế phép chia phức tạp bằng như hình 4.11

- phép nhân,
- phép dịch bit,

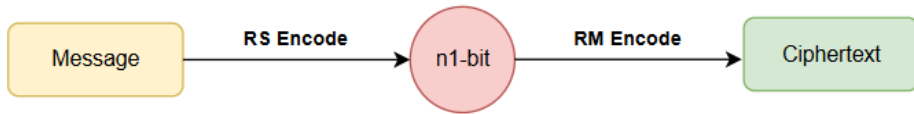
- phép trừ.

Điều này đặc biệt phù hợp cho thiết kế phần cứng FPGA do phép chia có chi phí tài nguyên lớn hơn đáng kể so với phép nhân và dịch bit.

4.3.6. ENCODE

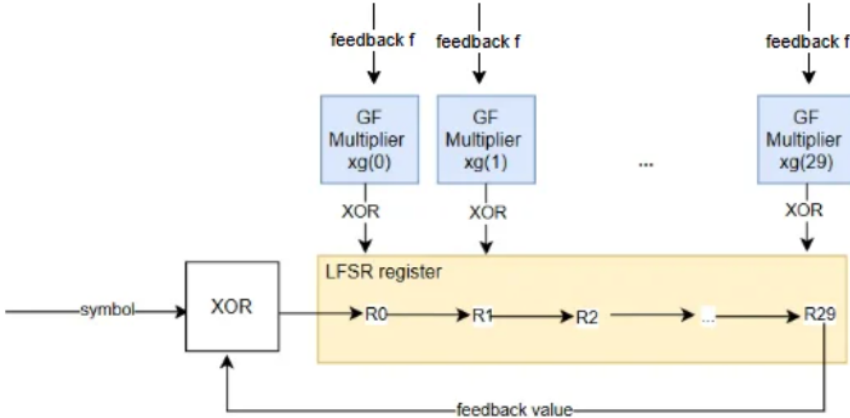
Đây là module sử dụng mã nối giữa Reed Solomon và Reed Muller, sử dụng để thêm mã sửa lỗi vào thông điệp trước khi truyền qua kênh nhiễu. Tuân tự thực hiện như Hình 4.12.

4



Hình 4.12: Quy trình thực hiện Encode

Giai đoạn 1: Reed-Solomon (RS) Encode: Đây là lớp mã ngoại (outer code), thêm các parity bytes vào message (m) xem 2.2.7 **phần mã hóa mã Reed Solomon rút gọn**. Sử dụng bộ thanh ghi dịch phản hồi tuyến tính (LFSR). Kiến trúc LFSR được biểu diễn như Hình 4.13. Kiến trúc bao gồm:



Hình 4.13: Kiến trúc module RS encode dựa trên LFSR

- Các thanh ghi dịch $R_0, R_1, \dots, R_{29}$,
- Các khối nhân trên trường hữu hạn $GF(2^8)$,
- Các cổng XOR dùng để tạo tín hiệu phản hồi (feedback),

- Giá trị đầu vào là các symbol của thông điệp.

Trong mỗi chu kỳ clock, một symbol đầu vào được đưa vào hệ thống. Giá trị này được XOR với giá trị phản hồi từ thanh ghi cuối cùng:

$$\text{feedback} = R_{29}$$

Kết quả XOR sau đó được nạp vào thanh ghi đầu tiên:

$$R_0 \leftarrow \text{symbol} \oplus \text{feedback}$$

Đồng thời, tín hiệu phản hồi được nhân với các hệ số của đa thức sinh $g(x)$ trong trường $GF(2^8)$. Các khối:

$$xg(0), xg(1), \dots, xg(29)$$

thực hiện phép nhân:

$$\text{feedback} \cdot g_i$$

trong đó g_i là hệ số của đa thức sinh.

Kết quả sau phép nhân được XOR với dữ liệu đang dịch trong các thanh ghi để cập nhật trạng thái mới:

$$R_i^{(new)} = R_{i-1}^{(old)} \oplus (\text{feedback} \cdot g_i)$$

với:

$$i = 1, 2, \dots, 29$$

Các thanh ghi hoạt động theo cơ chế dịch tuần tự:

$$R_0 \rightarrow R_1 \rightarrow R_2 \rightarrow \dots \rightarrow R_{29}$$

Sau khi toàn bộ thông điệp được đưa vào, nội dung của các thanh ghi sẽ chứa các parity symbols của mã Reed–Solomon.

Kiến trúc LFSR này giúp thực hiện phép chia đa thức:

$$m(x)x^{n-k} \bmod g(x)$$

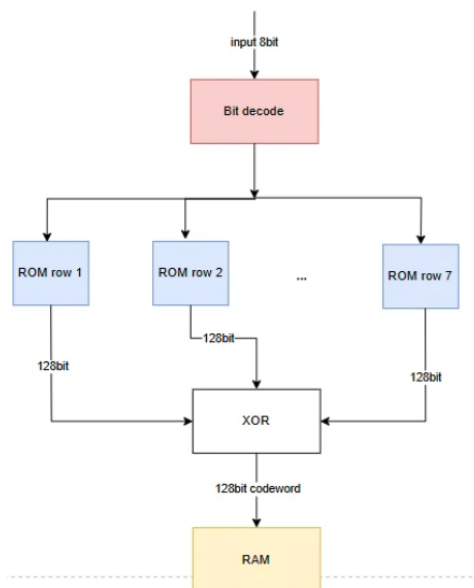
một cách hiệu quả trong phần cứng, thay thế cho các phép toán đa thức phức tạp bằng:

- thanh ghi dịch,

- phép XOR,
- phép nhân trong trường hữu hạn.

Giai đoạn 2: Reed-Muller (RM) Encode: Đây là lớp mã nội (inner code), xử lý đầu ra từ module RS phía trước. Thực hiện phép nhân vector-ma trận. Mỗi byte từ đầu vào (RS output) được coi là một vector, nhân với ma trận tạo G xem 2.2.7 **phần mã hóa mã Reed Muller nhân đôi**. Quá

4



Hình 4.14: Kiến trúc phần cứng của Reed Muller Encode

trình mã hóa bắt đầu từ vector thông điệp đầu vào gồm các bit:

$$\mathbf{m} = (m_0, m_1, \dots, m_7)$$

Mỗi bit đầu vào được đưa qua khối *Bit Decode* để xác định xem hàng tương ứng trong ma trận sinh có được kích hoạt hay không.

Kiến trúc sử dụng các khối ROM:

ROM row 1, ROM row 2, ..., ROM row 7

Mỗi ROM lưu trữ một hàng của ma trận sinh Reed–Muller dưới dạng codeword 128-bit.

Nếu bit thông điệp tương ứng bằng 1, hàng trong ROM sẽ được chọn và xuất ra giá trị 128-bit:

$\mathbf{g}_i$

Ngược lại, nếu bit đầu vào bằng 0, đầu ra tương ứng sẽ bằng 0.

Toàn bộ các vector được chọn sẽ được cộng modulo-2 bằng các cổng XOR:

$$\mathbf{c} = m_0\mathbf{g}_0 \oplus m_1\mathbf{g}_1 \oplus \cdots \oplus m_7\mathbf{g}_7$$

Trong đó:

$\oplus$

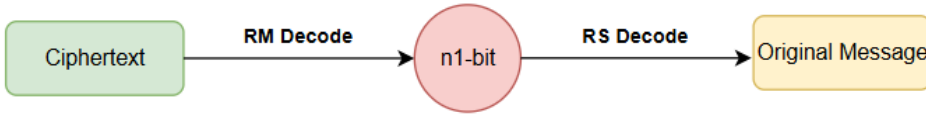
là phép XOR trên trường nhị phân $GF(2)$.

Kết quả cuối cùng tạo thành codeword Reed–Muller 128-bit:

$$\mathbf{c} \in \mathbb{F}_2^{128}$$

4.3.7. DECODE

Đây là module làm ngược lại với **Encode** để tìm ra thông điệp m từ ciphertext bị nhiễu. Quy trình được biểu diễn như Hình 4.15.



Hình 4.15: Quy trình thực hiện Decode

Giai đoạn 1: Reed-Muller (RM) Decode: Quá trình giải mã được thực hiện dựa trên biến đổi Hadamard nhằm tìm ra codeword có độ tương quan lớn nhất với tín hiệu nhận được xem 2.2.7 phần **Giải mã mã Reed Muller Nhân Đôi**.

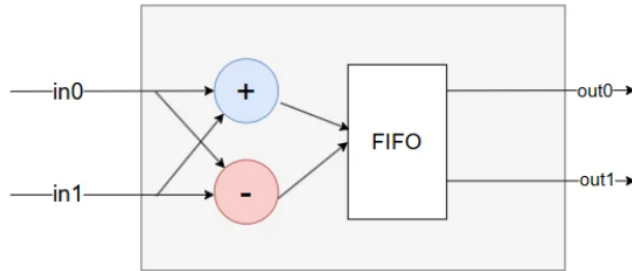
Kiến trúc giải mã bao gồm nhiều tầng butterfly hoạt động tuần tự để thực hiện phép biến đổi Hadamard nhanh (Fast Hadamard Transform – FHT). Đối với codeword 128-bit, phép biến đổi Hadamard cần:

$$\log_2(128) = 7 \text{ stage xử lý.}$$

Dòng dữ liệu được xử lý theo chuỗi:

Input Stream $\rightarrow$ Butterfly Stage 1 $\rightarrow \cdots \rightarrow$ Butterfly Stage 7 $\rightarrow$ Correlation Outputs

Butterfly Core được biểu diễn như Hình 4.16.



Hình 4.16: Kiến trúc phần cứng BuffterFly Core

4

Mỗi butterfly core nhận hai giá trị đầu vào:

$$in_0, \quad in_1$$

và tạo ra hai đầu ra:

$$out_0 = in_0 + in_1$$

$$out_1 = in_0 - in_1$$

Các phép cộng và trừ này được thực hiện trên dữ liệu signed nhằm tính toán mức độ tương quan giữa tín hiệu nhận và các codeword Reed–Muller.

FIFO được sử dụng để:

- đồng bộ dữ liệu giữa các stage,
- tạo độ trễ phù hợp cho butterfly,
- đảm bảo luồng dữ liệu liên tục theo pipeline.

Correlation Output

Sau khi dữ liệu đi qua toàn bộ 7 stage, hệ thống thu được tập giá trị tương quan:

$$C_0, C_1, \dots, C_{127}$$

Mỗi giá trị biểu diễn độ tương quan giữa tín hiệu nhận được và một codeword Reed–Muller tương ứng.

Khởi *Find Peak* sẽ tìm:

$$C_{\max} = \max(C_i)$$

Đồng thời xác định chỉ số:

$$i_{\max}$$

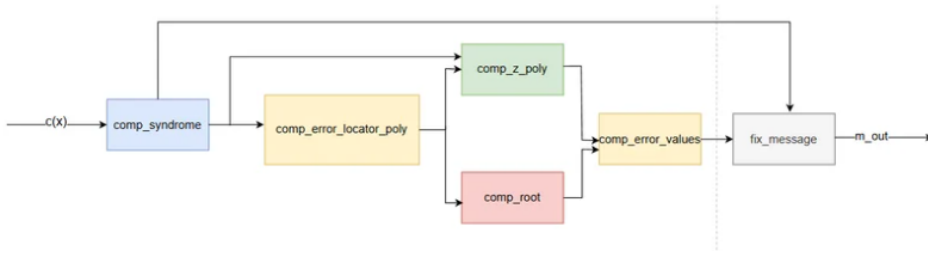
với:

$$C_{i_{\max}} = C_{\max}$$

Chỉ số này chính là symbol Reed–Solomon được giải mã:

$$RS\ Symbol = i_{\max}$$

Giai đoạn 2: Reed-Solomon (RS) Decode: Quá trình giải mã RS được thực hiện theo một chuỗi tính toán để tìm ra thông điệp chính xác xem 2.2.7 phần **Giải mã mã Reed Solomon Rút gọn**. Hình 4.17 mô tả kiến trúc tổng



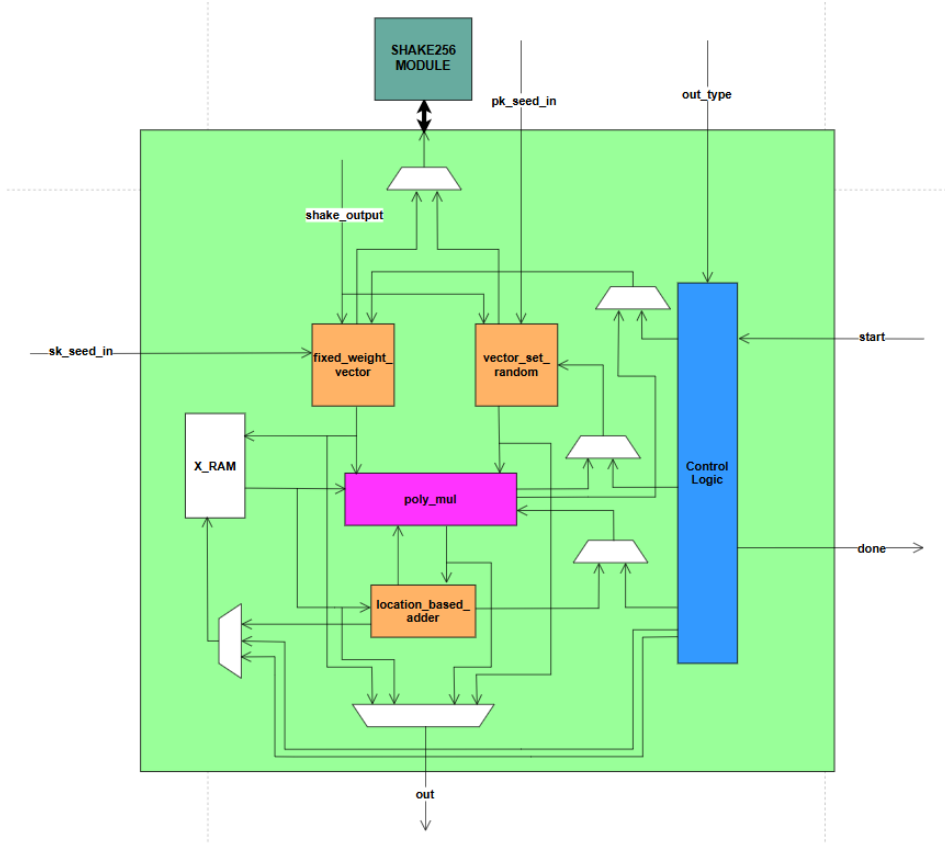
Hình 4.17: Kiến trúc phần cứng của module Reed Solomon decode

quát của bộ giải mã Reed-Solomon (RS Decoder) trong HQC. Quá trình giải mã RS bao gồm các modulo bên trong để tìm lỗi và giải mã lỗi.

1. comp_syndrome: Tính syndrome
2. comp_error_locator_poly: Tính đa thức định vị lỗi
3. comp_z_poly: Tính đa thức $z(x)$
4. comp_root: Tìm vị trí lỗi
5. comp_error_values: Tính giá trị lỗi
6. fix_message: Sửa lỗi để khôi phục thông điệp

4.3.8. KEY GENERATION

Module KEYGEN chịu trách nhiệm thực thi thuật toán sinh khóa của HQC. Chức năng chính của module là sinh ra cặp khóa công khai **pk** và khóa bí mật **sk** dựa trên các hạt giống (seed) đầu vào. Hình 4.18 biểu diễn



Hình 4.18: Sơ đồ khối thiết kế phần cứng của module KEYGEN

kiến trúc phần cứng module KEYGEN, các thành phần trong kiến trúc:

- **Control Logic:** Khối điều khiển trung tâm (FSM) chịu trách nhiệm điều phối toàn bộ hoạt động, kích hoạt các module con theo trình tự và quản lý việc truy cập vào tài nguyên chia sẻ (SHAKE256).
- **Fixed Weight Vector Generator:** Sinh ra đa thức (x,y) từ hạt giống khóa bí mật (**sk_seed**) có trọng số cố định bằng **w** và chiều dài là **n** bit.

- **Vector_set_random:** Sinh ra đa thức công khai **h** bằng cách mở rộng hạt giống khoá công khai (**pk_seed**) thông qua SHAKE256.
- **Poly_mult (Polynomial Multiplication):** Thực hiện phép nhân đa thức giữa **h** (từ Vector Set Random) và **y** (từ Fixed Weight Vector). Module này sử dụng kỹ thuật nhân đa thức thưa như đề cập trong phần cơ sở lý thuyết. Trong kiến trúc Joint Design tối ưu diện tích, khối poly_mult bên trong sơ đồ này không được khởi tạo vật lý (instantiated). Thay vào đó, các tín hiệu giao tiếp của nó được định tuyến ra các cổng ngoại vi (Shared Interface) để kết nối với bộ nhân đa thức dùng chung nằm ở cấp Top-level.
- **Location Based Adder:** Thực hiện phép cộng cuối cùng để tạo ra **s**. Module này đọc kết quả nhân (**h.y**) và thực hiện cộng (XOR) với vector **x** dựa trên các vị trí bit 1 được lưu trong X_RAM.
- **X_RAM:** Đây là block RAM chỉ chứa đa thức **x**, các đa thức **y**, **h**, **s** được lưu trữ nội bộ trong block RAM của **fixed_weight_vector_generator**, **poly_mult** and **location_based_adder**.

Hình 4.19 mô tả giao diện của KeyGen Module là các nhóm tín hiệu giao tiếp với tài nguyên dùng chung (Shared Interfaces). Thay vì thực hiện tính toán nội bộ, KeyGen xuất các tín hiệu điều khiển (pm_start, shake_valid) và dữ liệu thô (pm_mux_word, shake_din) ra bên ngoài. Các tín hiệu này sẽ được đưa qua bộ dồn kênh (Multiplexer) tại Top-level để truy cập vào các module SHAKE256 và POLY_MUL, hiện thực hóa kiến trúc Joint Design tối ưu diện tích. Bảng 4.7 dưới đây liệt kê chức năng của từng tín hiệu.

Bảng 4.7: Bảng đặc tả tín hiệu giao tiếp của Module KEYGEN

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
1. Global Control (Điều khiển hệ thống)			
clk	In	1	Xung nhịp đồng bộ.
rst	In	1	Tín hiệu Reset.
start	In	1	Kích hoạt quá trình tạo khóa.
operation	In	2	Mã lệnh điều khiển (Thường cố định cho KeyGen).
Tiếp tục ở trang sau...			

Bảng 4.7 – tiếp theo từ trang trước

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
done	Out	1	Báo hiệu đã tạo khóa xong (pk, sk sẵn sàng).
2. Seed Inputs (Nạp Hạt giống)			
sk_seed, pk_seed	In	32	Dữ liệu hạt giống cho khóa bí mật/công khai.
*_seed_addr	In	4	Địa chỉ nạp seed vào bộ đệm nội bộ.
*_seed_wen	In	1	Tín hiệu cho phép ghi seed.
3. KeyGen Outputs (Xuất kết quả)			
keygen_out	Out	Var	Dữ liệu đầu ra (pk, sk).
keygen_out_en	In	1	Cho phép đọc kết quả.
keygen_out_type	In	2	Chọn loại dữ liệu: 00(x), 01(y), 10(h), 11(s).
keygen_out_addr	In	Var	Địa chỉ đọc kết quả trong bộ nhớ nội bộ.
4. Shared SHAKE Interface (Giao tiếp Bộ băm)			
<i>*Kết nối với module SHAKE256 dùng chung</i>			
shake_din, _valid	Out	32/1	Gửi dữ liệu/seed vào bộ băm.
shake_din_ready	In	1	Báo hiệu SHAKE sẵn sàng nhận dữ liệu.
shake_dout_scram	In	32	Nhận chuỗi bit ngẫu nhiên từ SHAKE.
shake_dout_valid	In	1	Báo hiệu dữ liệu từ SHAKE hợp lệ.
shake_dout_ready	Out	1	Báo hiệu KeyGen sẵn sàng nhận dữ liệu.
shake_force_done	Out	1	Ngắt hoạt động của SHAKE khi đã đủ dữ liệu.
5. Shared PolyMul Interface (Giao tiếp Bộ nhân)			
<i>*Kết nối với module Poly_Mul dùng chung</i>			
pm_start	Out	1	Kích hoạt bộ nhân đa thức.
pm_loc_in	Out	Var	Vị trí bit 1 của vector thừa (Sparse input).
Tiếp tục ở trang sau...			

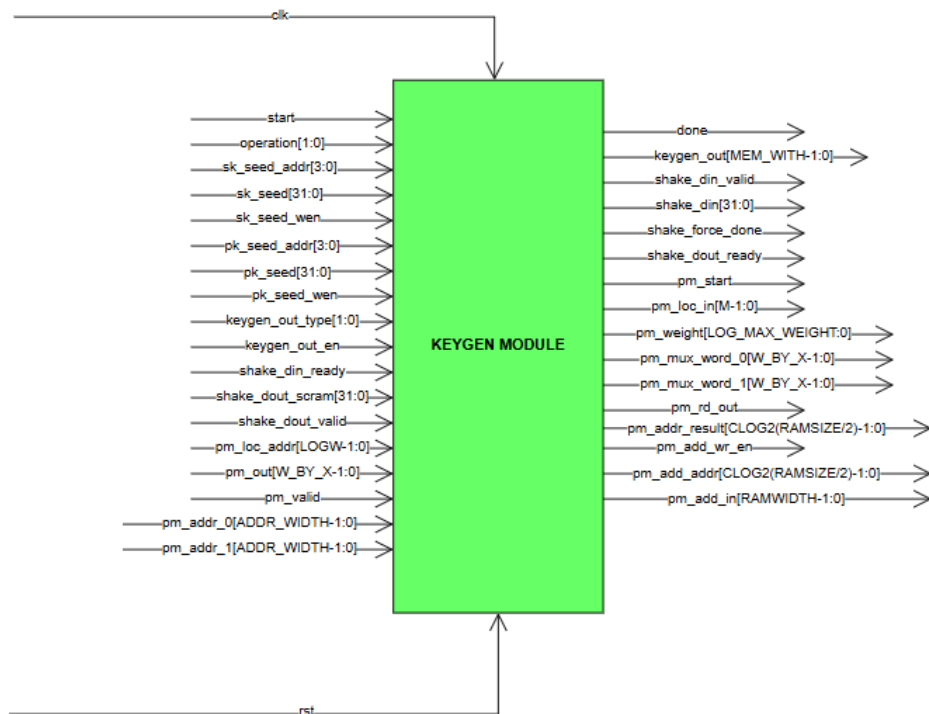
Bảng 4.7 – tiếp theo từ trang trước

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
pm_mux_word_0/1	Out	Var	Dữ liệu đa thức dày (Dense input) nạp vào bộ nhân.
pm_out	In	Var	Kết quả nhân trả về từ PolyMul.
pm_valid	In	1	Báo hiệu phép nhân hoàn tất.
pm_addr_0/1	In	Var	Địa chỉ bộ nhân yêu cầu đọc đa thức dày.
pm_loc_addr	In	Var	Địa chỉ bộ nhân yêu cầu đọc vị trí vector thưa.
pm_add_*	Out	Var	Tín hiệu điều khiển bộ cộng (Adder) tích hợp.

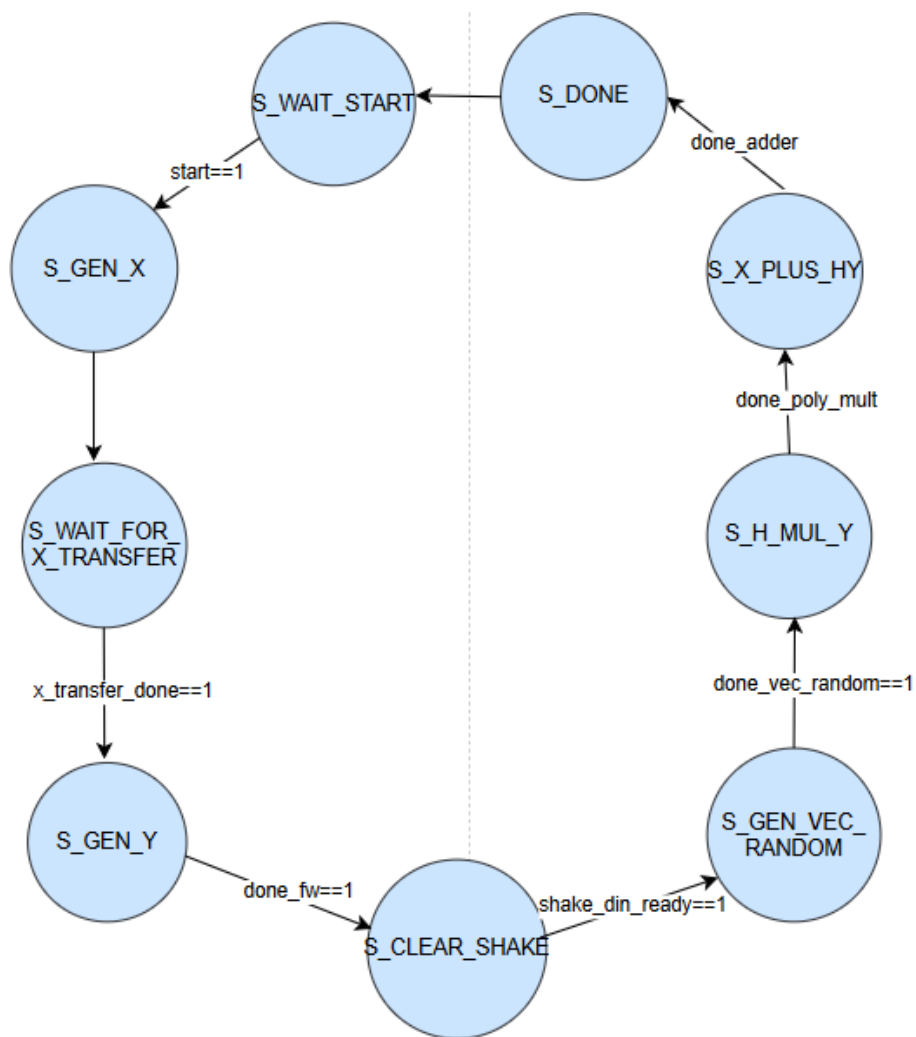
4

- **S_WAIT_START:** Trạng thái nghỉ/chờ của hệ thống. Reset toàn bộ các tín hiệu điều khiển. Nếu có xung kích hoạt $start == 1$, FSM chuyển sang trạng thái tiếp theo s_gen_x .
- **S_GEN_X:** Kích hoạt phần cứng sinh vector lỗi/bí mật x thông qua bộ lấy mẫu Fixed Weight Generation. Đợi tín hiệu hoàn thành từ bộ sinh mẫu ($done_fw == 1$).
- **S_WAIT_FOR_X_TRANSFER:** Trạng thái đồng bộ hóa/chờ đợi khi ở chế độ thiết kế Constant Time nhằm đảm bảo thời gian tính toán không phụ thuộc vào dữ liệu. Khi tín hiệu $x_transfer_done == 1$ (báo hiệu dữ liệu x đã được chuyển/nạp xong vào đúng vùng nhớ), FSM chuyển sang s_gen_y .
- **S_GEN_Y:** Kích hoạt sinh tiếp vector bí mật thứ hai y (tương tự như x).
- **S_CLEAR_SHAKE:** Đảm bảo bộ băm SHAKE được đưa về trạng thái sẵn sàng để sinh vector ngẫu nhiên tiếp theo (h). Reset các tín hiệu điều khiển vừa dùng ở trạng thái trước. Đợi đến khi bộ băm sẵn sàng nhận dữ liệu mới ($shake_din_ready == 1$), FSM chuyển sang $s_gen_vec_random$.

4



Hình 4.19: Giao diện kết nối của module KEYGEN



Hình 4.20: Key Generation FSM

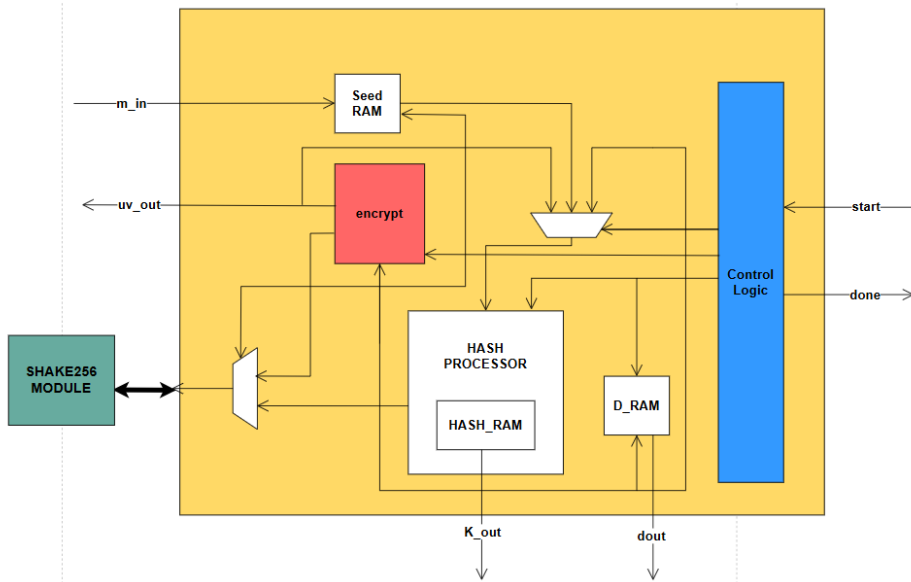
4

- **S_GEN_VEC_RANDOM:** Sinh ra vector công khai h có kích thước rất lớn từ hạt giống (Seed) thông qua bộ mở rộng SHAKE. Đợi đến khi hoàn thành sinh toàn bộ vector h (`done_vec_random == 1`) FSM chuyển sang trạng thái nhân đa thức `s_h_mul_y`.
- **S_H_MUL_Y:** Kích hoạt khối POLY MULTIPLICATION để thực hiện phép nhân chập đa thức vòng (Ring Multiplication) giữa h và y . Đợi khối nhân báo hoàn thành (`done_poly_mult == 1`), FSM chuyển sang `s_x_plus_hy`, đồng thời hạ các tín hiệu đọc RAM về 0
- **S_X_PLUS_HY:** Cộng vector bí mật x với kết quả phép nhân đa thức thu được ở trạng thái trước để tạo ra thành phần khóa công khai s . Đợi bộ cộng hoàn thành (`done_adder == 1`) chuyển sang trạng thái `s_done`.
- **S_DONE:** Chốt trạng thái hoàn thành. Giữ chân `done` ở mức cao tạm thời trước khi trả về 0 ở chu kỳ clock tiếp theo (hoặc khi hệ thống reset/khởi động lại).

4.3.9. ENCAPSULATION

Module ENCAP thực hiện thuật toán mã hóa HQC. Nhiệm vụ chính của nó là mã hóa bản tin đầu vào $\mathbf{m}$ sử dụng khóa công khai $(\mathbf{h}, \mathbf{s})$ để tạo ra bản mã $\mathbf{c} = (\mathbf{u}, \mathbf{v})$ và khóa chia sẻ $\mathbf{K}$ và $\mathbf{d}$. Sơ đồ khối kiến trúc phần cứng được thể hiện trong Hình 4.21. Thiết kế tập trung vào việc tối ưu hóa giao tiếp với tài nguyên dùng chung (SHAKE256) và tăng tốc độ xử lý thông qua cơ chế song song hóa. Các thành phần chính:

- **Hash Processor:** Đây là một module chuyên dụng được thiết kế để trừu tượng hóa việc giao tiếp với SHAKE256. Hash Processor đóng vai trò là trung gian, quản lý toàn bộ các yêu cầu truy xuất SHAKE256. Module này thực hiện các tác vụ sau:
 - Mở rộng $\mathbf{m}$ để tạo θ .
 - Sau khi tạo ra các $\mathbf{r1}$, $\mathbf{r2}$ và $\mathbf{e}$ bên trong module **encrypt**. Hash Processor được chạy để tạo ra $\mathbf{d}$.
 - Sau khi mã hoá xong thông điệp $\mathbf{m}$ bởi **encrypt**, sử dụng Hash Processor để tính $\mathbf{K}(\mathbf{m}, \mathbf{c})$.
- **Encrypt:** Tiếp nhận đầu vào là khóa công khai (h, s) , bản tin m , hạt giống θ và tạo ra bản mã (u, v) ở đầu ra. Sử dụng module tạo



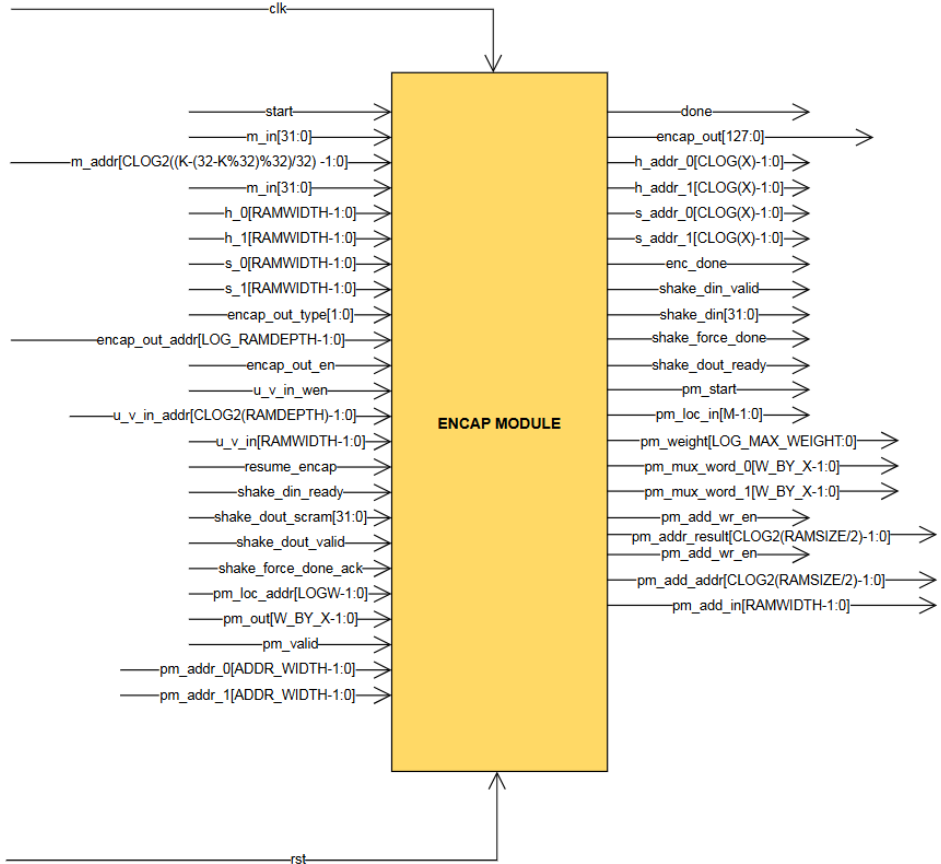
Hình 4.21: Sơ đồ khối thiết kế phần cứng của module ENCAP

vector trọng số cố định để tạo ra các vector trọng số cố định r_1, r_2 và e với trọng số w_r bằng cách mở rộng giá trị θ . Ngay sau khi r_2 được tạo xong, quá trình nhân đa thức $h \cdot r_2$ sẽ được bắt đầu song song với quá trình tạo vector e . Đối với phép nhân đa thức sử dụng module **poly_mult**. Các phép cộng r_1 trong việc tính toán u , và cộng e trong tính toán v được thực hiện bởi bộ cộng **location-based adder**, trong khi phép cộng với t được thực hiện bởi bộ cộng dựa trên **xor_based_adder**.

- **Encode(m)**: Thêm mã sửa lỗi vào thông điệp m trước khi truyền qua kênh nhiễu.

Hình 4.22 này mô tả giao diện của Module Encap được thiết kế tối ưu cho cả quy trình Đóng gói (Encapsulation) và Mã hóa lại (Re-encryption), vận hành thông qua cơ chế bắt tay (handshake) với các tín hiệu điều khiển $start$, $done$ và $resume$. Về mặt dữ liệu, module sử dụng giao diện nạp tuần tự cho bản tin đầu vào và tích hợp cơ chế truy cập bộ nhớ chủ động (Master Dual-port) để đọc song song Khóa công khai từ BRAM bên ngoài, giúp tối đa hóa băng thông cho bộ nhân. Đặc biệt, module không thực hiện toàn bộ tính toán nội bộ mà đóng vai trò điều phối, xuất các tín hiệu điều khiển tới các tài nguyên dùng chung (PolyMul, SHAKE) và

4



Hình 4.22: Giao diện kết nối của module ENCAP

cung cấp kết quả Bản mã cùng Khóa chia sẻ thông qua cổng đọc dữ liệu chuyên dụng. Bảng 4.8 mô tả chức năng của từng tín hiệu của module.

Bảng 4.8: Bảng đặc tả tín hiệu giao tiếp của Module Encap

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
1. Global Control (Điều khiển hệ thống)			
clk	In	1	Xung nhịp đồng bộ.
rst	In	1	Tín hiệu Reset.
start	In	1	Kích hoạt quá trình mã hóa/đóng gói.
done / enc_done	Out	1	Báo hiệu quá trình hoàn tất.
resume_encap	In	1	Tín hiệu điều khiển cho chế độ Re-encryption (trong Decap).
2. Message Input (Nạp Bản tin)			
m_in	In	32	Dữ liệu bản tin m đầu vào (nạp tuần tự).
m_addr	In	Var	Địa chỉ nạp bản tin vào bộ đệm nội bộ.
m_wen	In	1	Cho phép ghi bản tin.
3. Memory Read Interface (Đọc Khóa công khai)			
<i>*Truy xuất BRAM ngoài để lấy Public Key (h, s)</i>			
h_0, h_1	In	Var	Dữ liệu đa thức h (Dual-port).
h_addr_0/1	Out	Var	Địa chỉ yêu cầu đọc đa thức h .
s_0, s_1	In	Var	Dữ liệu đa thức s (Dual-port).
s_addr_0/1	Out	Var	Địa chỉ yêu cầu đọc đa thức s .
4. Encap Outputs (Xuất kết quả)			
encap_out	Out	128	Dữ liệu đầu ra (Ciphertext u, v và Shared Key).
encap_out_type	In	2	Chọn loại dữ liệu đọc ra (u, v, K).
encap_out_addr	In	Var	Địa chỉ đọc kết quả từ bộ đệm nội bộ.
Tiếp tục ở trang sau...			

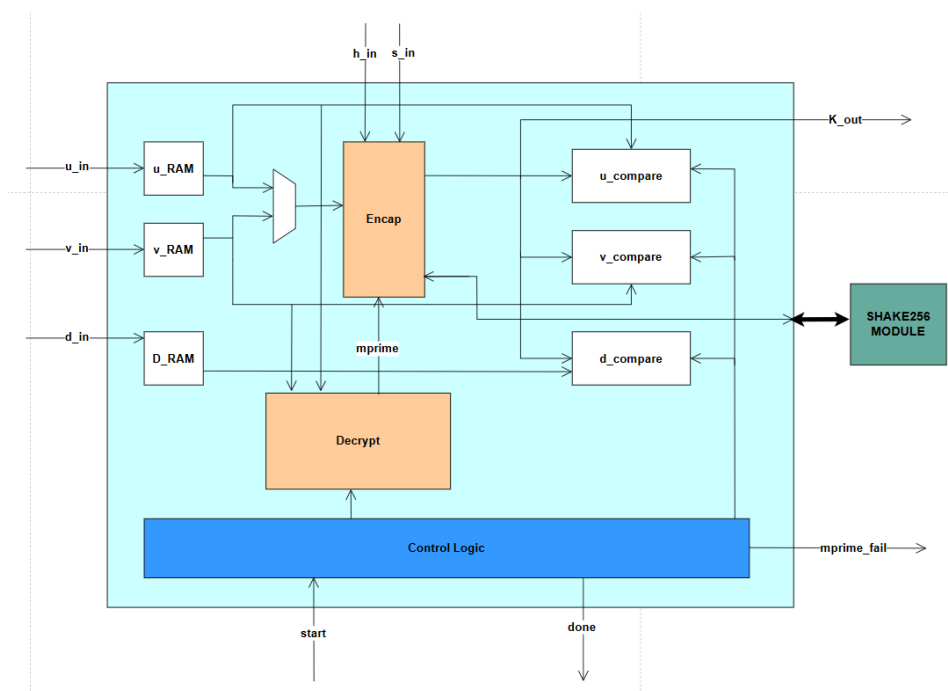
Bảng 4.8 – tiếp theo từ trang trước

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
encap_out_en	In	1	Cho phép đọc kết quả.
5. Shared SHAKE Interface (Giao tiếp Bộ băm)			
shake_din, _valid	Out	32/1	Gửi dữ liệu/seed vào bộ băm.
shake_dout_scram	In	32	Nhận chuỗi bit ngẫu nhiên từ SHAKE.
shake_force_done_ack	In	1	Xác nhận hoàn thành từ SHAKE.
...và các tín hiệu bắt tay khác			(Tương tự KeyGen).
6. Shared PolyMul Interface (Giao tiếp Bộ nhân)			
pm_start	Out	1	Kích hoạt bộ nhân đa thức.
pm_mux_word_0/1	Out	Var	Dữ liệu đa thức dày nạp vào bộ nhân.
pm_loc_in	Out	Var	Vị trí bit 1 của vector thừa (ví dụ r_2).
pm_add_in	Out	Var	Dữ liệu để thực hiện phép cộng tích lũy (Accumulate).
pm_add_wr_en	Out	1	Điều khiển ghi kết quả cộng.
pm_addr_result	Out	Var	Địa chỉ lưu kết quả nhân/cộng.

4.3.10. DECAPSULATION

Module DECAP chịu trách nhiệm thực thi thuật toán Mở gói khóa (Decapsulation). Module sử dụng khóa bí mật $\mathbf{sk} = (\mathbf{x}, \mathbf{y})$, Khóa công khai $\mathbf{pk} = (\mathbf{h}, \mathbf{s})$, Bản mã $\mathbf{c} = (\mathbf{u}, \mathbf{v})$ và giá trị băm $\mathbf{d}$ để khôi phục khóa $\mathbf{K}$ và xác minh tính toàn vẹn của bản mã. Hình 4.23 này biểu diễn thiết kế phần cứng của module DECAP. Các thành chính của module:

- **Input Memories (u_RAM, v_RAM, D_RAM):** Các khối bộ nhớ đệm nội bộ dùng để lưu trữ bản mã đầu vào $\mathbf{u}, \mathbf{v}$ và giá trị băm $\mathbf{d}$ nhận được từ bên ngoài.
- **Decrypt:** Thực hiện giải mã sơ bộ bản mã $\mathbf{u}, \mathbf{v}$ sử dụng khóa bí mật $\mathbf{x}, \mathbf{y}$ để khôi phục lại bản tin $\mathbf{m}'$ (**m-prime**). Module thực hiện phép tính cốt lõi của giải mã HQC: $s = v - (\mathbf{u} \cdot \mathbf{y})$. Sử dụng **poly_mul** để



Hình 4.23: Sơ đồ khối thiết kế phần cứng của module DECAP

tính toán phép nhân $u \cdot y$ và **xor_based_adder** để thực hiện phép tính $v - (u \cdot y)$.

- **Decode**($v - (u \cdot y)$): Giải mã sửa lỗi để tìm ra thông điệp m.
- **Encap (Re-encryption Engine)**: Lấy bản tin đã giải mã **m'** và khóa công khai **h, s** để tạo ra lại bản mã mới **u', v'** và giá trị băm mới **d'**.
- **Comparators (u/v/d_compare)**: Các bộ so sánh phần cứng thực hiện so khớp bit-by-bit giữa dữ liệu nhận được **u, v, d** và dữ liệu tái tạo **u', v', d'**.

4

Hình 4.24 mô tả giao diện của Module Decap được thiết kế để thực hiện quy trình mở gói khóa HQC và xác thực bản mã (IND-CCA2). Module hoạt động như một thiết bị chủ (Master) truy xuất song song các thành phần của bản mã (u, v) và khóa bí mật (y) từ bộ nhớ ngoài thông qua giao diện Dual-port. Đặc biệt, module tích hợp giao diện điều khiển nội bộ (Re-encryption Interface) để kích hoạt và nạp dữ liệu cho quá trình mã hóa lại, cho phép so sánh bản mã tái tạo với bản mã gốc nhằm phát hiện tấn công giả mạo. Kết quả cuối cùng là Khóa chia sẻ (K) được xuất ra nếu quá trình xác thực thành công. Chi tiết chức năng của từng tín hiệu được mô tả trong Bảng 4.9

Bảng 4.9: Bảng đặc tả tín hiệu giao tiếp của Module Decap

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
1. Global Control (Điều khiển hệ thống)			
clk	In	1	Xung nhịp đồng bộ.
rst	In	1	Tín hiệu Reset.
start	In	1	Kích hoạt quá trình mở gói.
done	Out	1	Báo hiệu quá trình hoàn tất (thành công hoặc thất bại).
2. Memory Read Interface (Đọc Bản mã & Khóa)			
<i>*Truy xuất BRAM ngoài để lấy Ciphertext (u, v) và Secret Key (y)</i>			
u_0/1, v_0/1	In	Var	Dữ liệu bản mã u và v (Dual-port Read).
u_addr_*, v_addr_*	Out	Var	Địa chỉ yêu cầu đọc bản mã.
y	In	Var	Dữ liệu vector thừa y (một phần của Secret Key).
Tiếp tục ở trang sau...			

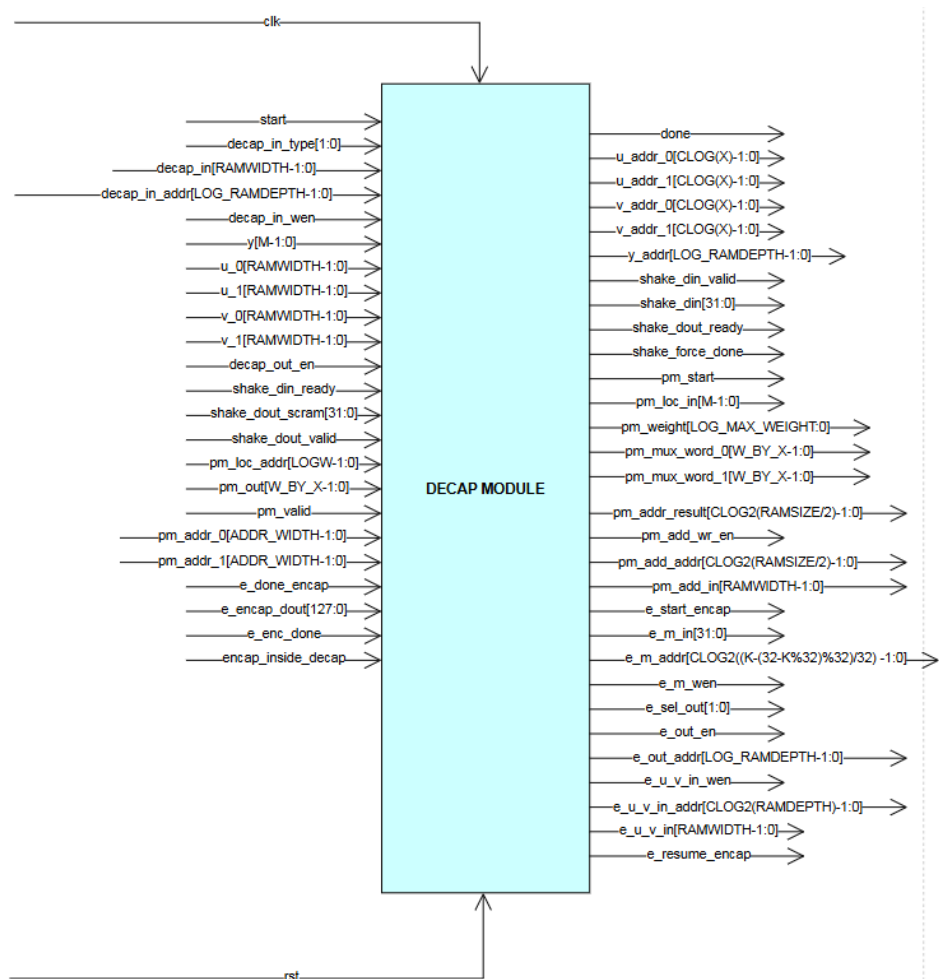
Bảng 4.9 – tiếp theo từ trang trước

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng
y_addr	Out	Var	Địa chỉ yêu cầu đọc vector y .
3. Decap Inputs (Nạp dữ liệu phụ)			
decap_in	In	Var	Dữ liệu đầu vào bổ sung (nếu có).
decap_in_addr	In	Var	Địa chỉ nạp vào bộ đệm nội bộ.
decap_in_wen	In	1	Cho phép ghi.
4. Re-Encryption Interface (Giao tiếp Mã hóa lại)			
<i>*Điều khiển Module Encap để kiểm tra tính toàn vẹn (FO Transform)</i>			
e_start_encap	Out	1	Lệnh kích hoạt module Encap chạy chế độ Re-encrypt.
e_m_in	Out	32	Gửi bản tin đã giải mã (m') sang Encap.
e_m_addr/wen	Out	Var	Điều khiển việc ghi m' vào Encap.
e_encap_out	In	128	Đọc lại bản mã tái tạo (c') từ Encap để so sánh.
e_enc_done	In	1	Tín hiệu báo Encap đã mã hóa xong.
encap_inside_decap	Out	1	Cờ báo hiệu Encap đang chạy dưới quyền Decap.
5. Shared Resources Interface (Tài nguyên dùng chung)			
shake_*	In/Out	Bus	Giao tiếp với bộ băm SHAKE256 (để băm m' và tạo K).
pm_*	In/Out	Bus	Giao tiếp với bộ nhân đa thức (tính $u \cdot y$).
6. Decap Output (Xuất kết quả)			
decap_out	Out	Var	Khóa chia sẻ (K) nếu xác thực đúng (hoặc null nếu sai).
decap_out_en	In	1	Cho phép đọc kết quả.

Tiếp tục ở trang sau...

Bảng 4.9 – tiếp theo từ trang trước

Tên Tín hiệu	Hướng	Độ rộng	Mô tả Chức năng



Hình 4.24: Giao diện kết nối của module DECAP

5

ĐÁNH GIÁ

5.1. TIÊU CHÍ ĐÁNH GIÁ

Để đánh giá hiệu quả hoạt động của phương án đề xuất, một số chỉ số chính được thu thập và phân tích, như được nêu dưới đây:

- **Timing:** Thời gian tiêu tốn cho các thao tác Tạo khóa, Mã hóa và Giải mã được đo bằng micro giây (μs) hoặc giây, tùy thuộc vào nền tảng thực thi.
- **Thông lượng (TP):** Thông lượng được định nghĩa là khối lượng dữ liệu được xử lý trên mỗi đơn vị thời gian và được tính bằng biểu thức sau:

$$TP = \frac{N_{Byte}}{T} \quad (5.1)$$

- **Tần số trên mỗi chu kỳ xung nhịp (Freq/CCs):** Chỉ số này cho biết hiệu quả hoạt động của hệ thống trên mỗi chu kỳ xung nhịp và được định nghĩa như sau:

$$Freq/CCs = \frac{F}{C} \quad (5.2)$$

trong đó F là tần số hoạt động tối đa (tính bằng MHz), và C là số chu kỳ xung nhịp cần thiết cho hoạt động. Tỷ lệ Freq/CCs càng cao thì hiệu suất càng tốt.

- **Diện tích $\times$ Thời gian:** Chỉ số này kết hợp diện tích và hiệu năng để thể hiện tổng chi phí phần cứng trên thời gian thực thi:

$$A \times T = N_{LUT} \times T \quad (5.3)$$

trong đó thời gian thực thi T được biểu diễn bằng micro giây (μs) thay vì mili giây để các giá trị $A \times T$ thu được nằm trong dải số thuận tiện cho việc so sánh trực quan giữa các thiết kế phần cứng có hiệu năng cao và N_{LUT} là số lượng LUT. Giá trị nhỏ hơn có nghĩa là hiệu quả phần cứng cao hơn. Đơn vị của $A \times T$ là $LUTs \cdot \mu s$

- **Total On-Chip Power:** Chỉ số này thể hiện tổng mức tiêu thụ điện năng của toàn bộ chip (đơn vị W hoặc mW).
- **Năng lượng trên mỗi phép toán (E_{op}):** Đối với các kịch bản triển khai trên thiết bị biên có ràng buộc năng lượng, năng lượng tiêu thụ cho mỗi phép toán mật mã là một chỉ số then chốt. Chỉ số này được định nghĩa như sau:

$$E_{op} = P_{total} \times T \quad (5.4)$$

5

trong đó P_{total} là tổng công suất tiêu thụ trên chip (đã định nghĩa ở trước đo trên) và T là thời gian thực thi của một phép toán. Đơn vị là micro Joule trên một phép toán ($\mu J/op$). Giá trị này càng nhỏ thì hiệu quả năng lượng của thiết kế càng cao, đặc biệt quan trọng khi triển khai mật mã hậu lượng tử trên các thiết bị Internet of Things hoạt động bằng pin.

Lưu ý về Throughput đối với KEM: Khác với các thuật toán mã hóa khối, một bộ Key-Encapsulation Mechanism (KEM) không trực tiếp xử lý một khối dữ liệu có kích thước cố định, mà tạo ra một khóa chia sẻ làm đầu ra. Trong báo cáo này, Throughput được định lượng dưới hai dạng: (i) số phép toán hoàn tất trong một đơn vị thời gian (operations per second), và (ii) thông lượng dữ liệu danh nghĩa tính theo kích thước bản mã đầu ra ($N_{Byte} = 4.481$ byte cho HQC-128). Cách diễn giải đầu tiên phản ánh khả năng đáp ứng số yêu cầu mã hóa đồng thời của hệ thống, trong khi cách thứ hai cho phép so sánh với các thuật toán mã hóa truyền thống.

5.2. KẾT QUẢ THỰC NGHIỆM

5.2.1. MÔI TRƯỜNG VÀ KỊCH BẢN ĐÁNH GIÁ

Để tiến hành đánh giá toàn diện và khách quan hiệu năng của bộ tăng tốc phần cứng, quá trình thực nghiệm được triển khai trên nền tảng SoC-FPGA Kria KV260 tích hợp chip Zynq UltraScale+. Cấu hình thiết kế được đưa vào thử nghiệm là phiên bản thuật toán HQC-128 đã được tối ưu hóa tài nguyên. Đây là phiên bản được tinh chỉnh chuyên biệt nhằm

cân bằng giữa tốc độ xử lý và diện tích phần cứng, đặc biệt phù hợp cho các ứng dụng thực tế trên thiết bị biên.

Về mặt phần mềm và tích hợp hệ thống, thực nghiệm sử dụng Khung phần mềm (framework) PYNQ hoạt động trên môi trường Jupyter Notebook. Lựa chọn này cho phép hệ điều hành đang chạy trên nhân xử lý ARM (PS) có thể tải trực tiếp tệp cấu hình phần cứng (Overlay) xuống vùng logic lập trình được (PL). Nhờ đó, chương trình điều khiển bằng Python có thể thiết lập kênh giao tiếp liền mạch, truyền và nhận dữ liệu trực tiếp với IP Core của thuật toán thông qua các giao thức tiêu chuẩn mà không gặp trở ngại về tương thích.

Nhằm khẳng định độ tin cậy của bản thiết kế, kịch bản kiểm thử được xây dựng với cường độ cao để thử thách tối đa tính ổn định và sự chính xác. Cụ thể, hệ thống sử dụng các hạt giống sinh số ngẫu nhiên (seed) để tự động tạo ra 10.000 bộ dữ liệu kiểm thử độc lập. Toàn bộ 10.000 bài kiểm tra này được ra lệnh thực thi liên tục không ngắt nghỉ. Kịch bản khắc khe này giúp mô phỏng điều kiện hoạt động thực tế, qua đó xác minh bộ tăng tốc phần cứng hoàn toàn không bị suy giảm hiệu năng, không xảy ra hiện tượng tràn bộ nhớ hay sai lệch trạng thái logic trong suốt quá trình vận hành dài hạn.

5

5.2.2. KIỂM CHỨNG TÍNH ĐÚNG ĐẮN CỦA THUẬT TOÁN

Mục tiêu cao nhất của tất cả hệ thống mật mã phần cứng là phải đảm bảo tính chính xác tuyệt đối trong quá trình vận hành. Để kiểm chứng toàn diện chức năng của lõi IP HQC-128, quá trình thực nghiệm được thiết kế dựa trên cơ chế đối chiếu nghiêm ngặt khóa chia sẻ. Cụ thể, trong mỗi chu trình kiểm thử độc lập, chương trình điều khiển sẽ kích hoạt module Đóng gói khóa trên phần cứng để tạo ra một bản mã cùng với một chuỗi khóa chia sẻ ban đầu. Ngay sau đó, bản mã này được chuyển tiếp vào module Mở gói khóa để tiến hành tính toán và trích xuất ra một chuỗi khóa chia sẻ thứ hai. Phép thử được hệ thống ghi nhận là thành công chỉ khi và chỉ khi hai chuỗi dữ liệu khóa này trùng khớp với nhau đến từng bit.

Trải qua quá trình kiểm thử tự động với 10.000 bộ dữ liệu đầu vào được sinh ngẫu nhiên. Dữ liệu thống kê từ quá trình thực nghiệm cho thấy tỷ lệ thành công của quá trình mã hóa và giải mã đạt mức tuyệt đối 100%. Toàn bộ 10.000 trường hợp đều vượt qua bài kiểm tra mà không ghi nhận bất kỳ một lỗi sai lệch dữ liệu nào. Sự trùng khớp hoàn hảo của 10.000 cặp khóa chia sẻ trong suốt hơn 1700 giây hoạt động liên tục là minh chứng thực nghiệm rõ ràng nhất, khẳng định kiến trúc phần cứng của đồ án đã tuân thủ chặt chẽ và xử lý hoàn toàn chính xác theo đúng

các tiêu chuẩn toán học của thuật toán mật mã hậu lượng tử HQC.

Hiệu suất về mặt thời gian là một trong những chỉ số quan trọng nhất để đánh giá năng lực thực tế của một bộ tăng tốc phần cứng. Trong đợt thực nghiệm với quy mô 10.000 mẫu thử, hệ thống đã hoàn thành toàn bộ khối lượng công việc trong tổng thời gian là 1722.0 giây. Kết quả tính toán trung bình cho thấy mỗi chu trình kiểm thử (bao gồm đầy đủ các giai đoạn từ đóng gói đến mở gói khóa) đạt mức 0.172 giây cho mỗi trường hợp.

Cần lưu ý rằng, con số này không chỉ đơn thuần phản ánh thời gian xử lý thuần túy của các module logic bên trong vùng PL, mà còn bao gồm cả độ trễ phát sinh từ quá trình tương tác hệ thống. Cụ thể, tổng thời gian thực thi chứa đựng cả phần hao phí (overhead) do việc điều phối của hệ điều hành chạy trên nhân xử lý ARM (PS) và các bước chuyển đổi dữ liệu thông qua framework PYNQ khi giao tiếp giữa hai vùng PS và PL. Việc đạt được mốc thời gian này trên một thiết bị biên như Kria KV260 cho thấy sự tối ưu đáng kể trong việc thiết kế luồng dữ liệu và tận dụng băng thông hệ thống.

Diễn giải kết quả dưới góc độ Decryption Failure Rate (DFR): Thuật toán HQC, với bản chất là một KEM dựa trên mã (*code-based KEM*), tồn tại một xác suất giải mã thất bại (DFR) nội tại không thể bằng không theo lý thuyết, đây là đặc tính cấu trúc của mọi sơ đồ mật mã dựa trên mã sửa lỗi. DFR lý thuyết của HQC-128 được chứng minh chặn trên bởi $2^{-128} \approx 2.94 \times 10^{-39}$ trong tài liệu đặc tả chính thức [28].

Với việc không ghi nhận trường hợp giải mã thất bại nào trong toàn bộ $N = 10.000$ phép thử, áp dụng phương pháp khoảng tin cậy Clopper-Pearson một phía với độ tin cậy 95%, biên trên thực nghiệm của DFR được xác định theo công thức:

$$\widehat{DFR}_{upper} = 1 - (1 - 0,95)^{1/N} = 1 - 0,05^{1/10000} \approx 2,995 \times 10^{-4}$$

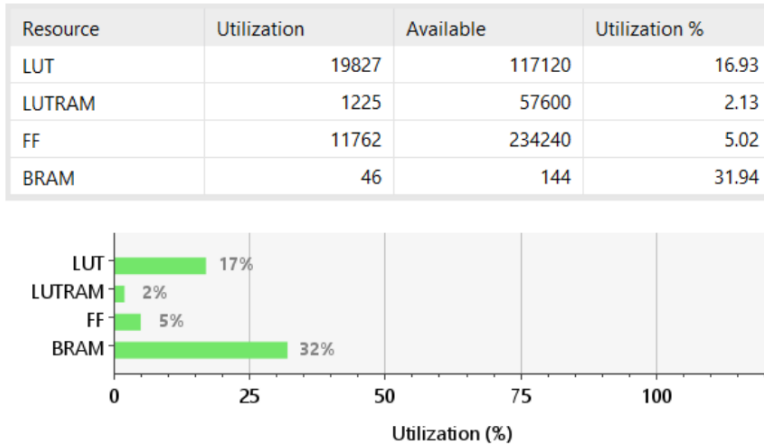
Cận thực nghiệm này hoàn toàn nhất quán với cận lý thuyết 2^{-128} . Việc kiểm chứng được DFR theo phương pháp tần suất đến mức 2^{-128} là bất khả thi về mặt thực hành (đòi hỏi $N > 2^{128}$ phép thử), do đó kết quả thực nghiệm với $N = 10.000$ đã đạt mức chứng minh phù hợp với chuẩn mực đánh giá KEM dựa trên mã trong cộng đồng học thuật. Kết quả không có phép giải mã thất bại nào trong 10.000 lần thử nghiệm là bằng chứng thực nghiệm đủ vững chắc khẳng định kiến trúc phần cứng của đồ án đã hiện thực hóa chính xác thuật toán mật mã hậu lượng tử HQC theo đặc tả chuẩn.

Bên cạnh kiểm tra trùng khớp khóa chia sẻ, độ phân tán của các giá trị

shared secret cũng được khảo sát: toàn bộ 10.000 chuỗi *shared secret* 512-bit đầu ra đều có giá trị duy nhất, không xảy ra hiện tượng trùng lặp. Đây là chỉ dấu thực nghiệm cho tính ngẫu nhiên (*entropy*) cao của *output*, củng cố thêm tính đúng đắn của khâu sinh số ngẫu nhiên trong thiết kế.

5.2.3. ĐÁNH GIÁ TÀI NGUYÊN PHẦN CỨNG

Hình 5.1 trình bày kết quả tổng hợp tài nguyên phần cứng của thiết kế HQC-128 thu được sau bước *implementation* hoàn chỉnh trên Vivado, với thiết bị mục tiêu là Xilinx Zynq UltraScale+ XCK26 (Kria KV260):



Hình 5.1: Tài nguyên phần cứng sử dụng (*post-implementation*)

Với mức chiếm dụng 16.93% LUT của thiết bị, thiết kế còn dư địa đáng kể cho việc tích hợp thêm các *module* mật mã phụ trợ (chẳng hạn AES, ASCON cho *symmetric encryption*) hoặc mở rộng sang các mức bảo mật cao hơn HQC-192 và HQC-256 trên cùng một SoC.

5.2.4. ĐÁNH GIÁ TẦN SỐ HOẠT ĐỘNG TỐI ĐA

Tần số hoạt động tối đa của thiết kế được xác định thông qua phương pháp tìm kiếm nhị phân (*binary search*) trên ràng buộc thời gian trong Vivado: ràng buộc tần số được tăng dần đến điểm mà *worst negative slack* (WNS) chuyển sang giá trị âm. Tần số tối đa $F_{\max}$ được suy ra từ *slack* thu được tại ràng buộc chu kỳ T_{target} gần nhất:

$$F_{\max} = \frac{1}{T_{\text{target}} - \text{WNS}} \quad (5.5)$$

Kết quả thu được trên thiết kế: $F_{\max} = 109.82$ MHz, với $\text{WNS} = 0.894$ ns

tại ràng buộc $T_{\text{target}} = 10$ ns. Tần số vận hành được chọn cho thiết kế là 100 MHz, lấy thấp hơn F_{max} một khoảng an toàn để đảm bảo tính tin cậy trong các điều kiện vận hành thực tế (biến thiên nhiệt độ, biến thiên điện áp nguồn).

5.2.5. ĐÁNH GIÁ THỜI GIAN THỰC THI

ĐO LƯỜNG ĐỘ TRỄ END-TO-END QUA FRAMEWORK PYNQ

Phép đo độ trễ ở cấp độ ứng dụng được thực hiện trong môi trường PYNQ, sử dụng hàm `time.time()` của Python để bao trọn một chu trình kiểm thử hoàn chỉnh: sinh seed, thực thi Keygen, thực thi Encap, thực thi Decap và kiểm chứng đối chiếu khóa chia sẻ. Phép đo này phản ánh thời gian đáp ứng thực tế mà ứng dụng phía hệ điều hành cảm nhận được, bao gồm cả thời gian tính toán thuần túy của vùng PL lẫn các chi phí phụ trợ phát sinh từ giao tiếp giữa nhân ARM và Programmable Logic.

Bảng 5.1 tổng kết phân bố thống kê của 10.000 phép đo thu được trên bo mạch KV260:

Bảng 5.1: Phân bố độ trễ end-to-end của một chu trình KEM hoàn chỉnh

Chỉ số	Giá trị
Số mẫu (N)	10.000
Trung vị (Median)	0.172 s
Trung bình (Mean)	0.172 s
Cực tiểu (Min)	0.169 s
Cực đại (Max)	0.184 s
Độ lệch chuẩn (Stddev)	1.3 ms
Tứ phân vị Q_1	0.171 s
Tứ phân vị Q_3	0.173 s
Tổng thời gian	1720.39 s

Độ lệch chuẩn ở mức 1.3 ms (chiếm dưới 0.8% trung vị) cho thấy thời gian thực thi *end-to-end* giữa các phép thử là rất ổn định. Quan trọng hơn, biến thiên nhỏ này không phát sinh từ phần cứng, vùng PL được thiết kế hoàn toàn theo nguyên tắc *constant-time* để chống lại các tấn công kênh kề (*side-channel timing attack*), mà có nguồn gốc từ *jitter* của hệ điều hành Linux đang vận hành trên PS, độ trễ ngẫu nhiên của bộ điều

phối tiến trình, hoạt động của trình thu gom rác Python và biến thiên của bộ nhớ đệm CPU. Tính bất biến của phần cứng được kiểm chứng độc lập trong Mục sau thông qua phép đếm chu kỳ ở cấp RTL.

ĐO LƯỜNG THỜI GIAN TÍNH TOÁN THUẦN PHẦN CỨNG

Để tách bạch hiệu năng kiến trúc phần cứng khỏi các chi phí phụ trợ ở tầng phần mềm, một bộ đếm chu kỳ chính xác được tích hợp trong *testbench* mô phỏng. Bộ đếm này được kích hoạt ngay trước xung *start* và bị vô hiệu hóa khi phát hiện cờ *done*, đo trực tiếp số chu kỳ xung nhịp mà lõi phần cứng cần để hoàn thành mỗi giai đoạn của thuật toán.

Bảng 5.2 báo cáo số chu kỳ chính xác đo được cho ba giai đoạn của HQC-128 của thiết kế:

Bảng 5.2: Số chu kỳ và thời gian tính toán thuần phần cứng

Giai đoạn	Số chu kỳ (cycles)	Thời gian @ 100 MHz (μ s)	Thời gian @ $F_{\max}$ (μ s)
Keygen	14006	140.06	127.54
Encap	30979	309.79	282.09
Decap	45753	457.53	416.62
Tổng (K+E+D)	90738	907.38	826.24

Tính *constant-time* của thiết kế được kiểm chứng bằng cách chạy *testbench* với N *seed* khác nhau (bao gồm các trường hợp biên như *seed* toàn 0, *seed* toàn 1, và các *seed* ngẫu nhiên): số chu kỳ thu được là hằng số đối với mọi đầu vào, khẳng định không tồn tại phụ thuộc dữ liệu trong đường tính toán, đây là một thuộc tính bắt buộc đối với mật mã phần cứng nhằm vô hiệu hóa các tấn công kênh kề dựa trên thời gian.

5.2.6. ĐÁNH GIÁ THROUGHPUT, $FREQ/CCS$ VÀ $A \times T$

Từ các đại lượng cơ sở đã thu được, các chỉ số hiệu năng tổng hợp được tính toán theo các công thức (5.1), (5.2), (5.3) đã định nghĩa ở Mục 5.1. Đối với chỉ số *Throughput* dạng Mbps, kích thước dữ liệu tham chiếu được chọn là kích thước bản mã HQC-128 ($N_{\text{Byte}} = 4.481 \text{ byte} = 35.848 \text{ bit}$) để đảm bảo tính nhất quán giữa các giai đoạn. Kết quả được tổng kết trong Bảng 5.3:

Bảng 5.3: Các chỉ số hiệu năng tổng hợp của thiết kế HQC-128

Chỉ số	Keygen	Encap	Decap
<i>Throughput</i> (op/s)	7.840	3.545	2.400
<i>Throughput</i> (Mbps)	281.05	127.07	86.04
Freq/CCs (MHz/kCCs)	7.84	3.55	2.40
$A \times T$ (LUTs· μ s)	2.53×10^6	5.59×10^6	8.26×10^6

Phân tích các chỉ số trong Bảng 5.3 cho phép định lượng nhiều khía cạnh của hiệu năng thiết kế:

5

- **Về *Throughput*:** Giai đoạn Keygen đạt thông lượng cao nhất (7.840 phép toán/giây) do số chu kỳ tính toán thấp nhất, trong khi Decap có thông lượng thấp hơn khoảng 3.27 lần (2.400 phép toán/giây). Kết quả này phản ánh đúng đặc tính của HQC: giai đoạn giải mã đòi hỏi nhiều phép tính phức tạp hơn do bao gồm các bước giải mã Reed-Muller và Reed-Solomon so với hai giai đoạn còn lại.
- **Về Freq/CCs:** Chỉ số này phản ánh hiệu quả kiến trúc trên mỗi đơn vị xung nhịp, giá trị càng cao càng chứng tỏ thiết kế tận dụng tốt mỗi chu kỳ xung. Giá trị 7.84 MHz/kCCs cho Keygen, 3.55 MHz/kCCs cho Encap và 2.40 MHz/kCCs cho Decap phản ánh trung thực sự chênh lệch về độ phức tạp tính toán giữa ba giai đoạn.
- **Về $A \times T$:** Chỉ số tích diện tích – thời gian thể hiện sự cân đối giữa chi phí phần cứng và tốc độ thực thi. Giá trị $A \times T$ cho cả ba giai đoạn nằm trong khoảng từ 2.53×10^6 đến 8.26×10^6 LUTs· μ s, phản ánh đặc trưng của thiết kế phiên bản là ưu tiên gọn nhẹ về tài nguyên (chỉ chiếm 16.93% LUT của thiết bị).

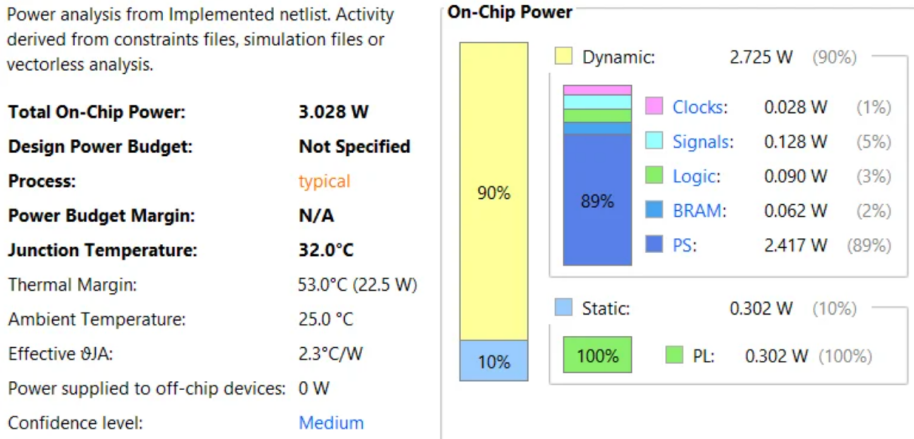
Cả ba chỉ số tổng hợp trên sẽ được sử dụng làm cơ sở so sánh khách quan với các công trình *hardware* HQC tham chiếu trong *tài liệu* ở Mục 5.3, đặc biệt với các thiết kế trên cùng họ FPGA Xilinx (Artix-7, UltraScale+) để đảm bảo tính *công bằng so sánh*.

5.2.7. ĐÁNH GIÁ CÔNG SUẤT VÀ NĂNG LƯỢNG TIÊU THỤ

Sau khi đã có đầy đủ các thông số về tài nguyên (Mục 5.2.3), tần số (Mục 5.2.4) và thời gian thực thi (Mục 5.2.5), phần này tiến hành đánh giá đặc tính tiêu thụ điện năng của thiết kế, đây là chỉ số then chốt đối

với các kịch bản triển khai mật mã hậu lượng tử trên thiết bị biên hoạt động bằng pin.

Phương pháp ước lượng công suất. Công suất tiêu thụ được trích xuất từ Vivado Power Analyzer trên *netlist* sau bước *implementation* hoàn chỉnh. Nhiệt độ chip hoạt động (*junction temperature*) được xác định ở mức 32.0°C trong điều kiện môi trường 25.0°C, với biên độ nhiệt an toàn 53.0°C tương đương 22.5 W, chứng tỏ thiết kế vận hành trong vùng nhiệt rất thuận lợi, không có rủi ro quá nhiệt. Mức độ tin cậy của phép ước lượng (*Confidence Level*) được công cụ đánh giá ở mức *Medium*, phù hợp với bước phân tích sơ bộ ở giai đoạn *implementation*.



Hình 5.2: Báo cáo công suất từ Vivado Power Analyzer cho thiết kế HQC-128 trên Kria KV260

Phân rã công suất tổng thể. Tổng công suất *on-chip* của toàn bộ SoC Zynq UltraScale+ (bao gồm cả Processing System và Programmable Logic) được báo cáo là 3.028 W, trong đó thành phần *Dynamic* chiếm 2.725 W (90%) và *Static* chiếm 0.302 W (10%). Bảng 5.4 trình bày phân rã chi tiết:

Bảng 5.4: Phân rã công suất tiêu thụ trên Kria KV260

Thành phần	Công suất (W)	Tỷ lệ
<i>Dynamic Power</i>		
Processing System (ARM + <i>hard IP</i>)	2.417	89%
PL — Signals	0.128	5%
PL — Logic	0.090	3%
PL — BRAM	0.062	2%
PL — Clocks	0.028	1%
Tổng <i>Dynamic</i>	2.725	100%
<i>Static Power (toàn bộ PL)</i>	0.302	—
Tổng <i>on-chip (system-level)</i>	3.028	—

5

Tách bạch công suất hệ thống và công suất bộ tăng tốc. Một quan sát quan trọng từ Bảng 5.4 là khối Processing System đơn lẻ chiếm 2.417 W, tức 89% tổng *dynamic power*. Đây là công suất nội tại của khối ARM Cortex-A53 cùng các *hard IP* đi kèm, vốn không thuộc phạm vi thiết kế của bộ tăng tốc HQC mà thuộc về kiến trúc cố định của SoC. Toàn bộ logic của bộ tăng tốc HQC được hiện thực hóa trong vùng Programmable Logic, do đó công suất thuộc về thiết kế này được tính bằng tổng các thành phần PL:

$$P_{PL} = P_{\text{Signals}} + P_{\text{Logic}} + P_{\text{BRAM}} + P_{\text{Clocks}} + P_{\text{Static}} = 0.610 \text{ W} \quad (5.6)$$

Sự phân tách này có ý nghĩa thiết yếu khi đối sánh với các công trình *hardware* HQC tham chiếu khác, vốn báo cáo công suất ở cấp *accelerator* chứ không bao gồm CPU *host*. Do đó, mọi phép so sánh trong Mục 5.3 sẽ sử dụng giá trị $P_{PL} = 0.610 \text{ W}$ thay vì tổng *on-chip* 3.028 W để đảm bảo tính *công bằng so sánh* (fair comparison).

Năng lượng tiêu thụ trên mỗi phép toán. Áp dụng công thức (5.4) đã định nghĩa ở Mục 5.1, năng lượng tiêu thụ cho mỗi giai đoạn của HQC-128 được tính bằng tích của công suất PL với thời gian thực thi tại F_{max} từ Mục 5.2.5. Kết quả được thể hiện ở Bảng 5.5:

Bảng 5.5: Năng lượng tiêu thụ trên mỗi phép toán (tại $F_{\max} = 109.82$ MHz)

Giai đoạn	Power PL (W)	Time (μ s)	Energy (μ J/op)
Keygen	0.610	127.54	77.80
Encap	0.610	282.09	172.07
Decap	0.610	416.62	254.14
Tổng KEM (K+E+D)	0.610	826.24	504.01

Phân tích kết quả năng lượng. Năng lượng tổng thể 504.01 μ J cho một chu trình KEM hoàn chỉnh là một con số có ý nghĩa đặc biệt với các kịch bản triển khai trên thiết bị biên. Để hình dung trực quan: với một viên pin lithium dung lượng 1000 mAh ở mức 3.7 V (trữ năng 13.320 J), thiết kế có thể thực hiện khoảng 26.4 triệu phép trao đổi khóa HQC-128 trước khi cạn pin nếu chỉ tính riêng năng lượng cho *quy trình mã hóa*. Trong các ứng dụng IoT thực tế, thiết bị thường thực hiện *trao đổi khóa* ở tần suất rất thấp (mỗi vài giờ hoặc mỗi phiên kết nối), do đó năng lượng tiêu thụ cho mật mã chiếm phần rất nhỏ trong tổng năng lượng vận hành, chúng tôi tính khả thi của việc tích hợp PQC vào các thiết bị đầu cuối có ràng buộc năng lượng.

Thành phần *Static Power* (0.302 W, tương đương 49.5% công suất PL) chiếm tỷ trọng đáng kể trong tổng công suất *accelerator*. Đây là đặc tính cố hữu của công nghệ silicon 16nm trên UltraScale+, không phụ thuộc vào hoạt động logic. Với các kịch bản triển khai thực tế trong đó *accelerator* không hoạt động liên tục, kỹ thuật *power gating* hoặc cấu hình lại *bitstream* khi không sử dụng có thể giảm đáng kể *static power*, mở ra hướng tối ưu năng lượng trong tương lai.

Các chỉ số công suất $P_{PL} = 0.610$ W và năng lượng $E_{KEM} = 504.01$ μ J/op sẽ được sử dụng trong Mục 5.3 để đối sánh với các thiết kế *hardware* HQC khác trong *literature*, đặc biệt là công trình của Schöffel et al. (*Best Paper Award, IEEE WF-IoT 2022*), đây là một trong số ít các công trình *hardware* HQC có báo cáo đầy đủ chỉ số năng lượng.

5.3. SO SÁNH HIỆU NĂNG

Nội dung phần 5.3 đối sánh hiệu năng từ thiết kế của đồ án với các công trình tiêu biểu trong *các tài liệu nghiên cứu* (literature) về HQC, theo hai khía cạnh: so với *phần mềm* (software) trên các nền tảng đa dạng (Mục 5.3.1) và so với các thiết kế *phần cứng* (hardware) khác (Mục 5.3.2). Để đảm bảo tính *công bằng* giữa các kết quả thu được trên những nền tảng có tần số xung nhịp khác nhau, đại lượng được dùng làm trục so sánh chính là số chu kỳ xung nhịp (*cycle count*), đây là một *tiêu chí đánh giá* (metric) độc lập với tần số, phản ánh chính xác hiệu quả tính toán của kiến trúc. Bên cạnh đó, các *tiêu chí đánh giá* phái sinh như *Time*, *Throughput* và *Freq/CCs* cũng được trình bày để cung cấp cái nhìn đa chiều.

5.3.1. PHẦN MỀM

5

Lựa chọn các công trình tham chiếu. Để đối sánh với *phần mềm*, ba công trình tiêu biểu được lựa chọn, mỗi công trình đại diện cho một phân khúc nền tảng phần mềm khác nhau, qua đó cho phép định vị thiết kế phần cứng của đồ án trong một phổ rộng các kịch bản triển khai HQC:

- **Chen et al., TCHES 2026 [29]:** đại diện cho phân khúc *tính toán hiệu năng cao*, công trình tối ưu HQC trên các nền tảng máy chủ và máy tính để bàn, bao gồm AMD EPYC 9754 với tập lệnh AVX2 và *Galois Field New Instructions* (GFNI), Apple M1, và ARM Cortex-A72 trên Raspberry Pi 4. Đây là công trình mới nhất, áp dụng kỹ thuật *additive FFT* kết hợp *Chinese Remainder Theorem* để tăng tốc phép nhân đa thức trong vành $\mathbb{F}_2[x]/(x^n + 1)$.
- **Jang et al., ePrint 2025 [30]:** đại diện cho *hệ thống nhúng phân khúc cao*, công trình tối ưu HQC trên ARM Cortex-M4, đây là vi điều khiển 32-bit được NIST chính thức khuyến nghị làm nền tảng tham chiếu cho đánh giá hiệu năng PQC trên thiết bị có ràng buộc tài nguyên.
- **Kim et al., ICT Express 2025 [31]:** đại diện cho *hệ thống nhúng phân khúc tiêu chuẩn*, công trình thuộc nhóm Đại học Kookmin (Hàn Quốc) tối ưu HQC trên ARM Cortex-M4 với cách tiếp cận sử dụng tổ hợp Toom-Cook 3-way kết hợp Karatsuba 2-way.

Phương pháp đối sánh. Tất cả các công trình tham chiếu đều báo cáo số chu kỳ xung nhịp cho ba phép toán cơ bản của HQC-128: Keygen,

Encap, và Decap. Đối với phần cứng của đồ án, *cycle count* được lấy từ kết quả đo trong Mục 5.2.5. Bảng 5.6 tổng hợp kết quả đối sánh:

Bảng 5.6: Đối sánh hiệu năng HQC-128 với các software implementations tiêu biểu (chu kỳ)

Công trình	Nền tảng	Keygen	Encap	Decap	Tổng K+E+D
Đồ án này	Kria KV260 FPGA	14.006	30.979	45.753	90.738
Chen et al. [29]	EPYC 9754 (AVX2+PCLMUL)	117.000	139.000	292.000	548.000
Chen et al. [29]	EPYC 9754 (AVX2+GFNI)	87.000	110.000	245.000	442.000
Chen et al. [29]	Apple M1 (ARM NEON)	93.000	125.000	259.000	477.000
Chen et al. [29]	Raspberry Pi 4 (Cortex-A72)	547.000	568.000	1.029.000	2.144.000
Jang et al. [30]	Cortex-M4 (ePrint 2025)	3.065.459	5.942.521	10.175.173	19.183.153
Kim et al. [31]	Cortex-M4 (ASM)	4.100.000	8.380.000	13.780.000	26.260.000

Tỷ lệ cải thiện (*speedup*) so với từng nền tảng phần mềm. Bảng 5.7 trình bày tỷ lệ cải thiện *cycle count* của thiết kế phần cứng so với mỗi công trình tham chiếu, tính theo công thức $speedup = cycles_{software} / cycles_{hardware}$:

5

Bảng 5.7: Tỷ lệ cải thiện hiệu năng của thiết kế phần cứng so với các software implementation

Nền tảng tham chiếu	Keygen	Encap	Decap	Tổng K+E+D
AMD EPYC 9754 (PCLMUL) [29]	8.35×	4.49×	6.38×	6.04×
AMD EPYC 9754 (GFNI) [29]	6.21×	3.55×	5.35×	4.87×
Apple M1 (ARM NEON) [29]	6.64×	4.03×	5.66×	5.26×
Raspberry Pi 4 (Cortex-A72) [29]	39.06×	18.34×	22.49×	23.63×
NUCLEO Cortex-M4 [30]	218.86×	191.83×	222.40×	211.41×
NUCLEO Cortex-M4 ASM [31]	292.73×	270.51×	301.18×	289.40×

Phân tích kết quả đối sánh với *software*. Các con số trong Bảng 5.6 và 5.7 cho thấy:

- **Đối với hệ thống nhúng phân khúc tiêu chuẩn (Cortex-M4) [30; 31]:** Tốc độ tăng từ 191× đến 301× tùy phép toán, tức cải thiện gần ba bậc về độ lớn. Mức *speedup* này khẳng định ý nghĩa thực tiễn của việc tích hợp HQC *accelerator* vào các ứng dụng IoT, *smart card* nơi thực thi PQC bằng *software* thuần không khả thi.
- **Đối với hệ thống nhúng phân khúc cao (Cortex-A72) [29]:** *Speedup*

đạt khoảng $18\times$ đến $39\times$. Kết quả này cho thấy tính khả thi của việc tích hợp bộ tăng tốc vào các thiết bị *edge gateway* hiệu năng trung bình.

- **Đối với phân khúc *tính toán hiệu năng cao* (AMD EPYC, Apple M1) [29]:** *Speedup* ở mức $3.5\times$ đến $8.4\times$. Dù các nền tảng này có tần số cao gấp 25-45 lần thiết kế phần cứng, việc hardware vẫn vượt trội về *cycle count* khẳng định hiệu quả tối ưu của kiến trúc tăng tốc chuyên dụng.

Kết luận về vị thế trong *landscape software*. Thiết kế phần cứng HQC của đồ án thể hiện hiệu năng vượt trội trên toàn bộ phổ nền tảng *software*, đặc biệt với phân khúc *nhúng*, đây vốn là *use case* mục tiêu của đồ án, với *speedup* trung bình đạt khoảng $200\times$.

5

5.3.2. PHẦN CỨNG

Phần này đối sánh thiết kế của đồ án với ba công trình hardware HQC tiêu biểu, mỗi công trình đại diện cho một triết lý thiết kế khác nhau. Do cả ba đều triển khai trên FPGA Artix-7 (công nghệ 28nm) trong khi đồ án dùng Kria KV260 với SoC Zynq UltraScale+ (công nghệ 16nm), số chu kỳ xung nhịp (*cycle count*) được dùng làm trục so sánh chính vì độc lập với công nghệ silicon và tần số vận hành, kèm theo phân tích đa chiều về tài nguyên và trade-off kiến trúc.

ĐỐI SÁNH VỚI ANTOGNAZZA ET AL. (IEEE TRANSACTIONS ON COMPUTERS 2025) [5]

Phương pháp của công trình tham chiếu. Antognazza et al. trình bày một kiến trúc unified RTL hỗ trợ đồng thời cả ba mức bảo mật HQC-128/192/256 với khả năng chọn parameter set tại runtime. Để đạt hiệu năng cao, thiết kế của họ áp dụng nhiều kỹ thuật tối ưu song song hóa ở mức kiến trúc:

- Đối với bộ nhân đa thức, họ sử dụng phương pháp shift-and-add được song song hóa: thiết kế cải thiện độ trễ phép nhân theo hệ số l bằng cách xử lý l chỉ số sparse song song, đổi tài nguyên phần cứng lấy tốc độ.
- Đối với bộ giải mã Reed-Solomon, họ triển khai thuật toán Enhanced Parallel Inversionless Berlekamp-Massey (ePIBMA) với $2t_{\max}+1$ Processing Element được khởi tạo, cùng Enhanced Chien Search and Error Evaluation (eCSEE) thực thi song song hai bước tìm vị trí lỗi và giá trị lỗi.

Phương pháp của đồ án. Thiết kế của đồ án là phiên bản ưu tiên tối thiểu hóa tài nguyên logic và tính đơn giản của kiến trúc. Toàn bộ ba phase Keygen, Encap, Decap được hiện thực hóa trên một Overlay duy nhất tái sử dụng được, với đường dữ liệu được thiết kế gọn nhẹ thay vì nhân bản nhiều đơn vị tính toán song song. Cách tiếp cận này chấp nhận số chu kỳ cao hơn để đổi lấy footprint logic nhỏ và khả năng tái sử dụng linh hoạt giữa các phase.

Phân tích nguyên nhân chênh lệch cycle. Bảng 5.8 cho thấy đồ án có cycle count cao hơn Antognazza et al. khoảng $2.5\times$ ở cả ba phase (Keygen $2.51\times$, Encap $2.64\times$, Decap $2.50\times$). Chênh lệch này bắt nguồn trực tiếp từ ba khác biệt kiến trúc:

- **Mức độ song song của bộ nhân đa thức:** Antognazza et al. xử lý nhiều chỉ số sparse song song, giảm độ trễ phép nhân theo hệ số song song, trong khi thiết kế của đồ án xử lý tuần tự để tiết kiệm read port và logic điều khiển.
- **Độ phức tạp của bộ giải mã:** các thuật toán ePIBMA và eCSEE của họ song song hóa quá trình giải mã Reed-Solomon, giảm số chu kỳ đáng kể nhưng đòi hỏi nhiều Processing Element chạy đồng thời.
- **Mức độ trưởng thành thiết kế:** công trình của họ kế thừa kinh nghiệm tối ưu nhiều năm của nhóm tác giả về polynomial multiplier và RS decoder, vốn nằm ngoài phạm vi của một đồ án tốt nghiệp.

5

Bảng 5.8: Phân tích trade-off đa chiều giữa đồ án và Antognazza et al. [5] (HQC-128)

Metric	Đồ án này	Antognazza et al.	Nhận xét
Platform	KV260 (UltraScale+, 16nm)	Artix-7 (28nm)	Khác thể hệ FPGA
LUT	19,827	26,561	Đồ án ít hơn 25.4%
FF	11,762	13,636	Đồ án ít hơn 13.7%
BRAM	46	28	Đồ án nhiều hơn
DSP	0	0	Bằng nhau
Fmax (MHz)	109.82	143	Antognazza cao hơn
Keygen cycles	14,006	5,577	Antognazza thấp hơn $2.51\times$
Encap cycles	30,979	11,726	Antognazza thấp hơn $2.64\times$
Decap cycles	45,753	18,304	Antognazza thấp hơn $2.50\times$
Tổng cycles K+E+D	90,738	35,607	Antognazza thấp hơn $2.55\times$

Bảng 5.8 cho thấy, đây là một trade-off area-time. Về phía ưu thế của đồ án: thiết kế tiêu thụ ít LUT hơn 25.4% (19,827 so với 26,561) và ít FF hơn 13.7% (11,762 so với 13,636), thể hiện footprint logic gọn gàng hơn, đây một lợi thế cho các kịch bản triển khai trên thiết bị có ràng buộc về diện tích logic.

Tuy nhiên, công trình tham chiếu này vẫn vượt trội ở hai khía cạnh. Thứ nhất, về số chu kỳ, họ thấp hơn 2.5 lần ở mọi phase nhờ mức độ song song hóa cao. Thứ hai, đồ án tiêu thụ nhiều BRAM hơn (46 so với 28 khối), điều này phản ánh chiến lược lưu trữ trung gian của thiết kế, một điểm có thể tối ưu trong tương lai. Xét theo chỉ số $\text{Area} \times \text{Time product}$ tổng hợp, lợi thế tốc độ $2.5 \times$ của công trình tham chiếu lớn hơn phần diện tích logic mà đồ án tiết kiệm được, nên họ vẫn hiệu quả hơn về tích diện tích-thời gian.

ĐỐI SÁNH VỚI AGUILAR-MELCHOR ET AL. (HLS-HQC, 2022) [6]

Phương pháp của công trình tham chiếu. HLS-HQC sử dụng phương pháp High-Level Synthesis, chuyển tự động code C/C++ sang RTL thông qua công cụ tổng hợp. Mục tiêu của họ là rapid prototyping, chính các tác giả thừa nhận rằng handcrafted RTL có thể vượt trội về số chu kỳ xung nhịp, nhưng với cái giá là thời gian và công sức thiết kế tăng lên.

Phương pháp của đồ án. Đồ án sử dụng handcrafted RTL viết tay hoàn toàn bằng Verilog, cho phép tối ưu sâu ở mức kiến trúc dataflow và pipeline mà công cụ HLS khó đạt được.

Phân tích so sánh. Thiết kế của đồ án vượt trội rõ rệt về cycle: speedup $2.89 \times$ ở Keygen, $2.88 \times$ ở Encap, và đặc biệt $4.22 \times$ ở Decap.

Bảng 5.9: Đối sánh đồ án với Aguilar-Melchor et al. [6] (HQC-128)

Metric	Đồ án này	HLS-HQC perf	Nhận xét
Platform	KV260 (UltraScale+, 16nm)	Artix-7 (28nm)	Khác thể hệ FPGA
LUT	19,827	11,484	HLS ít hơn
FF	11,762	8,798	HLS ít hơn
BRAM	46	6	HLS ít hơn nhiều
DSP	0	0	Bằng nhau
Fmax (MHz)	109.82	150	HLS cao hơn
Keygen cycles	14,006	40,427	Đồ án thấp hơn $2.89 \times$
Encap cycles	30,979	89,110	Đồ án thấp hơn $2.88 \times$
Decap cycles	45,753	193,082	Đồ án thấp hơn $4.22 \times$

Đây là một trường hợp trade-off ngược chiều: HLS-HQC đạt footprint tài nguyên nhỏ hơn ở mọi loại (đặc biệt chỉ 6 BRAM so với 46 của đồ án) nhưng phải trả giá bằng cycle count cao hơn nhiều, pha Decap chậm hơn tới 4.22 lần. Điều này phản ánh đúng đặc tính của công cụ HLS: công cụ tổng hợp tự động ưu tiên tối thiểu hóa tài nguyên và tốc độ phát triển nhanh, chấp nhận đánh đổi hiệu năng tính toán so với RTL viết tay. Thiết kế handcrafted RTL của đồ án cho phép tối ưu sâu đường dữ liệu, đúng

như nhận định của chính các tác giả về ưu thế của RTL thủ công. Sự khác biệt này đặc biệt rõ ở pha Decap, đây là pha phức tạp nhất với cả giải mã Reed-Muller và Reed-Solomon cho nên Decap là nơi tối ưu thủ công ở mức RTL phát huy hiệu quả cao nhất. Như vậy, đồ án đánh đổi footprint tài nguyên lớn hơn để đạt hiệu năng tính toán vượt trội, một lựa chọn phù hợp với tài nguyên của KV260 khi chỉ chiếm 16.93% LUT của thiết bị.

ĐỐI SÁNH VỚI SCHÖFFEL ET AL. (IEEE WFIOT 2022, BEST PAPER AWARD) [3]

Phương pháp của công trình tham chiếu. Schöffel et al. trình bày thiết kế HW/SW co-design đầu tiên của HQC, triển khai trên cả ASIC (22nm) và FPGA (Artix-7). Kiến trúc của họ dựa trên một RISC-V core, trong đó phần lớn control logic và thuật toán cấp cao chạy bằng phần mềm C biên dịch với GCC mức tối ưu O2, chỉ các điểm nóng tính toán được tăng tốc bằng instruction set extension và loosely-coupled accelerator (R-Unit, Sampling-Unit, RM-Decoder). Phiên bản FPGA của họ dùng khoảng ~8,000 LUT, ~2,400 FF, 3 BRAM, F_{max} 100 MHz.

Phương pháp của đồ án. Đồ án triển khai toàn bộ thuật toán KEM được hiện thực hóa trong vùng Programmable Logic, không có phần nào chạy trên CPU phần mềm.

Phân tích so sánh. Thiết kế của đồ án vượt trội rất đáng kể, đặc biệt ở pha Decap với speedup 12.18×.

Bảng 5.10: Đối sánh đồ án với Schöffel et al. [3] (HQC-128)

Metric	Đồ án này	Schöffel FPGA	Nhận xét
Platform	KV260 (UltraScale+, 16nm)	Artix-7 (28nm)	Khác thể hệ FPGA
LUT	19,827	~8,000	Schöffel ít hơn nhiều
FF	11,762	~2,400	Schöffel ít hơn nhiều
BRAM	46	3	Schöffel ít hơn nhiều
DSP	0	0	Bằng nhau
Fmax (MHz)	109.82	100	Tương đương
Keygen cycles	14,006	56,000	Đồ án thấp hơn 4.00×
Encap cycles	30,979	131,000	Đồ án thấp hơn 4.23×
Decap cycles	45,753	557,000	Đồ án thấp hơn 12.18×

Từ bảng 5.10 cho thấy, Schöffel et al. đạt footprint tài nguyên nhỏ nhất trong ba công trình (khoảng ~8,000 LUT, 3 BRAM) nhưng cycle count cao nhất, đây là hệ quả trực tiếp của phương pháp HW/SW co-design. Vì phần lớn thuật toán chạy trên RISC-V phần mềm, thiết kế của họ tốn rất ít logic chuyên dụng nhưng phải gánh overhead của instruction fetch, decode, và memory access trên CPU. Khoảng cách cực lớn ở pha Decap

(12.18×) đặc biệt có ý nghĩa: Decap là pha nặng nhất với bước re-encryption theo biến đổi Fujisaki-Okamoto, nơi thực thi phần mềm trên RISC-V trở thành nút thắt nghiêm trọng. Thiết kế của đồ án đã loại bỏ hoàn toàn overhead CPU này, đổi lấy footprint logic lớn hơn.

ĐÁNH GIÁ VỊ THẾ TỔNG THỂ

Tổng hợp ba đối sánh trên cho thấy một bức tranh trade-off area-time nhất quán trong hệ sinh thái hardware HQC. Ba công trình tham chiếu trải trên một phổ rộng: từ Schöffel et al. với footprint nhỏ nhất nhưng cycle cao nhất (HW/SW co-design), qua HLS-HQC ở vùng tài nguyên thấp và cycle trung bình cao, đến Antognazza et al. với cycle thấp nhất nhưng đầu tư tài nguyên lớn nhất (unified RTL tối ưu chuyên sâu).

Thiết kế của đồ án định vị ở vùng cân bằng có chủ đích: vượt trội hai phương pháp HLS và HW/SW co-design về cycle count (đặc biệt ở pha Decap, lần lượt 4.22× và 12.18×), đồng thời duy trì footprint LUT và FF nhỏ hơn cả công trình unified RTL state-of-the-art. So với công trình unified RTL chuyên dụng, đồ án có cycle cao hơn 2.5×, đây là một khoảng cách hợp lý phản ánh đúng tương quan giữa một thiết kế tối ưu tài nguyên trong phạm vi đồ án và một thiết kế chuyên dụng tối ưu nhiều năm. Khi xét đến các giá trị bổ sung của đồ án như tái sử dụng Overlay duy nhất cho cả ba phase và tích hợp demo end-to-end mã hóa hình ảnh cho thấy kết quả này khẳng định tính khả thi và cạnh tranh của thiết kế trong bức tranh hardware HQC hiện hành.

6

KẾT LUẬN

Nghiên cứu này đã trình bày một thiết kế phần cứng toàn diện cho hệ mật mã hậu lượng tử Hamming Quasi-Cyclic trên nền tảng FPGA Xilinx Kria KV260. Thông qua phương pháp đồng thiết kế phần cứng/phần mềm (HW/SW co-design), các thành phần cốt lõi bao gồm bộ nhân đa thức mật độ thấp, các hàm băm cryptographic và các module mã hóa/giải mã đã được tối ưu hóa và tích hợp chặt chẽ thông qua giao tiếp điều khiển AXI. Kiến trúc đề xuất đã thực thi thành công các quy trình Tạo khóa (Key Generation), Đóng gói khóa (Encapsulation) và Giải gói khóa (Decapsulation), đóng góp một bước tiến quan trọng trong việc hiện thực hóa các hệ thống mật mã mã hóa dựa trên lý thuyết mã sửa lỗi (code-based cryptography) có hiệu suất cao và bảo mật.

Những đóng góp chính của công trình này không chỉ dừng lại ở việc triển khai thành công thuật toán HQC mà còn nằm ở việc đề xuất một khung thiết kế module hóa, tận dụng tối đa khả năng tính toán song song của FPGA hiện đại để giải quyết các thách thức về tài nguyên và thời gian thực thi trong môi trường thực tế.

6.1. ƯU ĐIỂM CỦA THIẾT KẾ ĐỀ XUẤT

Việc hiện thực thuật toán HQC trên nền tảng SoC-FPGA đã mang lại nhiều ưu điểm vượt trội:

- **Hiệu suất tính toán tối ưu:** Kiến trúc đề xuất cho thấy khả năng xử lý vượt trội, đặc biệt là trong giai đoạn nhân đa thức và băm dữ liệu một phần mà phần mềm thực hiện rất chậm.

- **Lỗi HQC IP tích hợp:** Cả ba chức năng KeyGen, Encapsulation và Decapsulation được tích hợp vào chung một IP duy nhất. Lỗi này sử dụng kiến trúc chia sẻ tài nguyên (Resource Sharing) với các bộ nhân đa thức (Polynomial Multiplication) và bộ băm SHAKE256 dùng chung để tiết kiệm diện tích FPGA.
- **Tính linh hoạt và Module hóa:** Thiết kế áp dụng cấu trúc phân lớp với các khối độc lập như bộ nhân đa thức, module SHAKE, KeyGen, Encap và Decap. Sự tách biệt này cho phép tái sử dụng hoặc tái cấu hình các thành phần cho các giao thức mật mã khác, tăng khả năng thích ứng với các tiêu chuẩn mật mã trong tương lai.
- **Khả năng ứng dụng cho hệ thống nhúng:** Việc triển khai thành công trên FPGA chứng minh tính khả thi của việc đưa các hệ mật mã dựa trên mã sửa lỗi vào các thiết bị biên (edge computing) và hệ thống nhúng có tài nguyên hạn chế nhưng yêu cầu mức độ bảo mật cao như IoT hay các hệ thống tự hành.

6

6.2. HẠN CHẾ CỦA THIẾT KẾ HIỆN TẠI

Mặc dù đạt được những kết quả khả quan, thiết kế vẫn tồn tại một số hạn chế cần được xem xét:

- **Băng thông giao tiếp:** Việc dựa vào giao thức AXI4 memory-mapped có thể tạo ra điểm nghẽn về băng thông khi xử lý các tập dữ liệu cực lớn hoặc trong các ứng dụng yêu cầu truyền tin tốc độ cao liên tục.
- **Hiệu năng tính toán còn khoảng cách so với state-of-the-art:** So với công trình unified RTL của Antognazza et al. [29], thiết kế của đồ án có cycle count cao hơn từ $2.50\times$ đến $2.64\times$ tùy giai đoạn. Khoảng cách này phản ánh việc thiết kế chưa áp dụng các kỹ thuật tối ưu cấp cao như Enhanced Parallel Inversionless Berlekamp-Massey Algorithm (ePIBMA) hay Enhanced Chien Search and Error Evaluation (eCSEE) cho khối Reed-Solomon decoder, đồng thời pipeline depth của bộ nhân đa thức và bộ băm SHAKE256 chưa được khai thác triệt để.

6.3. HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

Dựa trên những kết quả đạt được và các hạn chế còn tồn tại, nghiên cứu này đề xuất một số hướng phát triển tiềm năng nhằm nâng cao hiệu suất, tính bảo mật và khả năng ứng dụng của bộ tăng tốc HQC:

- **Tối ưu hóa sâu việc chia sẻ tài nguyên (Deep Resource Sharing):** Các nỗ lực trong tương lai nên tập trung vào việc áp dụng kỹ thuật ghép kênh theo thời gian (time-multiplexing) cho các đơn vị tính toán dùng chung giữa Poly Mul và SHAKE. Việc tận dụng tối đa các khối DSP và cấu trúc RAM phân tán sẽ giúp giảm dấu chân tài nguyên trên FPGA, cho phép triển khai hệ thống trên các dòng chip giá rẻ, công suất thấp phục vụ cho các ứng dụng IoT và Edge Computing.
- **Cải tiến cơ chế giao tiếp và truyền dữ liệu:** Chuyển đổi từ giao thức AXI4 Memory-Mapped sang AXI4-Stream kết hợp với bộ điều khiển DMA (Direct Memory Access) chuyên dụng. Sự thay đổi này sẽ loại bỏ các điểm nghẽn về băng thông trong quá trình truyền tải các vector đa thức lớn, đặc biệt quan trọng cho các ứng dụng yêu cầu xử lý dữ liệu quy mô lớn hoặc truyền tin tốc độ cao.
- **Chuyển đổi sang thiết kế ASIC:** Việc điều chỉnh thiết kế cho quy trình sản xuất vi mạch chuyên dụng (ASIC) sẽ mang lại những cải thiện đáng kể về diện tích và hiệu suất năng lượng. Quá trình này sẽ đòi hỏi việc tối ưu hóa lại các khối logic cốt lõi để đạt được sự cân bằng tối ưu giữa công suất tiêu thụ, tốc độ xử lý và chi phí sản xuất thương mại.

Các hướng nghiên cứu đề xuất ở trên nhằm giải quyết những hạn chế hiện tại, đồng thời tận dụng tối đa những điểm mạnh của thiết kế đã triển khai, đảm bảo tính phù hợp và tầm ảnh hưởng của nó trong lĩnh vực mật mã hậu lượng tử đang không ngừng phát triển.

Tóm lại, đồ án này đã chứng minh tính khả thi và hiệu quả của việc triển khai hệ mật mã hậu lượng tử HQC (Hamming Quasi-Cyclic) trên nền tảng FPGA, đạt được hiệu suất đáng kể và tính module hóa cao. Kiến trúc đề xuất đã thiết lập một nền tảng vững chắc cho các bước tiến xa hơn trong các hệ thống mật mã bảo mật hiệu suất cao. Bằng cách giải quyết các thách thức đã xác định và theo đuổi các hướng nghiên cứu đã vạch ra, công trình này có thể đóng góp đáng kể vào việc phát triển các giải pháp mật mã hậu lượng tử có khả năng mở rộng và bền vững cho các môi trường tính toán thế hệ tiếp theo.

TÀI LIỆU THAM KHẢO

- [1] M. Schöffel, J. Feldmann, and N. Wehn, “Efficient hardware implementation of constant time sampling for hqc,” University of Kaiserslautern-Landau, Technical Report, 2023.
- [2] C. Aguilar-Melchor, J.-C. Deneuville, A. Dion, J. Howe, R. Malmain, V. Migliore, M. Nawan, and K. Nawaz, “Towards automating cryptographic hardware implementations: A case study of hqc,” in *Proceedings of the International Workshop on Code-Based Cryptography (CBCrypto 2022)*, Trondheim, Norway, May 2022. [Online]. Available: <https://hal.science/hal-04116827v1>
- [3] M. Schöffel, J. Feldmann, and N. Wehn, “Code-based cryptography in iot: A hw/sw co-design of hqc,” in *8th IEEE World Forum on Internet of Things, WF-IoT 2022, Yokohama, Japan, October 26 - Nov. 11, 2022*. IEEE, 2022, pp. 1–7. [Online]. Available: <https://doi.org/10.1109/WF-IoT54382.2022.10152031>
- [4] S. Deshpande, C. Xu, M. Nawan, K. Nawaz, and J. Szefer, “Fast and efficient hardware implementation of hqc,” in *Selected Areas in Cryptography – SAC 2023*, C. Carlet, K. Mandal, and V. Rijmen, Eds. Springer Nature Switzerland, 2024.
- [5] F. Antognazza, A. Barengi, and G. Pelosi, “An efficient and unified RTL accelerator design for HQC-128, HQC-192, and HQC-256,” *IEEE Transactions on Computers*, vol. 74, no. 7, pp. 2306–2320, Jul 2025.
- [6] C. Aguilar-Melchor, J.-C. Deneuville, A. Dion, J. Howe, R. Malmain, V. Migliore, M. Nawan, and K. Nawaz, “Towards automating cryptographic hardware implementations: a case study of HQC,” in *Code-Based Cryptography Workshop (CBCrypto 2022)*, ser. LNCS, vol. 13839. Trondheim, Norway: Springer, May 29-30 2022, pp. 62–76. [Online]. Available: <https://eprint.iacr.org/2022/1425>
- [7] C. A. Melchor, N. Aragon, S. Bettaieb *et al.*, “Hamming quasi-cyclic (hqc) specification,” https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf, 2025, nIST Post-Quantum Cryptography Standardization Project.

- [8] O. Carter, “What is Public Key Cryptography? A Guide to Secure Encryption,” May 2024, [Online]. Available: https://dev.to/olivia_carter/what-is-public-key-cryptography-a-beginners-guide-to-secure-encryption-3hc1.
- [9] National Institute of Standards and Technology (NIST), “Post-quantum cryptography standardization,” 2024, available online: <https://csrc.nist.gov/projects/post-quantum-cryptography>.
- [10] D. J. Bernstein, J. Buchmann, and E. Dahmen, “Introduction to post-quantum cryptography,” *Post-Quantum Cryptography*, pp. 1–14, 2008.
- [11] C. A. Melchor *et al.*, “Hqc: A hamming quasi-cyclic code-based public key encryption scheme,” *NIST Post-Quantum Cryptography Standardization Submission*, 2019.
- [12] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.
- [13] E. R. Berlekamp, R. J. McEliece, and H. C. A. van Tilborg, “On the inherent intractability of certain coding problems,” *IEEE Transactions on Information Theory*, vol. 24, no. 3, pp. 384–386, 1978.
- [14] W. C. Huffman and V. Pless, in *Fundamentals of error-correcting codes*. Cambridge university press, 2010. [Online]. Available: <https://www.amazon.fr/Fundamentals-Error-Correcting-Codes-Cary-Huffman/dp/0521131707>
- [15] R. Misoczki, J.-P. Tillich, N. Sendrier, and P. S. L. M. Barreto, “Mdpc-mceliece: New mceliece variants from moderate density parity-check codes,” in *IEEE International Symposium on Information Theory (ISIT)*. IEEE, 2013, pp. 2069–2073. [Online]. Available: <https://eprint.iacr.org/2012/409.pdf>
- [16] National Institute of Standards and Technology (NIST), “Sha-3 standard: Permutation-based hash and extendable-output functions,” <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>, 2015, FIPS PUB 202.

- [17] G. Bertoni, J. Daemen, M. Peeters, and G. V. Assche, “The keccak reference,” <https://keccak.team/files/Keccak-reference-3.0.pdf>, 2011.
- [18] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “Sponge functions,” 2007.
- [19] N. Aragon, P. Gaborit, and G. Zémor, in *HQC-RMRS, an instantiation of the HQC encryption framework with a more efficient auxiliary error-correcting code*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.10741>
- [20] S. Lin and D. J. Costello, in *Error control coding, volume 2*. Prentice Hall Englewood Cliffs, 2004.
- [21] F. J. MacWilliams and N. J. A. Sloane, in *The theory of errorcorrecting codes, volume 16*. Elsevier, 1977.
- [22] S. Gao and T. Mateer, “Additive fast fourier transforms over finite fields,” in *IEEE Transactions on Information Theory*. IEEE, 2010.
- [23] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed. McGraw-Hill, 2013.
- [24] AMD Xilinx, *Kria K26 System-on-Module Data Sheet (DS987)*, 2023, v1.8. [Online]. Available: <https://docs.xilinx.com/r/en-US/ds987-k26-som>
- [25] AMD Xilinx, *Zynq UltraScale+ MPSoC Data Sheet: Overview (DS890)*, Xilinx, Inc., 2021, v3.6.
- [26] W. Wang, S. Tian, B. Jungk, N. Bindel, P. Longa, and J. Szefer, “Parameterized hardware accelerators for lattice-based cryptography and their application to the hw/sw co-design of qtesla,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 3, pp. 269–306, Jun 2020. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/8591>
- [27] M. P. G. V. A. Guido Bertoni, Joan Daemen, *Keccak and the SHA-3 Standardization*, 2013.
- [28] P. Gaborit, C. Aguilar-Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, E. Persichetti, G. Zémor, J. Bos, A. Dion,

- J. Lacan, J.-M. Robert, P. Véron, P. L. Barreto, S. Ghosh, S. Gueron, T. Güneysu, R. Misoczki, J. Richter-Brokmann, N. Sendrier, J.-P. Tillich, and V. Vasseur, “Hamming quasi-cyclic (HQC),” Submission to the NIST Post-Quantum Cryptography Standardization Project, August 2025.
- [29] M.-S. Chen, C.-M. Chiu, C.-T. Peng, and B.-Y. Yang, “Accelerating HQC with additive FFT,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2026, no. 2, pp. 520–544, Apr 2026. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/12898>
- [30] J. Jang, M. Lee, D. Kwon, S. Hong, S. Kim, and S. Lee, “Efficient polynomial multiplication for HQC on ARM Cortex-M4,” *Cryptology ePrint Archive*, Paper 2025/1939, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1939>
- [31] D. Kim, J. Choi, S. Yoon, and S. C. Seo, “Optimized implementation of HQC on Cortex-M4,” *ICT Express*, vol. 11, no. 5, pp. 939–944, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405959525001018>
- [32] N. Sendrier, in *Secure Sampling of Constant Weight Words – Application to BIKE*. *Cryptology ePrint Archive*, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1631>
- [33] F. Antognazza, A. Barengi, G. Pelosi, and R. Susella, in *A High Efficiency Hardware Design for the Post-Quantum KEM HQC*. IEEE, 2024, p. 431–441.
- [34] AMD Xilinx, *Kria KV260 Vision AI Starter Kit Data Sheet (DS986)*, 2022, v1.2. [Online]. Available: <https://docs.xilinx.com/r/en-US/ds986-kv260-starter-kit>
- [35] AMD Xilinx, *Kria KV260 Vision AI Starter Kit User Guide (UG1089)*, 2023, v1.3.